



## 第5章 S3C2440嵌入式系统

### 本章主要内容

- |                 |          |
|-----------------|----------|
| 1、S3C2440综述     | 5、DMA    |
| 2、S3C2440存储控制器  | 6、中断控制器  |
| 3、NAND FLASH控制器 | 7、I/O端口  |
| 4、时钟和功率管理       | 8、PWM定时器 |

# 本章教学要求

**理解：**

- 1、S3C2440的存储控制器；
- 2、NAND Flash控制器，
- 3、时钟和功率管理
- 4、DMA

**掌握：**

- 1、S3C2440的组成 及 各部分主要完成的功能
- 2、中断控制器等的工作原理及应用
- 3、I/O端口
- 4、PWM定时器

## 5.1 S3C2440综述

三星公司推出的 16/32位 RISC 微处理器 S3C2440A，为手持设备和一般类型应用提供了低价格、低功耗、高性能 小型微控制器 的解决方案。

S3C2440A 采用了 ARM920T 的内核，0.13um 的 CMOS 标准宏单元和存储器单元。特别适合于对 成本 和 功率 敏感型的应用。它采用了新的总线架构 AMBA。

Advanced Micro controller  
Bus Architecture

## 5.1 S3C2440综述

S3C2440A 的杰出的特点是：其核心处理器 (CPU) 是由 ARM 公司 设计的 16/32 位 ARM920T 的 RISC 处理器。ARM920T 实现了 MMU, AMBA BUS 和 Harvard 高速缓冲体系结构。这一结构具有独立的 16KB 指令Cache 和 16KB 数据Cache。每个都是由具有 8 字长 的行组成。通过提供一套完整的通用系统外设, S3C2440A 减少 整体系统成本 和 无需配置额外的组件。

## 5.1 S3C2440综述

S3C2440A 集成了以下片上功能（了解）：

- （1）1.2V 内核供电，1.8V/2.5V/3.3V存储器供电，3.3V 外部I/O供电，具备 16KB的I-Cache 和 16KB的D-Cache/MMU微处理器；
- （2）外部存储控制器（SDRAM 控制和片选逻辑）；
- （3）LCD 控制器（最大支持 4K色STN 和 256K色TFT）提供1通道LCD 专用DMA；
- （4）通道DMA并有外部请求引脚；



## 5.1 S3C2440综述

- (5) 3 通道UART（串行异步通信）  
(IrDA1.0, 64字节Tx FIFO, 64字节Rx FIFO)；
- (6) 2 通道SPI（串行外设接口）；
- (7) 1 通道IIC-BUS接口（串行总线）；
- (8) 1 通道IIS-BUS音频编解码器接口；
- (9) AC' 97（音频标准）解编码器接口；
- (10) 兼容SD主接口协议1.0版 和 MMC 卡协议2.11兼容版；

## 5.1 S3C2440综述

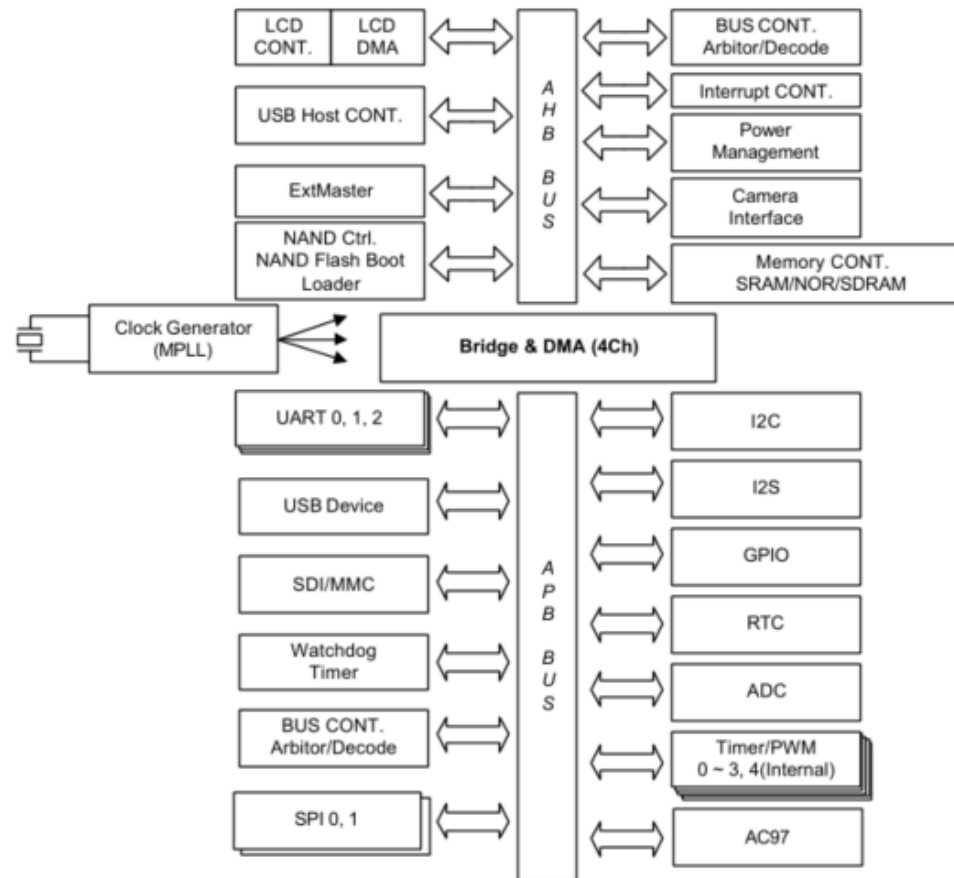
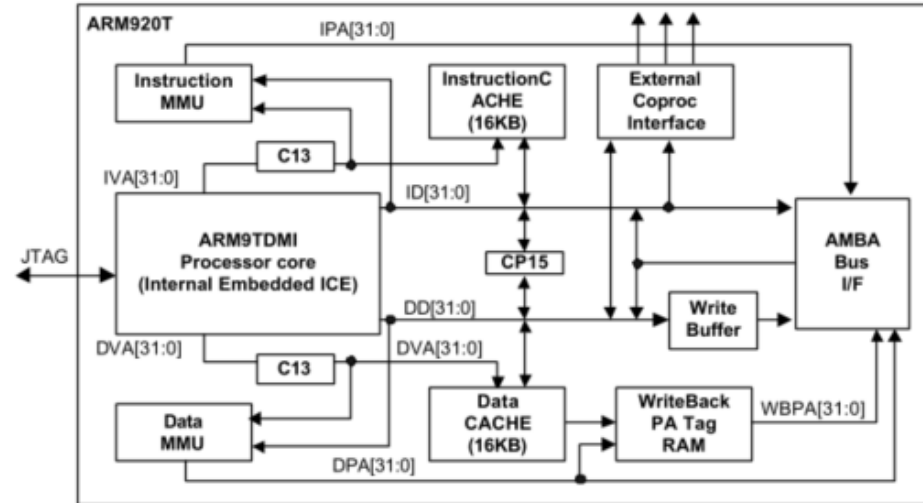
- (11) 2端口USB主机/1端口USB设备（1.1 版）；
- (12) 4通道PWM定时器 和 1通道内部定时器 / 看门狗定时器；
- (13) 8通道10比特ADC 和 触摸屏接口；
- (14) 具有日历功能的RTC；
- (15) 相机接口（最大4096 × 4096像素的投入支持。2048 × 2048像素的投入，支持缩放）；

## 6.1 S3C2440综述

- (16) 130个通用I/O口 和 24通道外部中断源；
- (17) 具有普通，慢速，空闲和掉电模式；
- (18) 具有PLL（锁相环）片上时钟发生器。

下图为 S3C2440 的内部结构图：



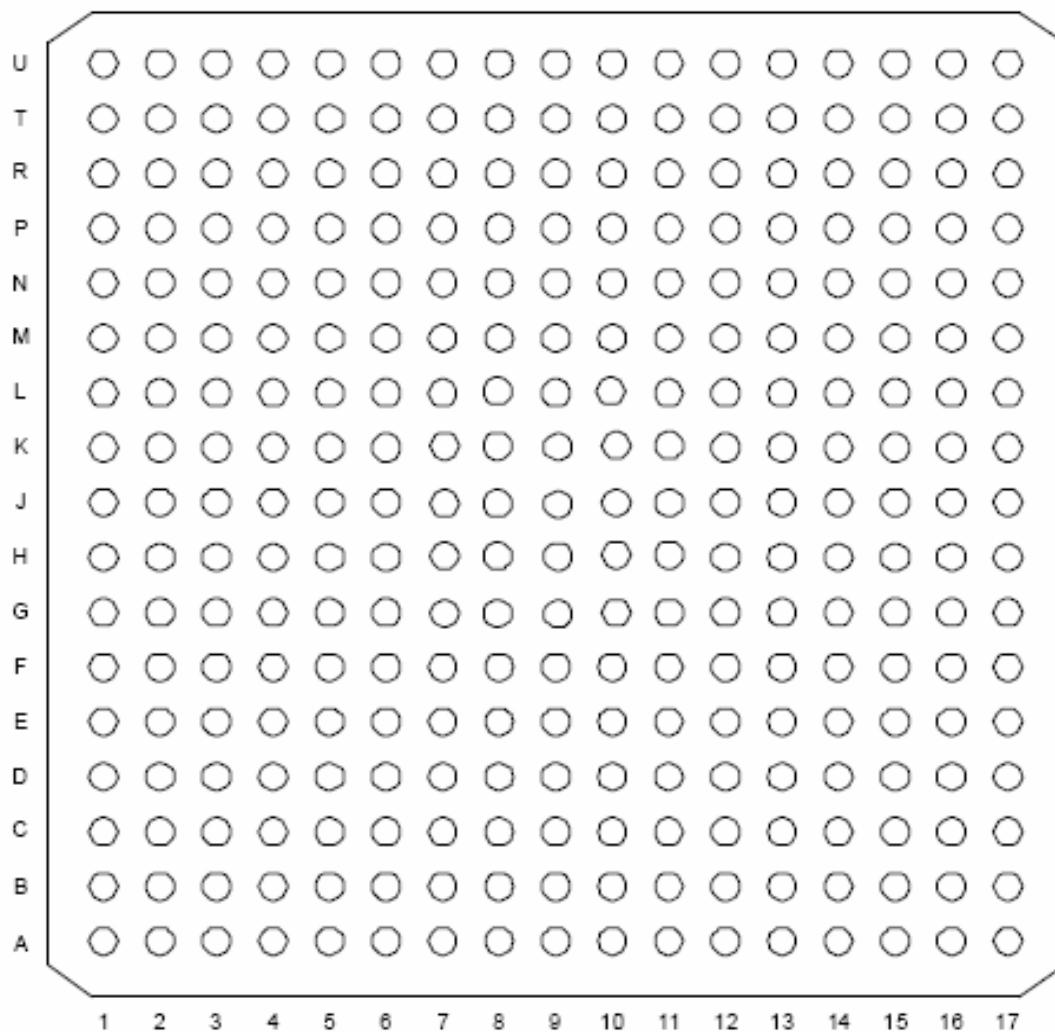


## 5.1.1 S3C2440信号描述

### (1) 管脚分配

289个引脚

289个引脚的  
编号与功能  
分配详见数  
据手册。



BOTTOM VIEW

## 5.1.1 S3C2440信号描述

### (2) 总线控制信号

**S3C2440 总线控制**具体信号引脚如下表所示：

| 信号         | I/O<br>类型 | 描述                                                                                                                      |
|------------|-----------|-------------------------------------------------------------------------------------------------------------------------|
| OM[1:0]    | I         | OM[1:0]在产品制造时可以将S3C2440设置为测试模式；它还决定bank0的总线的宽度；接在该引脚上的上拉或下拉电阻决定了其在复位期间的逻辑电平。<br>00: Nand-boot; 01:16位; 10:32位; 11: 测试模式 |
| ADDR[26:0] | O         | ADDR[26:0]是地址总线，输出对应bank区内的要访问的地址。                                                                                      |
| DATA[31:0] | I/O       | DATA[31:0]是数据总线，在读取存储器时，从该总线输入数据，在写存储器时，在该总线上输出数据。该总线的宽度可通过编程设定为 8/16/32位。                                              |

## 5.1.1 S3C2440信号描述

|           |   |                                                                                     |
|-----------|---|-------------------------------------------------------------------------------------|
| nGCS[7:0] | 0 | nGCS[7:0]是通用片选，当要访问的存储器位于某个BANK的地址区域时，那么该BANK对应的片选就会被激活。操作周期的数量和BANK宽度的大小可以通过编程改变。  |
| nWE       | 0 | nWE是写使能，表示当前总线周期是一个写周期                                                              |
| nOE       | 0 | nOE是输出使能，表示当前总线周期是一个读周期                                                             |
| nXBREQ    | 1 | nXBREQ总线占用请求，允许其他需要占用总线的总线主控制器请求本地总线的控制权。nXBACK信号被激活时表示总线控制请求被批准了。                  |
| nXBACK    | 0 | nXBACK是总线占用应答，表示S3C2440同意交出当地总线的控制，并转移到其他总线主控制器。                                    |
| nWAIT     | 1 | nWAIT是信号要求延长一个当前总线周期，只要nWAIT是低电平，当前总线周期就没有结束。如果nWAIT信号没有在系统中使用，必须用一个上拉电阻接在nWAIT引脚上。 |

## 其它信号自学

## 5.2 S3C2440的存储控制器

### 5.2.1 S3C2440存储控制器特性

S3C2440的存储控制器有以下特性：

- (1) 小/大端（通过软件选择）；
- (2) 地址空间：每BANK有128MB（共有1GB/8BANKS）；
- (3) 除BANK0（只能是16/32位宽）之外，其他BANK都具有可编程的访问大小（8/16/32位）；



## 5.2.1 S3C2440存储控制器特性

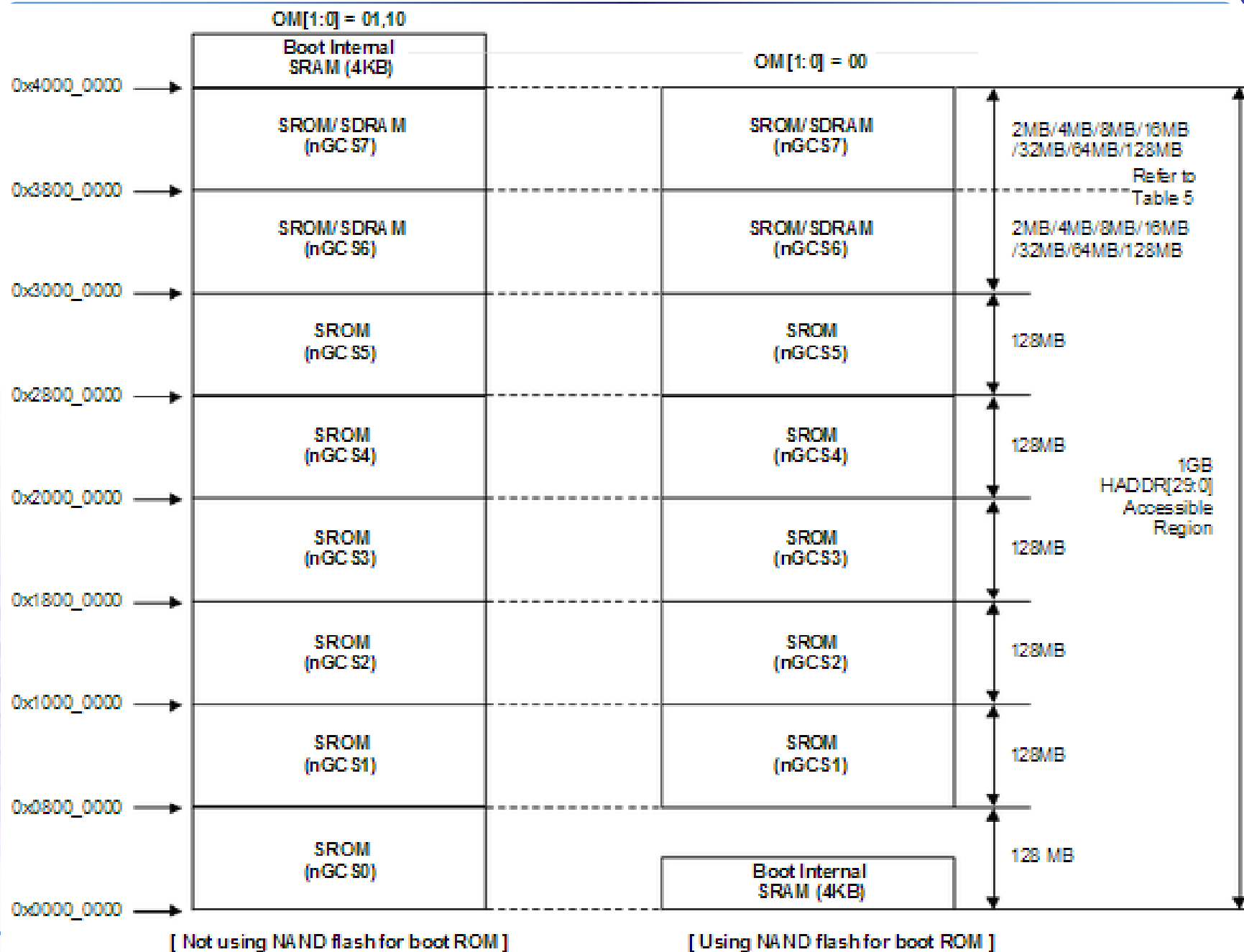
- (4) 总共8个存储器BANK，6个是ROM、SRAM等类型存储器BANK，剩下的2个可以为ROM、SRAM、**SDRAM**等存储器BANK；
- (5) 7个**固定的**存储器BANK的起止地址；
- (6) 最后一个BANK的起始地址是**可调整的**；
- (7) 最后2个BANK的大小是可编程的；
- (8) 所有存储器BANK的访问周期都是可编程的；

## 5.2.1 S3C2440存储控制器特性

(9) 总线访问周期可以通过插入外部等待(nWAIT)来延长;

(10) 支持SDRAM的自刷新和掉电模式。

下图为S3C2440复位后的存储图。



## 5.2.2 S3C2440存储控制器功能

### (1) BANK0总线宽度

BANK0 的数据总线 (nGCS0) 必须首先设置成 16 位或32位的, 因为 BANK0 通常作为 引导ROM区 (映射到地址0X00000000)。在复位时, 系统将检测 OM[1:0] 上的逻辑电平, 并依据这个电平来决定 BANK0 区存储器的总线宽度, 具体如下表所示:

数据宽度选择

| OM1 (操作模式1) | OM0 (操作模式0) | 导入ROM数据宽度       |
|-------------|-------------|-----------------|
| 0           | 0           | Nand Flash Mode |
| 0           | 1           | 16位             |
| 1           | 0           | 32位             |
| 1           | 1           | 测试模式            |

## 5. 2. 2 S3C2440存储控制器功能

### (2) 存储器（SRAM/SDRAM）地址引脚的连接

存储器（SRAM/SDRAM）地址引脚的连接方式由于数据宽度的设置不同而有所区别。

#### 存储器地址连接

| 存储器地址引脚   | S3C2440地址/8<br>位数据总线（8位芯<br>片） | S3C2440地址/16<br>位数据总线（16位<br>芯片） | S3C2440地址/32<br>位数据总线（32位芯<br>片） |
|-----------|--------------------------------|----------------------------------|----------------------------------|
| <b>A0</b> | <b>A0</b>                      | <b>A1</b>                        | <b>A2</b>                        |
| <b>A1</b> | <b>A1</b>                      | <b>A2</b>                        | <b>A3</b>                        |



## 5.2.3 S3C2440存储控制器寄存器

### (1) 总线宽度/等待控制寄存器 (BWSCON)

总线宽度/等待控制寄存器 (BWSCON) 的地址描述如下：

# 总线宽度/等待控制寄存器

| 寄存器    | 地址         | R/W | 描述           | 复位值        |
|--------|------------|-----|--------------|------------|
| BWSCON | 0X48000000 | R/W | 总线宽度和等待状态控制器 | 0X00000000 |

## 总线宽度/等待控制寄存器各位描述

UB/LB分别是存储器高8位, 低8位的数据线使能信号

| BWSCON | 位       | 描述                                                                          | 初始状态          |
|--------|---------|-----------------------------------------------------------------------------|---------------|
| ST7    | [31]    | 决定SRAM映射在BANK7时是否使用UB/LB；0：不使用UB/LB（引脚对应nWBE[3:0]）；1：使用UB/LB（引脚对应nBE[3:0]）； | 高低字节使能信号<br>0 |
| WS7    | [30]    | 决定BANK7中的SRAM存储器的等待状态，0：等待禁止；1：等待使能                                         | 0             |
| DW7    | [29:28] | 决定BANK7数据总线宽度；00:8位；01:16位；10:32位；11：保留                                     | 0             |

| BWSCON | 位       | 描述                                                                                      | 初始状态 |
|--------|---------|-----------------------------------------------------------------------------------------|------|
| ST6    | [27]    | 决定SRAM映射在BANK6时是否使用UB/LB;<br>0: 不使用UB/LB (引脚对应nWBE[3:0]);<br>1: 使用UB/LB (引脚对应nBE[3:0]); | 0    |
| WS6    | [26]    | 决定BANK6中的SRAM存储器的等待状态;<br>0: 等待禁止; 1: 等待使能                                              | 0    |
| DW6    | [25:24] | 决定BANK6数据总线宽度;<br>00:8位; 01:16位; 10:32位; 11: 保留                                         | 0    |
| ST5    | [23]    | 决定SRAM映射在BANK5时是否使用UB/LB;<br>0: 不使用UB/LB (引脚对应nWBE[3:0]);<br>1: 使用UB/LB (引脚对应nBE[3:0]); | 0    |
| WS5    | [22]    | 决定BANK5中的SRAM存储器的等待状态,<br>0: 等待禁止; 1: 等待使能                                              | 0    |
| DW5    | [21:20] | 决定BANK5数据总线宽度;<br>00:8位; 01:16位; 10:32位; 11: 保留                                         | 0    |

| BWSCON | 位       | 描述                                                                                      | 初始状态 |
|--------|---------|-----------------------------------------------------------------------------------------|------|
| ST4    | [19]    | 决定SRAM映射在BANK4时是否使用UB/LB;<br>0: 不使用UB/LB (引脚对应nWBE[3:0]);<br>1: 使用UB/LB (引脚对应nBE[3:0]); | 0    |
| WS4    | [18]    | 决定BANK4中的SRAM存储器的等待状态,<br>0: 等待禁止; 1: 等待使能                                              | 0    |
| DW4    | [17:16] | 决定BANK4数据总线宽度;<br>00:8位; 01:16位; 10:32位; 11: 保留                                         | 0    |
| ST3    | [15]    | 决定SRAM映射在BANK3时是否使用UB/LB;<br>0: 不使用UB/LB (引脚对应nWBE[3:0]);<br>1: 使用UB/LB (引脚对应nBE[3:0]); | 0    |
| WS3    | [14]    | 决定BANK3中的SRAM存储器的等待状态,<br>0: 等待禁止; 1: 等待使能                                              | 0    |
| DW3    | [13:12] | 决定BANK3数据总线宽度;<br>00:8位; 01:16位; 10:32位; 11: 保留                                         | 0    |

| BWSCON | 位     | 描述                                                                                      | 初始状态 |
|--------|-------|-----------------------------------------------------------------------------------------|------|
| ST2    | [11]  | 决定SRAM映射在BANK2时是否使用UB/LB;<br>0: 不使用UB/LB (引脚对应nWBE[3:0]);<br>1: 使用UB/LB (引脚对应nBE[3:0]); | 0    |
| WS2    | [10]  | 决定BANK2中的SRAM存储器的等待状态,<br>0: 等待禁止; 1: 等待使能                                              | 0    |
| DW2    | [9:8] | 决定BANK2数据总线宽度;<br>00:8位; 01:16位; 10:32位; 11: 保留                                         | 0    |
| ST1    | [7]   | 决定SRAM映射在BANK1时是否使用UB/LB;<br>0: 不使用UB/LB (引脚对应nWBE[3:0]);<br>1: 使用UB/LB (引脚对应nBE[3:0]); | 0    |
| WS1    | [6]   | 决定BANK1中的SRAM存储器的等待状态,<br>0: 等待禁止; 1: 等待使能                                              | 0    |
| DW1    | [5:4] | 决定BANK1数据总线宽度;<br>00:8位; 01:16位; 10:32位; 11: 保留                                         | 0    |



| BWSCON   | 位     | 描述                 | 初始状态 |
|----------|-------|--------------------|------|
| DW0      | [2:1] | 显示BANK0的数据总线宽度（只读） | —    |
| Reserved | [0]   | 不使用                | —    |

## 注意：

①在这个存储控制器里，所有类型的主时钟都对应着总线时钟。例如：SRAM的HCLK就是总线时钟，SDRAM的SCLK也和总线时钟相同；

## 5.2.3 S3C2440存储控制器寄存器

(2) 总线控制寄存器 (BANKCON<sub>n</sub>: nGCS0~nGCS5)  
总线控制寄存器 BANKCON0~BANKCON5 的地址信息如下:

| 寄存器      | 地址         | R/W | 描述            | 复位值    |
|----------|------------|-----|---------------|--------|
| BANKCON0 | 0X48000004 | R/W | BANKCON0控制寄存器 | 0X0700 |
| BANKCON1 | 0X48000008 | R/W | BANKCON1控制寄存器 | 0X0700 |
| BANKCON2 | 0X4800000C | R/W | BANKCON2控制寄存器 | 0X0700 |
| BANKCON3 | 0X48000010 | R/W | BANKCON3控制寄存器 | 0X0700 |
| BANKCON4 | 0X48000014 | R/W | BANKCON4控制寄存器 | 0X0700 |
| BANKCON5 | 0X48000018 | R/W | BANKCON5控制寄存器 | 0X0700 |

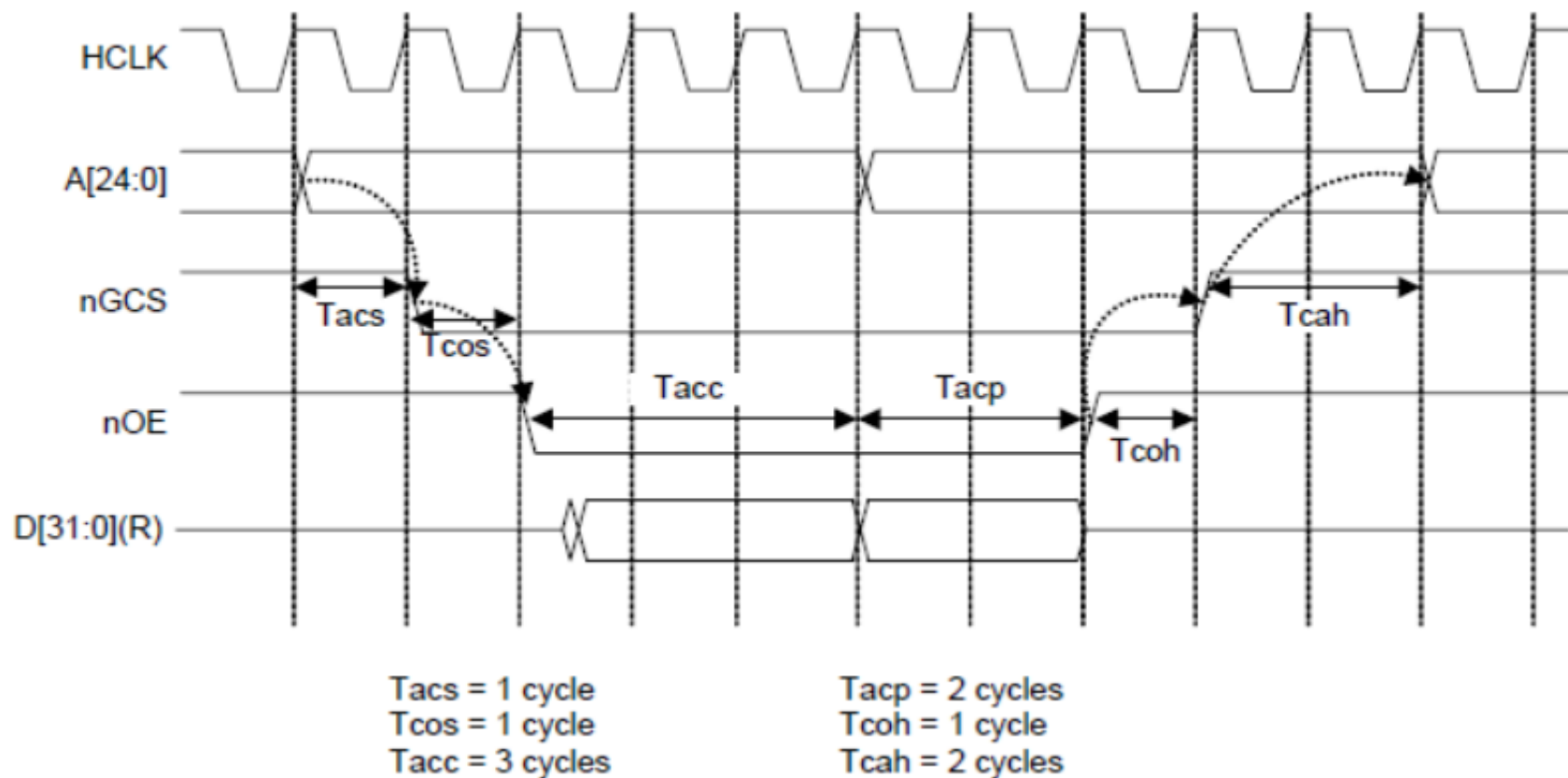
0X0700    0000 0111 0000 0000    Tacc [10:8]=111

# 总线控制寄存器 BANKCON0~BANKCON5的各位描述如下：

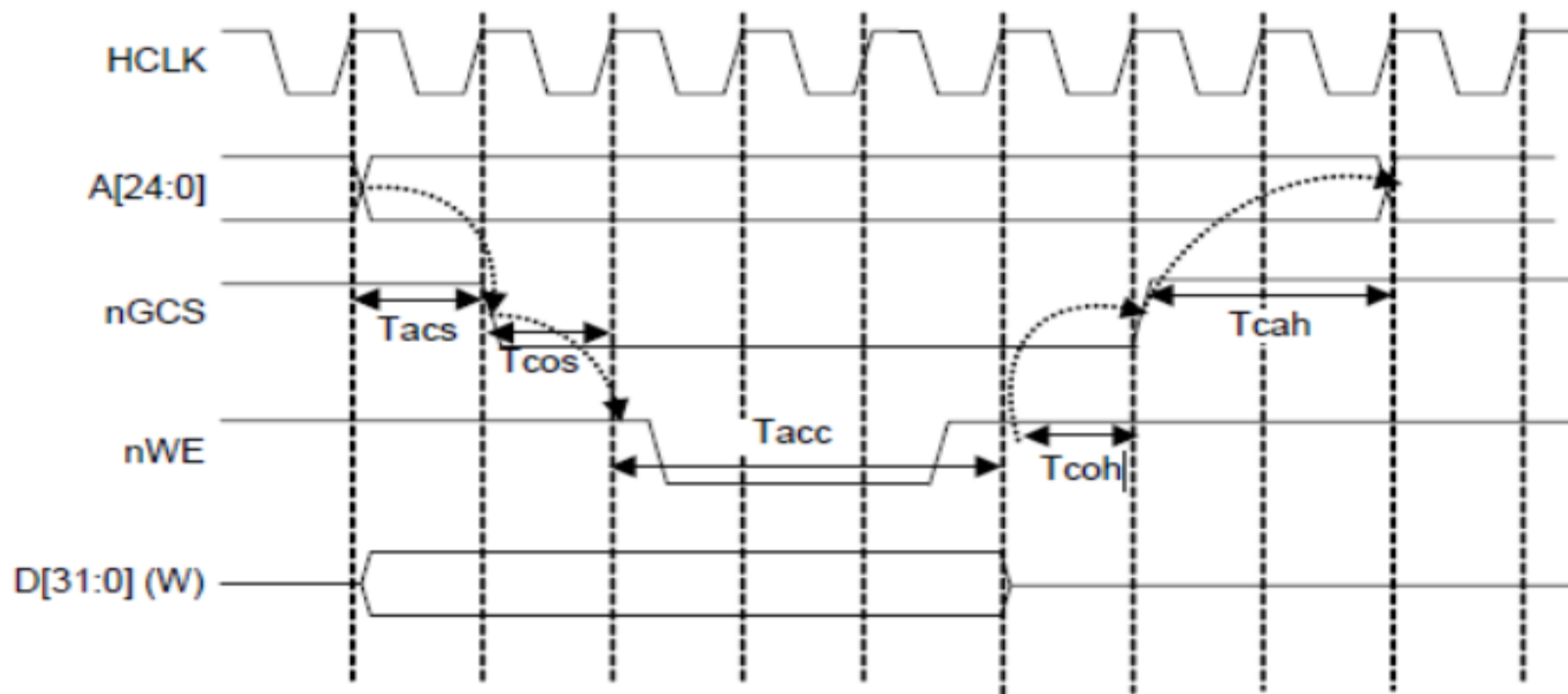
片选信号

| BANKCONn | 位       | 描述                                                                                                                             | 初始值 |
|----------|---------|--------------------------------------------------------------------------------------------------------------------------------|-----|
| Tacs     | [14:13] | 在nGCSn起效前，地址信号的建立时间。<br>00:0 clock; 01:1 clock;<br>10:2 clocks; 11:4 clocks                                                    | 00  |
| Tcos     | [12:11] | 在nOE起效之前，片选的建立时间。<br>00:0 clock; 01:1clock;<br>10:2 clocks; 11:4 clocks                                                        | 00  |
| Tacc     | [10:8]  | 访问周期<br>000:1 clock; 001:2 clocks;<br>010:3 clocks; 011:4 clocks<br>100:6 clock; 101:8 clocks;<br>110:10 clocks; 111:14 clocks | 111 |
| Tcoh     | [7:6]   | nOE之后，片选的保持时间。<br>00:0 clock; 01:1clock;<br>10:2 clocks; 11:4 clocks                                                           | 00  |

| BANKCONn | 位     | 描述                                                                              | 初始值 |
|----------|-------|---------------------------------------------------------------------------------|-----|
| Tcah     | [5:4] | nGCSn之后，地址信号的保持时间。<br>00:0 clock; 01:1 clock;<br>10:2 clocks; 11:4 clocks       | 00  |
| Tacp     | [3:2] | Page模式的访问周期（在Page模式下）。<br>00:2 clocks; 01:3 clocks;<br>10:4 clocks; 11:6 clocks | 00  |
| PMC      | [1:0] | Page模式配置。<br>00: 正常 (1 data) ;01:4 data;<br>10:8 data;11:16 data                | 00  |



- **Tacs**: 片选信号nGCSn起效前，地址信号建立时间
- **Tcos**: 在nOE（或nWE）起效前，片选信号建立时间
- **Tacc**: 访问周期
- **Tcoh**: nOE结束后（即电平升高），片选保持时间
- **Tcah**: nGCSn结束后（即电平升高），地址信号保持时间
- **Tacp**: Page模式的访问周期



$T_{acs} = 1 \text{ cycle}$   
 $T_{cos} = 1 \text{ cycle}$   
 $T_{acc} = 3 \text{ cycles}$

$T_{coh} = 1 \text{ cycle}$   
 $T_{cah} = 2 \text{ cycles}$

- **Tacs**: 片选信号nGCSn起效前，地址信号建立时间
- **Tcos**: 在nOE（或nWE）起效前，片选信号建立时间
- **Tacc**: 访问周期
- **Tcoh**: nWE结束后（即电平升高），片选保持时间
- **Tcah**: nGCSn结束后（即电平升高），地址信号保持时间
- **Tacp**: Page模式的访问周期



## 5.2.3 S3C2440存储控制器寄存器

### (3) BANK控制寄存器 (BANKCON<sub>n</sub>:nGCS6~nGCS7)

**BANK 控制寄存器 BANKCON6~BANKCON7的地址信息如下：**

| 寄存器      | 地址         | R/W | 描述         | 复位值     |
|----------|------------|-----|------------|---------|
| BANKCON6 | 0X4800001C | R/W | BANK6控制寄存器 | 0X18008 |
| BANKCON7 | 0X48000020 | R/W | BANK7控制寄存器 | 0X18008 |

**各个位的具体描述如下：**

| BANKCON <sub>n</sub> | 位       | 描述                                                                            | 起始状态 |
|----------------------|---------|-------------------------------------------------------------------------------|------|
| MT                   | [16:15] | 决定BANK6和BANK7的存储器类型。<br>00: ROM或SRAM; 01: 保留<br>10: 保留; 11: Sync. <b>DRAM</b> | 11   |

# 存储器类型=ROM或SRAM[MT=00]（15位）

| BANKCONn | 位       | 描述                                                                                             | 起始状态 |
|----------|---------|------------------------------------------------------------------------------------------------|------|
| Tacs     | [14:13] | nGCSn起效之前，地址信号的建立时间<br>00:0 clock; 01:1 clock<br>10:2 clocks; 11:4 clocks                      | 00   |
| Tcos     | [12:11] | nOE起效之前，片选信号的建立时间<br>00:0 clock; 01:1 clock<br>10:2 clocks; 11:4 clocks                        | 00   |
| Tacc     | [10:8]  | 访问时间<br>000:1 clock; 001:2 clocks;<br>010:3 clocks; 011:4 clocks<br>100:6 clocks; 101:8 clocks | 111  |
| Tcoh     | [7:6]   | nOE之后，片选的保持时间<br>00:0 clock; 01:1 clock<br>10:2 clocks; 11:4 clocks                            | 00   |
| Tcah     | [5:4]   | nGCSn之后，地址信号保持时间<br>00:0 clock; 01:1 clock<br>10:2 clocks; 11:4 clocks                         | 00   |

# 存储器类型=ROM或SRAM[MT=00]（15位）

| BANKCONn | 位     | 描述                                                                   | 起始状态 |
|----------|-------|----------------------------------------------------------------------|------|
| Tacp     | [3:2] | Page模式下的访问周期<br>00:2 clocks; 01:3 clocks<br>10:4 clocks; 11:6 clocks | 00   |
| PMC      | [1:0] | Page模式配置<br>00:正常（1 data）； 01:4数据连续访问<br>10:8数据连续访问； 11:16数据连续访问     | 00   |

存储器类型=SDRAM[MT=1

行地址选通信号

到列地址选通信号

| BANKCONn | 位     | 描述                                                 | 起始状态 |
|----------|-------|----------------------------------------------------|------|
| Trcd     | [3:2] | RAS到CAS延迟<br>00:2 clocks; 01:3 clocks; 10:4 clocks | 10   |
| SCAN     | [1:0] | 列地址位数<br>00:8位; 01:9位; 10:10位                      | 00   |

## 5.2.3 S3C2440存储控制器寄存器

### (4) SDRAM刷新控制寄存器

刷新控制寄存器的地址信息如下所示：

| 寄存器 | 地址         | R/W | 描述           | 复位值      |
|-----|------------|-----|--------------|----------|
| 刷新  | 0X48000024 | R/W | SDRAM刷新控制寄存器 | 0XAC0000 |

它的设置决定了 SDRAM的刷新 是否允许、刷新模式、RAS预充电的时间、RAS和CAS最短时间、CAS保持时间 以及 刷新计数值，各位的描述如下所示：

| 刷新              | 位       | 描述                                                                                                                                                                       | 起始状态 |
|-----------------|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| REFEN           | [23]    | SDRAM刷新使能, 0: 停止; 1: 使能                                                                                                                                                  | 1    |
| TREFMD          | [22]    | SDRAM刷新模式<br>0: 自动刷新; 1: 自我刷新（在自我刷新模式下），SDRAM控制信号被置于适当的电平                                                                                                                | 0    |
| Trp             | [21:20] | SDRAM RAS预充电时间, 00: 2 clocks;<br>01: 3 clocks; 10: 4 clocks; 11: 不支持                                                                                                     | 10   |
| Tsrc            | [19:18] | SDRAM半行周期时间。<br>00: 4 clocks; 01: 5 clocks; 10: 6 clocks;<br>11: 7 clocks;<br>SDRAM运行周期时间 (Trc) = Tsrc + Trp, 如果<br>Trp = 3 clocks 和 Tsrc = 7 clocks, Trc = 10<br>clocks | 11   |
| Reserved        | [17:16] | 不使用                                                                                                                                                                      | 00   |
| Reserved        | [15:11] | 不使用                                                                                                                                                                      | 0000 |
| Refresh Counter | [10:0]  | SDRAM刷新计数器值, 刷新时间 = $(2^{11} - \text{刷新计数值} + 1) / \text{HCLK}$ 。如果刷新时间是 15.6us, HCLK 是 60MHz, 刷新计数器值计算如下:<br>刷新计数值 = $2^{11} - 60 * 15.6 + 1$                           | 0    |

## 5.2.3 S3C2440存储控制器寄存器

### (5) BANKSIZE寄存器

**BANKSIZE** 寄存器是控制 **BANK** 大小的寄存器，主要是决定 **BANK6/7** 的存储区域的空间大小。**BANKSIZE** 寄存器的地址信息如下：

| 寄存器      | 地址         | R/W | 描述              | 复位值 |
|----------|------------|-----|-----------------|-----|
| BANKSIZE | 0X48000028 | R/W | 可灵活设置的BANK尺寸寄存器 | 0X0 |



| BANKSIZE | 位     | 描述                                                                                                                             | 起始状态 |
|----------|-------|--------------------------------------------------------------------------------------------------------------------------------|------|
| BURST_EN | [7]   | ARM内核 <b>猝发操作使能</b><br>0: 禁止; 1: 使能                                                                                            | 0    |
| Reserved | [6]   | 猝发写命令与猝发读命令类似，允许DRAM芯片中的一行被激活后，连续写入若干个数据。                                                                                      |      |
| SCKE_EN  | [5]   |                                                                                                                                |      |
| SCLK_EN  | [4]   | 只有在SDRAM访问周期期间，SCLK才使能，这样做是可以减少功耗。当SDRAM不被访问时，SCLK变成低电平。<br>0: SCLK总是激活;<br>1: SCLK只有在访问期间（推荐的）激活，推荐设置为1                       | 0    |
| Reserved | [3]   | 不使用                                                                                                                            | 0    |
| BK76MAP  | [2:0] | BANK6/7的存储空间分布，<br>010: 128MB/128MB; 001: 64MB/64MB;<br>000: 32MB32MB; 111:16MB/16MB;<br>110:8MB/8MB; 101:4MB/4MB; 100:2MB/2MB | 010  |

## 5.2.3 S3C2440存储控制器寄存器

### (6) SDRAM模式寄存器集寄存器 (MRSR)

SDRAM模式寄存器集寄存器的地址信息描述如下：

| 寄存器    | 地址         | R/W | 描述             | 复位值 |
|--------|------------|-----|----------------|-----|
| MRSRB6 | 0X4800002C | R/W | 模式寄存器集BANK6寄存器 | X   |
| MRSRB7 | 0X48000030 | R/W | 模式寄存器集BANK7寄存器 | X   |

SDRAM模式寄存器集寄存器各位的设置如下：

| MRSR     | 位       | 描述                                                                  | 起始状态 |
|----------|---------|---------------------------------------------------------------------|------|
| Reserved | [11:10] | 不使用                                                                 | —    |
| WBL      | [9]     | 猝发写的长度<br>0: 猝发（固定的）<br>1: 保留                                       | X    |
| TM       | [8:7]   | 测试模式<br>00: 模式寄存器集（固定的）<br>01、10、11: 保留                             | X    |
| CL       | [6:4]   | CAS（列地址通选）反应时间<br>000:1 clock; 010:2 clocks<br>011:3 clocks; 其他: 保留 | X    |
| BT       | [3]     | 猝发类型<br>0: 连续的（固定）<br>1: 保留                                         | X    |
| BL       | [2:0]   | 猝发时间<br>000:1（固定的）<br>其他: 保留                                        | X    |

## 5. 2. 3 S3C2440存储控制器寄存器

**注意：**当代码在 SDRAM 中运行时，绝不能够重新配置 MRSR 寄存器。

**重要说明：**在掉电模式下，SDRAM 必须进入 SDRAM 的自我刷新模式。

## 5.3 NAND Flash控制器

由于 NOR Flash 价格较高，而 SDRAM 和 NAND Flash 存储器相对经济，这样促使一些用户在 NAND Flash 上存储启动代码，在 SDRAM 上执行主程序。

S3C2440A 的驱动代码可以在执行在存储在外部NAND Flash 存储器上的代码。为了支持 NAND Flash 的 boot loader，S3C2440A 配备了一个内部的 SRAM 缓冲器名为“Steppingstone”（bootram）。

## 5.3 NAND Flash控制器

启动时，NAND Flash 上的前 4KByte 字节将被装载到 Steppingstone 中，并且装载到 Steppingstone 上的启动代码会被执行。

一般情况下，启动代码会拷贝 NAND Flash 上的内容到 SDRAM 中，在引导代码执行完毕后就跳转到 SDRAM 执行。使用硬件的 ECC（纠错码）对 NAND Flash 的数据进行有效性检查。在完成拷贝的基础上，主程序将在 SDRAM 上被执行。



## 5.3.1 NAND Flash 控制器的特性（了解）

### （1） NAND Flash 控制器的特性

①自动启动：启动代码在重启时被传输到 4kbytes 的 Steppingstone 上。传输后代码会在 Steppingstone 上被执行；

②NAND Flash 存储器接口：支持 256字、512字节、1000 字 和 2000 字节 页；

### 5.3.1 NAND Flash 控制器的特性（了解）

③软件模式：用户可以直接访问 NAND Flash，例如利用这个特性可以被用于对 NAND Flash 存储器的读/擦除/编程；

④接口：8/16 位的 NAND Flash 存储器接口总线；

⑤硬件 ECC 生成，检测和指示（软件纠错）；

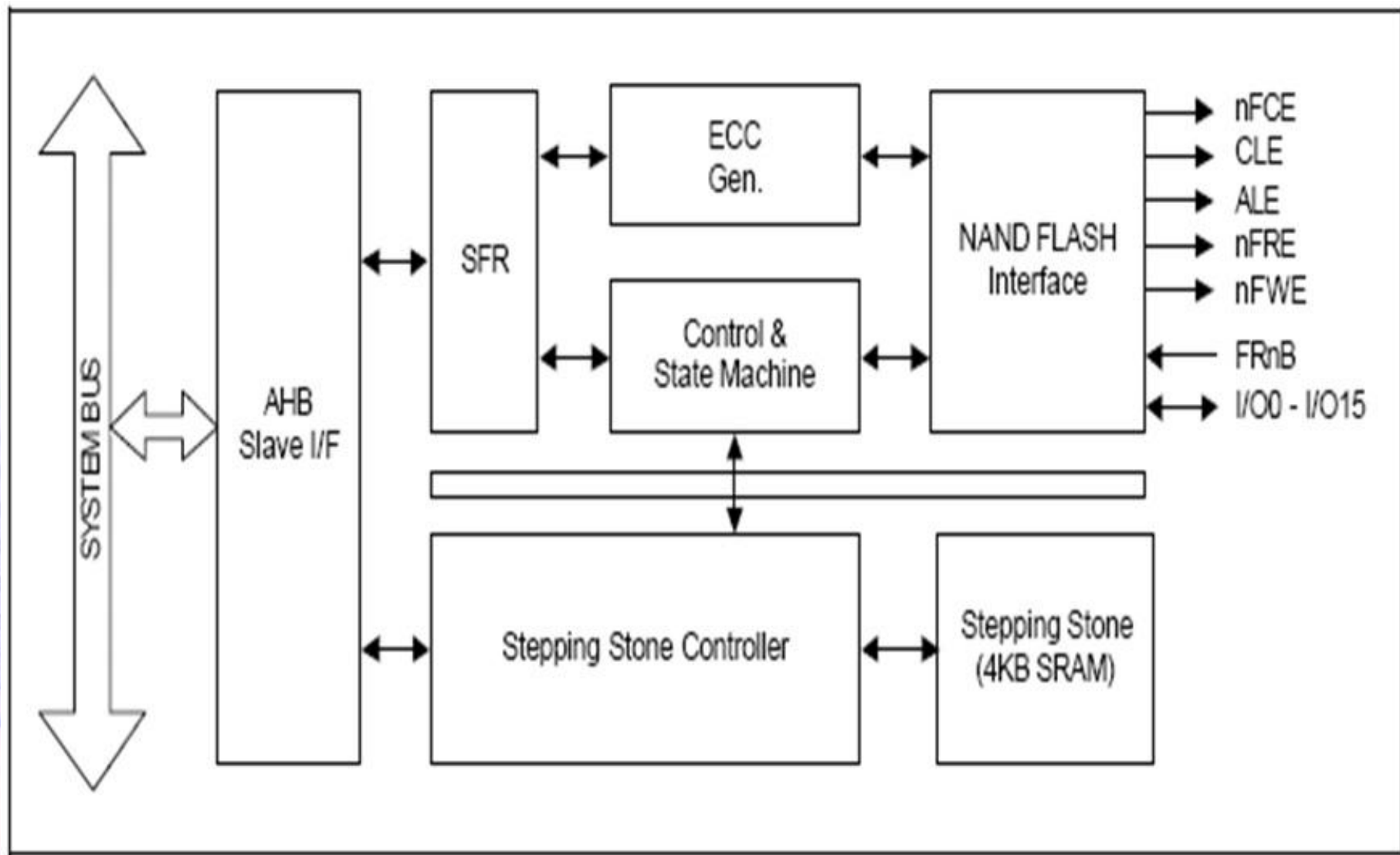
## 5.3.1 NAND Flash 控制器的特性（了解）

⑥SFR (特殊功能寄存器) 接口：支持小端模式，对于数据和 ECC 数据寄存器的 字节/半字/字访问，对于其他寄存器的字访问；

⑦Steppingstone 接口：支持大小端，字节/半字/字访问；

⑧Steppingstone 4kB 内部 SRAM 缓冲器可以在 NAND Flash 启动后被用于其他目的。

## (2) NAND Flash模块图



### 5.3.1 NAND Flash 控制器的特性（了解）

在重启期间，NAND Flash 控制器通过引脚状态得到连接的NAND Flash 的信息（NCON (Advflash)，GPG13 (页大小)，GPG14 (地址周期)，GPG15 (总线宽度)——参考引脚配置）。在上电或重启以后，NAND Flash 控制器自动的装载 4KB的boot loader 代码。在装载boot loader 代码后，其在 steppingstone 中被执行。

注：在自动重启期间，ECC没有检查，因为NAND Flash的前4KB数据一般认为没有位错误。

## 5.3.1 NAND Flash 控制器的特性（了解）

### （4）引脚配置

OM[1:0]=00：使能NAND Flash存储器启动；

NCON：NAND Flash存储器选择

0：普通NAND Flash（256 字/512 字节页大小，3个/4个 地址周期）；

1：先进NAND Flash（1K字/2K字节页大小，4个/5个 地址周期）。



## 5.3.1 NAND Flash 控制器的特性（了解）

### GP13: NAND Flash存储器页容量选择

0: 页=256 字 (NCON=0) 或页=1K字 (NCON=1) ;

1: 页=512 字节 (NCON=0) 或页=2K字节 (NCON=1)。

### GP14: NAND Flash存储器地址周期选择

0: 3 个地址周期 (NCON=0) 或4 个地址周期 (NCON=1) ;

1: 4 个地址周期 (NCON=0) 或5 个地址周期 (NCON=1) 。

## 5.3.1 NAND Flash 控制器的特性（了解）

### GPG15 NAND Flash存储器总线宽度选择

- 0：8 位总线宽度；
- 1：16 位总线宽度。

NAND Flash存储器配置如下表所示。

| NCON0             | GPG13     | GPG14   | GPG15         |
|-------------------|-----------|---------|---------------|
| 0：正常NAND<br>Flash | 0：256Word | 0：3地址周期 | 0：8位总线宽<br>度  |
|                   | 1：512Byte | 1：4地址周期 |               |
| 1：高级NAND<br>Flash | 0：1KWord  | 0：4地址周期 | 1：16位总线<br>宽度 |
|                   | 1：2KByte  | 1：5地址周期 |               |

## 5.4 时钟和功率管理

时钟和功率管理模块由 **三部分** 组成：**时钟控制**、**USB控制** 和 **功率控制**。

S3C2440 的时钟控制逻辑能够产生系统所需要的时钟，包括：CPU的FCLK、AHB总线接口的HCLK 和 APB总线接口的PCLK。

S3C2440 有两个PLL（锁相环），一个用于FCLK、HCLK、PCLK；另一个用于USB模块（48MHZ）。时钟控制逻辑能够由软件控制不将PLL 连接到各接口模块以降低处理器时钟频率，从而降低功耗。

## 5.4 时钟和功率管理

S3C2440 有各种针对不同任务提供的**最佳功率管理策略**，功率管理模块能够使系统工作在如下四种模式：**正常模式、低速模式、空闲模式和 掉电模式。**

### (1) 正常模式

**功率管理模块向 CPU 和 所有外部设备提供时钟。**这种模式下，当所有外设都开启时，系统功率将达到**最大**。用户可以通过**软件控制各种外设的开关。**

## 5.4 时钟和功率管理

### (2) 低速模式

没有PLL的模式。与正常模式不同，低速模式直接使用外部时钟（XT1p11或者EXTCLK）作为FCLK，这种模式下，功率仅由外部时钟决定。

### (3) 空闲模式

功率管理模块仅关掉FCLK，而继续提供时钟给其他外设。空闲模式可以减少由于CPU核心产生的功耗。任何中断请求都可以将CPU从中断模式唤醒。



## 5.4 时钟和功率管理

### (4) 掉电模式

功率管理模块断开内部电源，因此 CPU 和除唤醒逻辑单元以外的外设都不会产生功耗。要执行掉电模式需要有两个独立的电源，其中一个给唤醒逻辑单元供电，另一个给包括CPU在内的其他模块供电。在掉电模式下，第二个电源将被关掉。掉电模式可以由外设中断 EINT[15:0]或RTC唤醒。

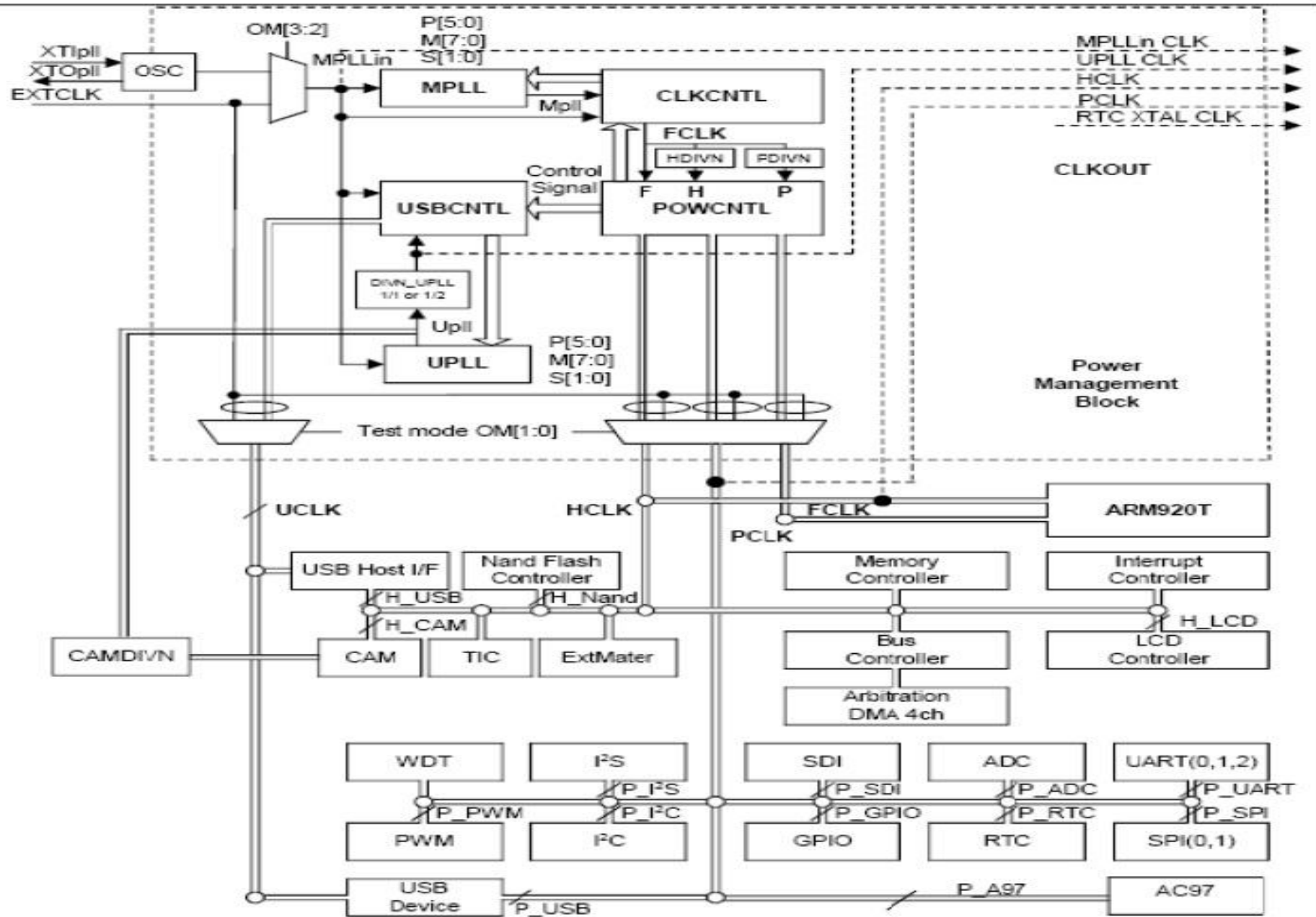


## 5.4 时钟和功率管理

### 5.4.1 时钟和功率管理功能描述

#### (1) 时钟发生器

时钟发生器功能框图如下：



## 5.4 时钟和功率管理

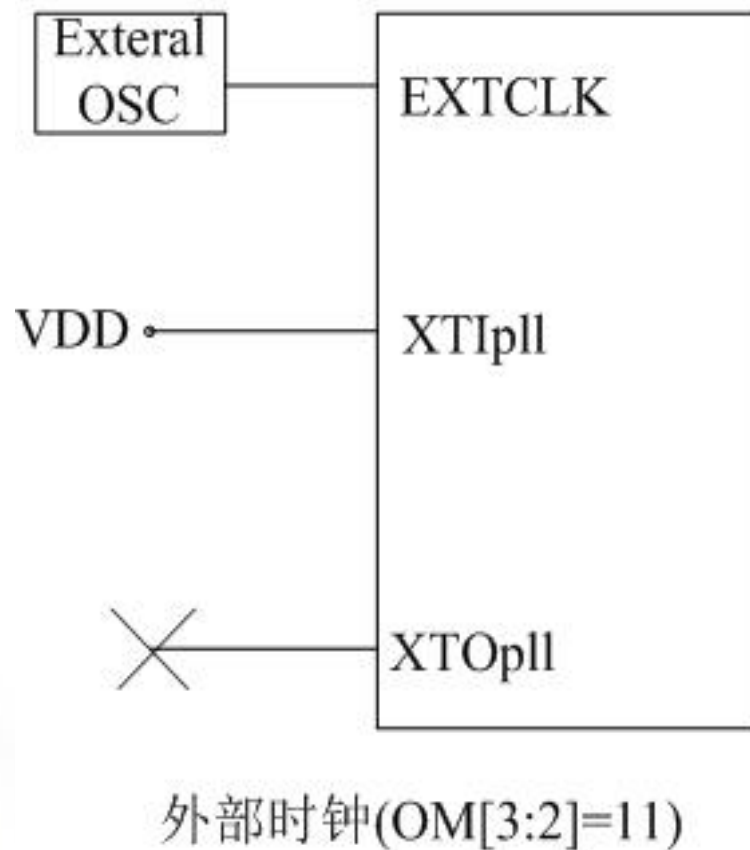
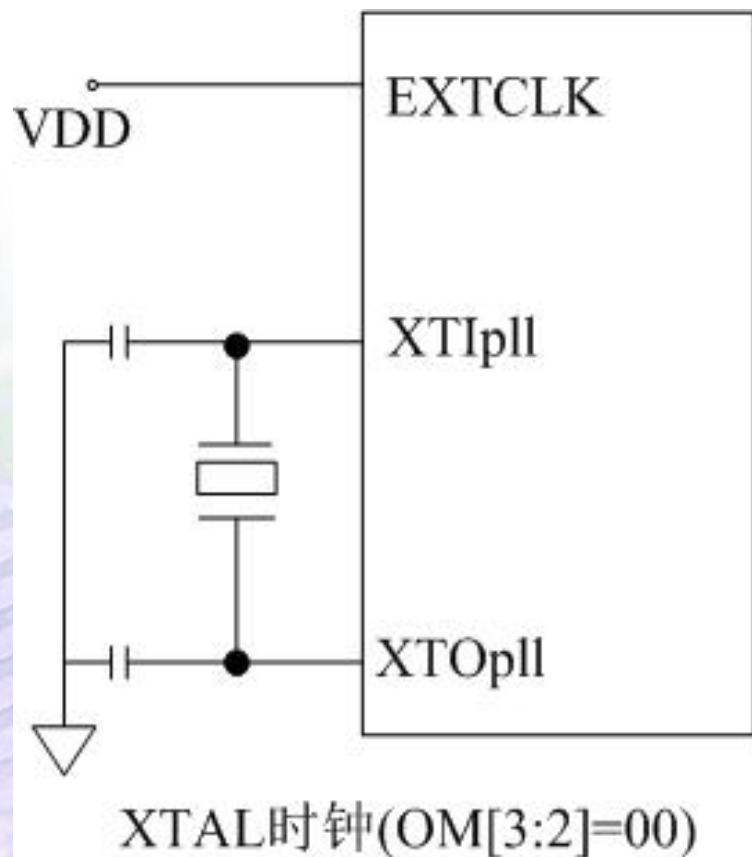
### (2) 时钟源选择

控制引脚（OM3和OM2）的组合模式与 S3C2440 时钟源选择关系如下表所示。引脚 OM[3:2] 的状态在 nRESET 上升沿通过控制引脚 OM3 和 OM2 被锁定。

| 模式OM[3:2] | MPLL状态 | UPLL状态 | 主时钟源 | USB时钟源 |
|-----------|--------|--------|------|--------|
| 00        | on     | on     | 晶振   | 晶振     |
| 01        | on     | on     | 晶振   | 外部时钟   |
| 10        | on     | on     | 外部时钟 | 晶振     |
| 11        | on     | on     | 外部时钟 | 外部时钟   |

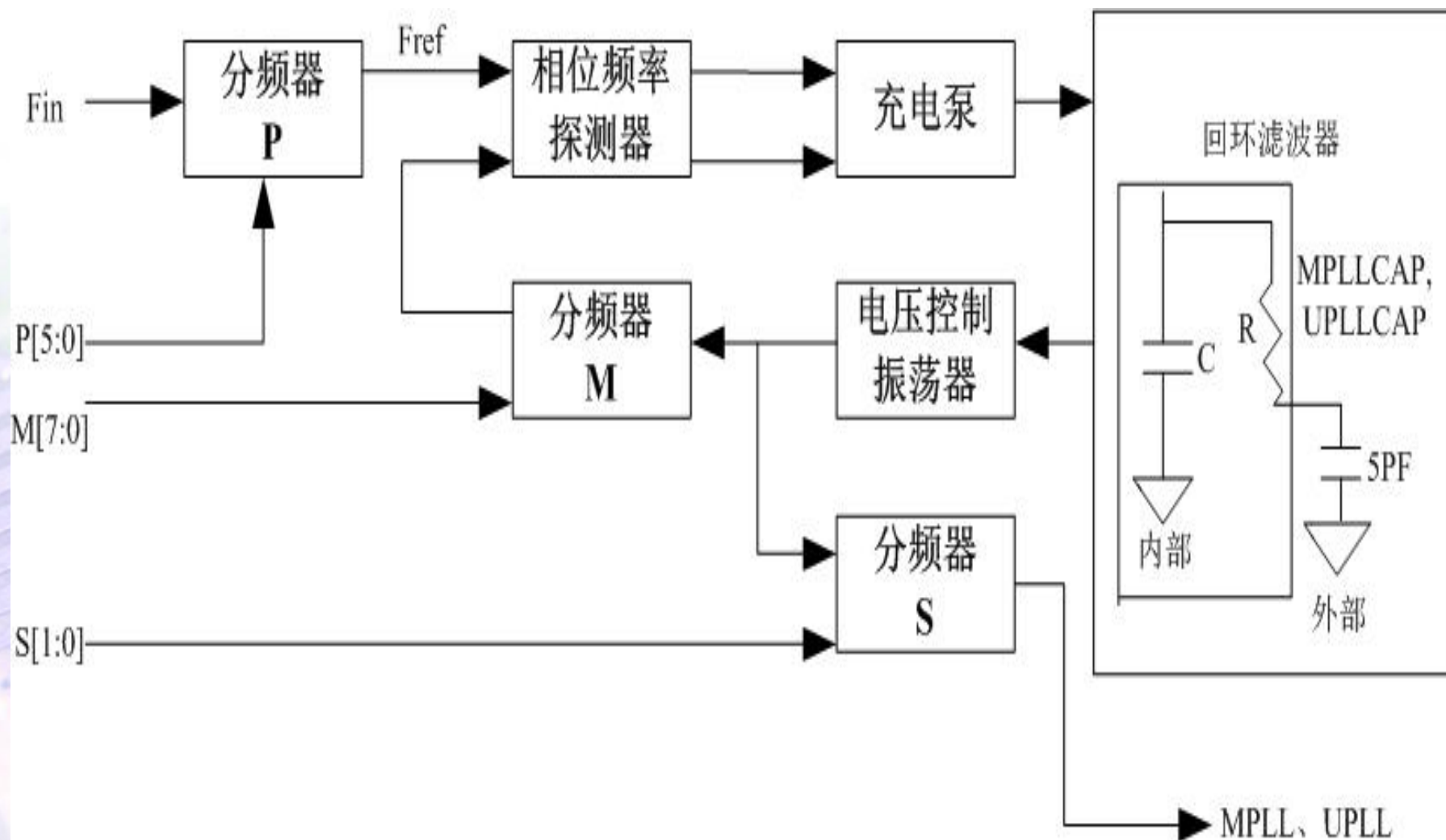
## 5.4 时钟和功率管理

下图为振荡回路举例：



## 5.4 时钟和功率管理

### (3) 锁相环PLL

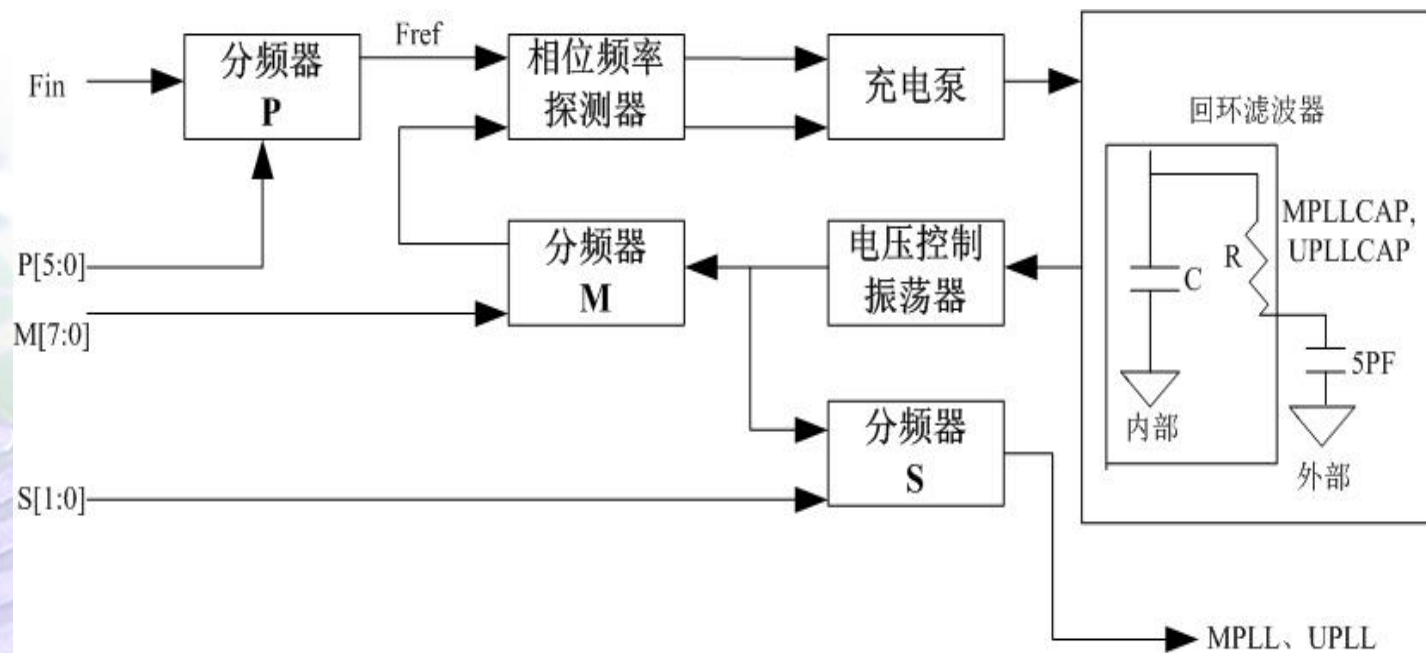


## 5.4 时钟和功率管理

$$M_{pll} = (2 * m * F_{in}) / (p * 2^s)$$

$$m = M(\text{分频器M的分频值}) + 8$$

$$p = P(\text{分频器P的分频值}) + 2$$



S3C2440上的时钟源是 **12MHz**，开发板上常用的设置为  $m=75$  (0x4B)， $p=3$ ， $s=1$

$$\text{则 } M_{pll} = (2 * 75 * 12) / (3 * 2^1) = 300M$$



## 5.4 时钟和功率管理

### (4) 时钟控制逻辑

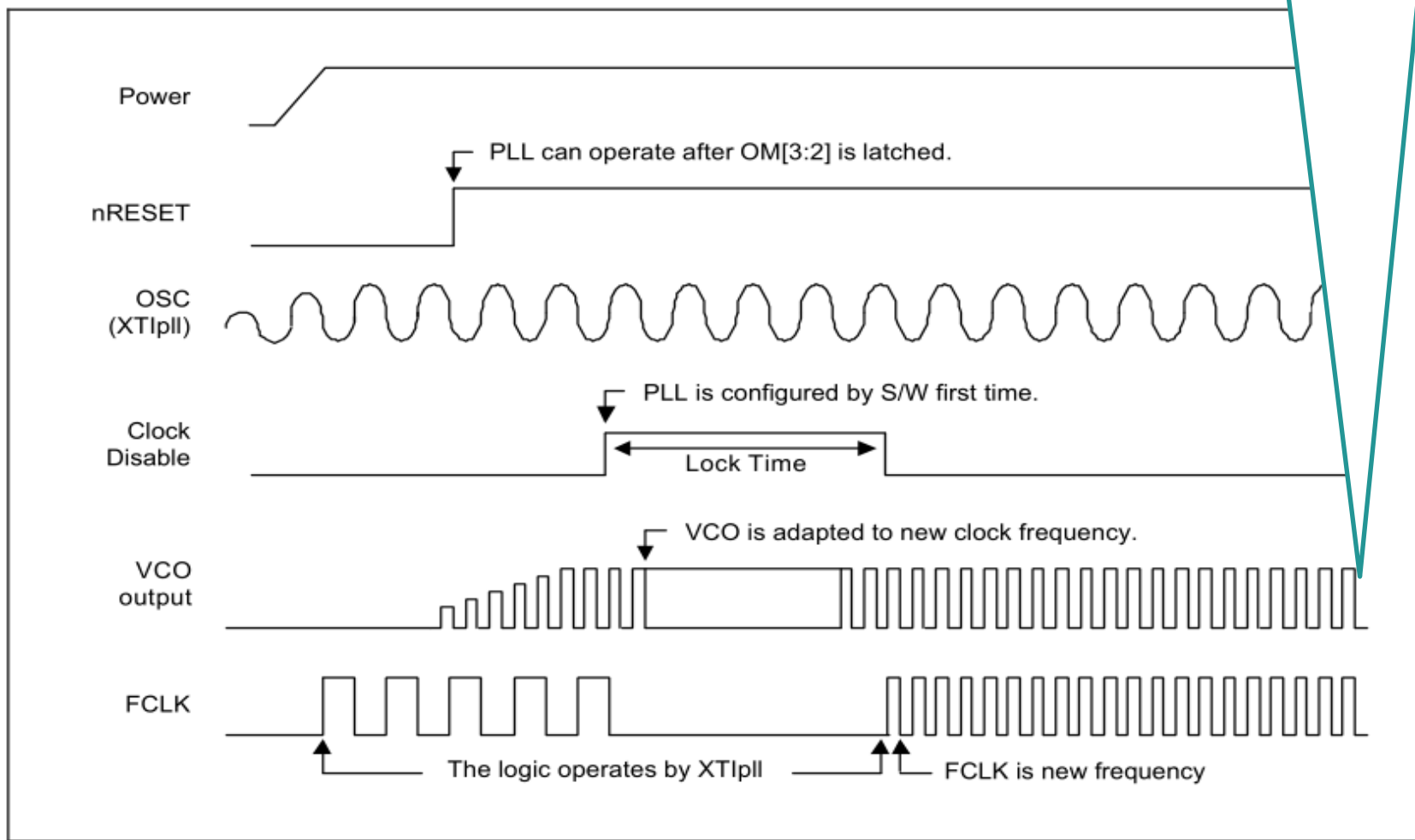
时钟控制逻辑决定使用 哪个 时钟源，例如是 PLL时钟 还是直接的 外部时钟 (XTIpll或者EXTCLK)。当 PLL 被配置为一个新的频率值时，时钟控制逻辑在 PLL输出稳定之前 禁止 FCLK，直到 PLL锁定系统时钟后 取消禁止。时钟控制逻辑同时在 开电源重启时 和 从掉电模式唤醒时起作用。

## 5.4 时钟和功率管理

### ①加电重启

上电重启的次序图如下：

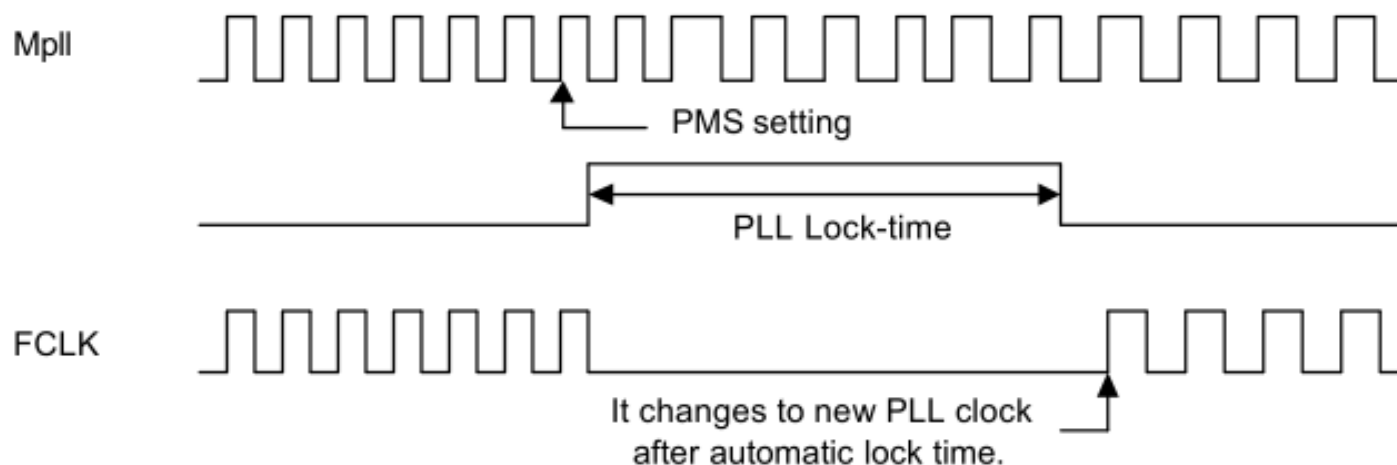
电压控制振荡器



## 5.4 时钟和功率管理

### ②在普通操作模式下改变 PLL 设置

通过设置 **P、M、S** 的值改变低速时钟时序图如下：



## 5.4 时钟和功率管理

### (5) USB模块控制

USB主接口 和 USB驱动接口 需要 48MHZ 的时钟。在 S3C2440 中，UPLL (USB dedicated PLL) 为 USB 产生 48MHZ 的时钟。在 UPLL 没有被设置之前 UCLK 不产生。对应于不同条件下的 UCLK 和 UPLL 状态如下表所示：

| 条件               | UCLK状态                         | UPLL状态 |
|------------------|--------------------------------|--------|
| 重启之后             | XTIpll或者EXTCLK                 | 0n     |
| UPLL配置之后         | 低电平：PLL锁定时间<br>48MHZ：PLL锁定时间之后 | 0n     |
| CLKSLOW寄存器关掉UPLL | XTIpll或者EXTCLK                 | 0ff    |
| CLKSLOW寄存器打开UPLL | 48MHZ                          | 0n     |

## 5.4 时钟和功率管理

### (6) FCLK、HCLK 和 PCLK

- ① FCLK信号是ARM920T内核使用的时钟信号；
- ② HCLK信号是AHB总线的时钟信号，在ARM920T内核、存储控制器、中断控制器、LCD控制器、DMA和USB主模块使用；
- ③ PCLK信号是APB总线的时钟信号，在外部设备中使用，如WDT、IIS、I<sup>2</sup>C、PWM定时器、MMC接口、ADC、UART、GPIO、RTC和SPI。

## 5.4 时钟和功率管理

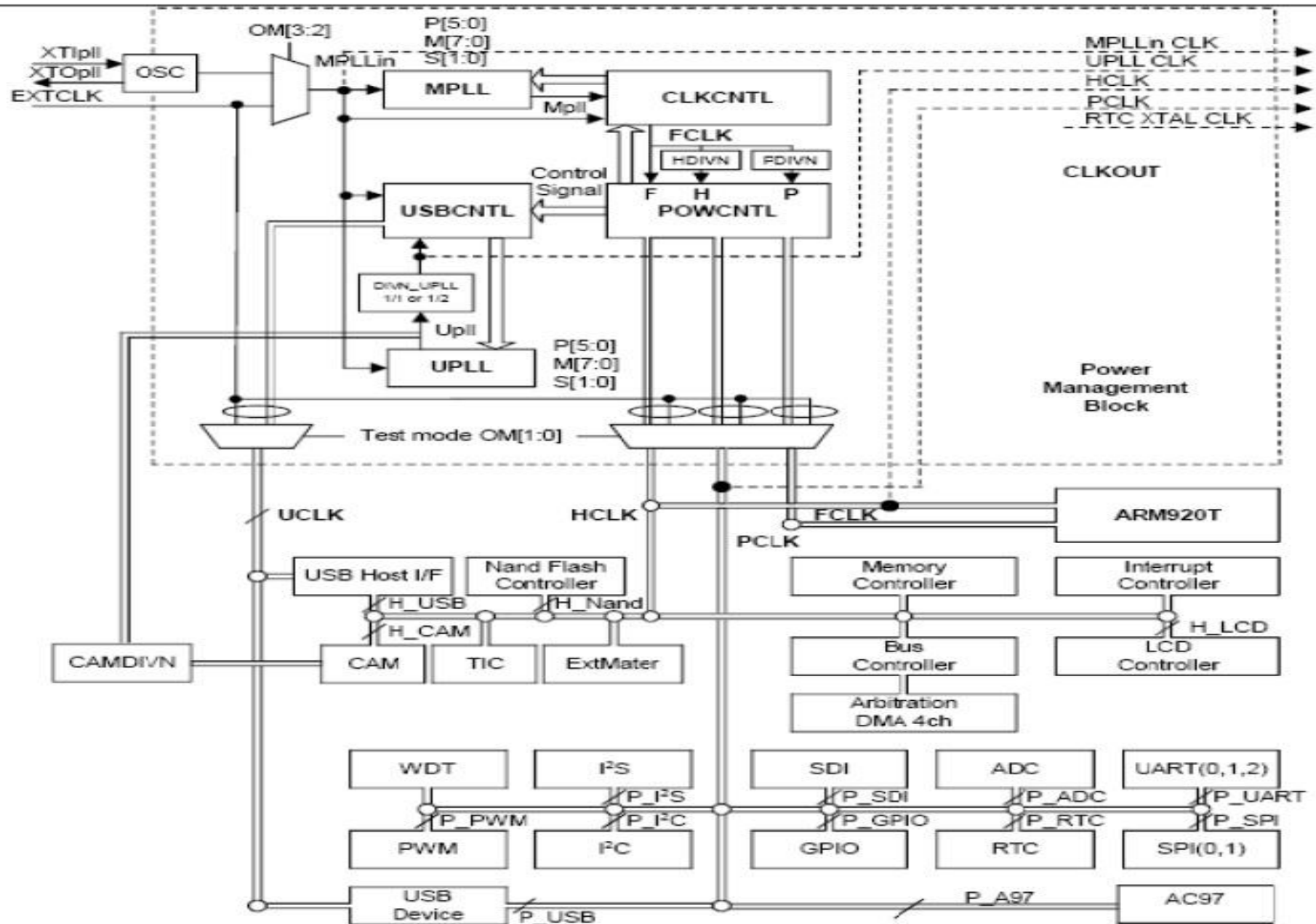
S3C2440 在 FCLK、HCLK 和 PCLK之间可以选择划分比率。比率由 CLKDIVN 控制寄存器的 HDIVN 和 PDIVN 决定，具体如下表所示：



| HDIVN<br>[2:1] | PDIVN<br>[0] | HCLK3_HALF<br>/HCLK4_HALF | FCLK | HCLK   | PCLK    | 划分比率      |
|----------------|--------------|---------------------------|------|--------|---------|-----------|
| 0              | 0            | —                         | FCLK | FCLK   | FCLK    | 1:1:1（缺省） |
| 0              | 1            | —                         | FCLK | FCLK   | FCLK/2  | 1:1:2     |
| 1              | 0            | —                         | FCLK | FCLK/2 | FCLK/2  | 1:2:2     |
| 1              | 1            | —                         | FCLK | FCLK/2 | FCLK/4  | 1:2:4     |
| 3              | 0            | 0/0                       | FCLK | FCLK/3 | FCLK/3  | 1:3:3     |
| 3              | 1            | 0/0                       | FCLK | FCLK/3 | FCLK/6  | 1:3:6     |
| 3              | 0            | 1/0                       | FCLK | FCLK/6 | FCLK/6  | 1:6:6     |
| 3              | 1            | 1/0                       | FCLK | FCLK/6 | FCLK/12 | 1:6:12    |
| 2              | 0            | 0/0                       | FCLK | FCLK/4 | FCLK/4  | 1:4:4     |
| 2              | 1            | 0/0                       | FCLK | FCLK/4 | FCLK/8  | 1:4:8     |
| 2              | 0            | 0/1                       | FCLK | FCLK/8 | FCLK/8  | 1:8:8     |
| 2              | 1            | 0/1                       | FCLK | FCLK/8 | FCLK/16 | 1:8:16    |

## 5.4 时钟和功率管理

在设定 **P、M、S** 的值之后，需要设定 **CLKDIVN** 寄存器的值，其会在 **PLL** 锁定的时间后有效，当然在重启和改变电源管理模式之后也可以使用。



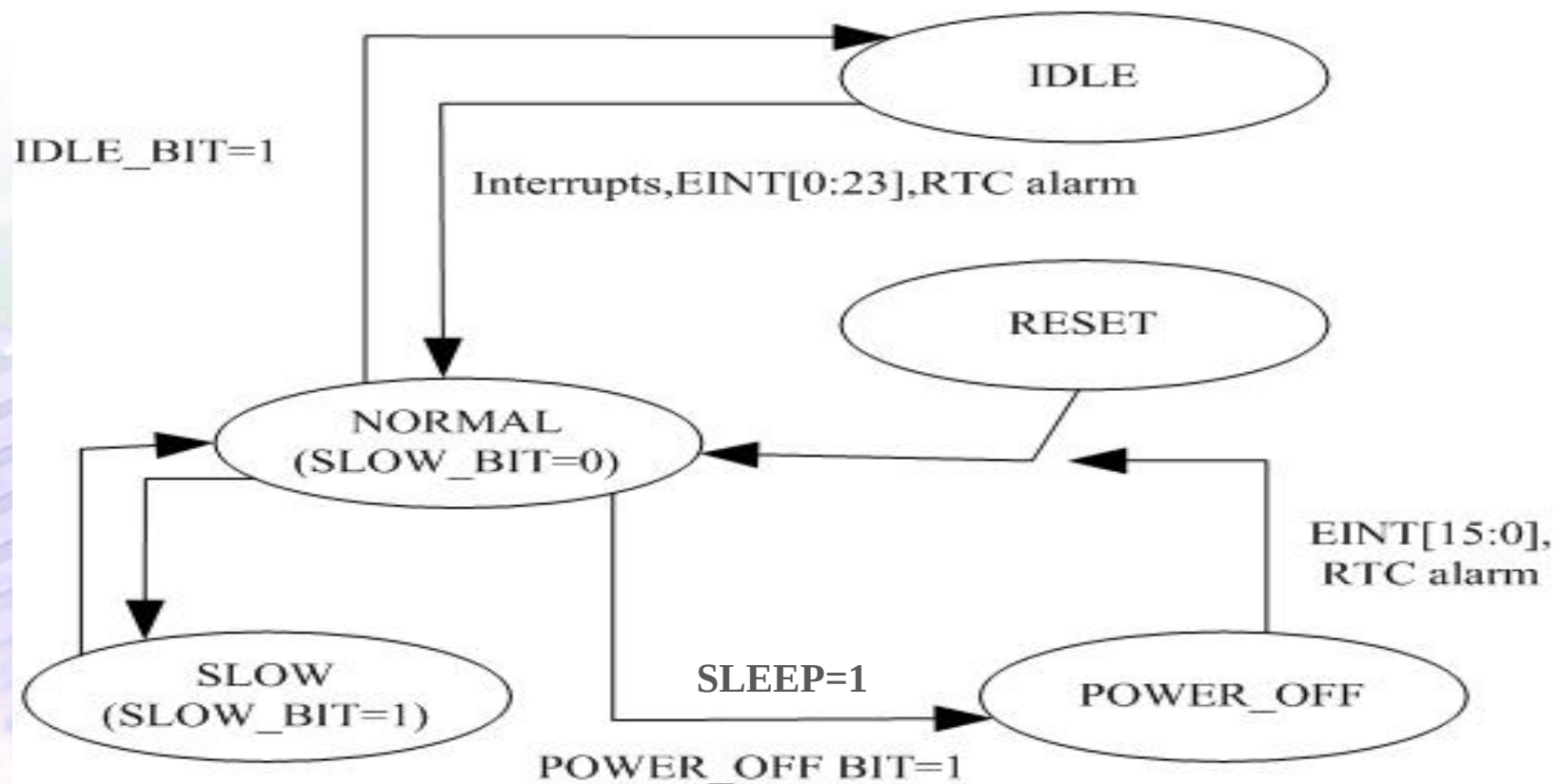
## 5.4 时钟和功率管理

### (7) 功率管理

功率管理模块通过软件来控制系统模式，以降低 S3C2440 的功耗。这些方案与 PLL、时钟控制逻辑 和 唤醒信号有关。

## 5.4 时钟和功率管理

S3C2440 有 4种 电源模式，在模式之间的切换 不是没有限制地允许的，有效的模式切换如下图所示：



## 5.4 时钟和功率管理

### 不同电源模式下的时钟电源管理：

| 模式        | ARM920T | AHB 模块<br>/WDT | 电源管理       | GPIO | 32.768KHZ<br>RTC时钟 | APB 模块&<br>USBH/LCD/<br>NAND |
|-----------|---------|----------------|------------|------|--------------------|------------------------------|
| NORMAL    | ○       | ○              | ○          | SEL  | ○                  | SEL                          |
| IDLE      | ×       | ○              | ○          | SEL  | ○                  | SEL                          |
| SLOW      | ○       | ○              | ○          | SEL  | ○                  | SEL                          |
| POWER_OFF | OFF     | OFF            | 等待唤醒<br>事件 | 原状态  | ○                  | SEL                          |

SEL：通过编程选择○或×

○：使能； ×：禁止； OFF：电源关断。



## 5.4 时钟和功率管理

### ①正常模式

在正常模式下，**所有的外设 和 基本模块**，包括：**电源管理模块、CPU核、总线控制器、寄存器控制器、中断控制器和DMA**，都能完全正常工作。但是，除基本模块外，可以通过软件设置对每一个外设时钟进行选择性的停止，以降低功耗。

## 5.4 时钟和功率管理

### ②空闲模式

在空闲模式下，**仅停止** 为 **CPU** 提供时钟信号，**只对**：总线控制器、**存储器控制器** 和 **电源管理模式** 等外设提供时钟。如果要退出空闲模式，则 **EINT[23:0]** 或者 **RTC** 告警中断，或者其他的中断必须处于活动状态（如果想使用 **EINT**，则 **GPIO** 模块在启动前必须是开启的）。

## 5.4 时钟和功率管理

### ③低速模式

低速模式是 **非PLL模式**。与正常模式不同的是：**低速模式没有通过 PLL电路**。而是直接使用了 **一个外部时钟（ETIpII或者 EXTCLK）** 作为 **FCLK** 信号。在此模式中，电源的功耗仅取决于外部时钟的频率，将 **PLL** 自身的耗电排除在外。

## 5.4 时钟和功率管理

在低速模式，可以通过 一个低速模式和把 PLL 自身的耗电量 排除在外来降低电源损耗。FCLK 是没有 PLL 时，将  $F_{inN}$  等分后的频率。分割的比率是由 CLKSLW 和 CLKDIVN 控制寄存器中的 SLOW\_VAL 决定的。下表列出了 SLOW模式下 CLKSLW 和 CLKDIVN 控制寄存器的设置。

| SLOW_VAL | FCLK                 | HCLK                        |                             | PCLK                       |                            | UCLK  |
|----------|----------------------|-----------------------------|-----------------------------|----------------------------|----------------------------|-------|
|          |                      | 1/1<br>Option<br>(HDIVN=00) | 1/2<br>Option<br>(HDIVN=01) | 1/1<br>Option<br>(PDIVN=0) | 1/2<br>Option<br>(PDIVN=1) |       |
| 000      | EXTCLK或<br>XTIp11/1  | EXTCLK或<br>XTIp11/1         | EXTCLK或<br>XTIp11/2         | HCLK                       | HCLK/2                     | 48MHZ |
| 001      | EXTCLK或<br>XTIp11/2  | EXTCLK或<br>XTIp11/2         | EXTCLK或<br>XTIp11/4         | HCLK                       | HCLK/2                     | 48MHZ |
| 010      | EXTCLK或<br>XTIp11/4  | EXTCLK或<br>XTIp11/4         | EXTCLK或<br>XTIp11/8         | HCLK                       | HCLK/2                     | 48MHZ |
| 011      | EXTCLK或<br>XTIp11/6  | EXTCLK或<br>XTIp11/6         | EXTCLK或<br>XTIp11/12        | HCLK                       | HCLK/2                     | 48MHZ |
| 100      | EXTCLK或<br>XTIp11/8  | EXTCLK或<br>XTIp11/8         | EXTCLK或<br>XTIp11/16        | HCLK                       | HCLK/2                     | 48MHZ |
| 101      | EXTCLK或<br>XTIp11/10 | EXTCLK或<br>XTIp11/10        | EXTCLK或<br>XTIp11/20        | HCLK                       | HCLK/2                     | 48MHZ |
| 110      | EXTCLK或<br>XTIp11/12 | EXTCLK或<br>XTIp11/12        | EXTCLK或<br>XTIp11/24        | HCLK                       | HCLK/2                     | 48MHZ |
| 111      | EXTCL或<br>XTIp11/14  | EXTCLK或<br>XTIp11/14        | EXTCLK或<br>XTIp11/28        | HCLK                       | HCLK/2                     | 48MHZ |

## 5.4 时钟和功率管理

### ④掉电模式

断开内部电源，所以 CPU 和 内部逻辑没有电源消耗，除了在此模式下唤醒的逻辑。激活掉电模式需要 两个独立的电源，一个为唤醒逻辑供电，另一个为其它的内部逻辑包括CPU供电，并且可以被电源开关控制。在掉电模式下，为 CPU 和 内部逻辑供电的第二个电源将被关掉。退出掉电模式，可以通过 EINT[15:0] 或者 RTC 告警中断发出。



## 5.4 时钟和功率管理

进入掉电模式的过程如下（了解）：

- A. 设置 GPIO 配置；
- B. 在 INTMSK 寄存器中屏蔽所有中断；
- C. 配置适当的唤醒电源，包括：RTC报警。唤醒源相关的 EINTMSK 位不必被屏蔽。
- D. 设置 USB 为挂起模式  
（杂项控制寄存器 MISCCR[13:12]=11b）；

## 5.4 时钟和功率管理

E. 将一些有用的值存入一般状态寄存器 **GSTATUS3、4**，这些寄存器在掉电模式下是被保持的；

F. 通过 **MISCCR[1:0]** 将数据总线D[31:0]的上拉电阻配置成断开（禁用）。如果有外部总线保持器，如 74VCH162245，则连接（启用）上拉电阻，否则断开上拉电阻；

G. 通过清除LCD控制寄存器 **LCDCON1.ENVID**位停止LCD；

## 5.4 时钟和功率管理

- H. 读取 `rREFRESH` 和 `rCLKCON`寄存器的值来填充页表缓存；
- I. 通过设置 `REFRESH[22]=1b`，使 SDRAM 进入自刷新模式（低功耗模式）；
- J. 等待，直到SDRAM 自刷新生效；
- K. 通过设置 `MISCCR[19:17]=111b`，使 SDRAM 信号在掉电模式期间被保护起来（`SCLK0`、`SCLK1`、`SCKE`）；
- L. 置位 `CLKCON` 寄存器的掉电模式位。

## 5.4 时钟和功率管理

从掉电模式唤醒的过程如下（了解）：

- A. 如果有唤醒源被触发，内部的复位信号就会动作。这和外部的 nReset 引脚触发非常相似；
- B. 查询 GSTATUS2[1] 位，是否是从掉电模式被唤醒而产生了boot操作；
- C. 通过将 MISCCR[19:17] 设置为 000b，释放 SDRAM 信号保护；

## 5.4 时钟和功率管理

D. 配置 SDRAM 控制器；

E. 等待 SDRAM 从自刷新模式释放；

F. GSTATUS3/4 的信息可以被用户使用，因为 GSTATUS3/4 的值已经在掉电模式下被保存了；

G. 对于 EINT[3:0]，检查 SRCPND 寄存器；对于 EINT[15:4]，检查 EINTPND 寄存器；对于 RTC 报警唤醒，检查 RTC 时间，因为在唤醒时 SRCPND 寄存器的 RTC 位不被置位。



## 5.5 DMA

### 5.5.1 DMA简介

S3C2440 拥有 4个通道 的 DMA 控制器，  
相连于 系统总线 和 外围总线。DMA控制器的  
每一个通道可以在：系统总线上的存储器与外  
围总线之间、系统总线上的设备之间，以及外  
围总线设备之间操作数据传输而不被限制。

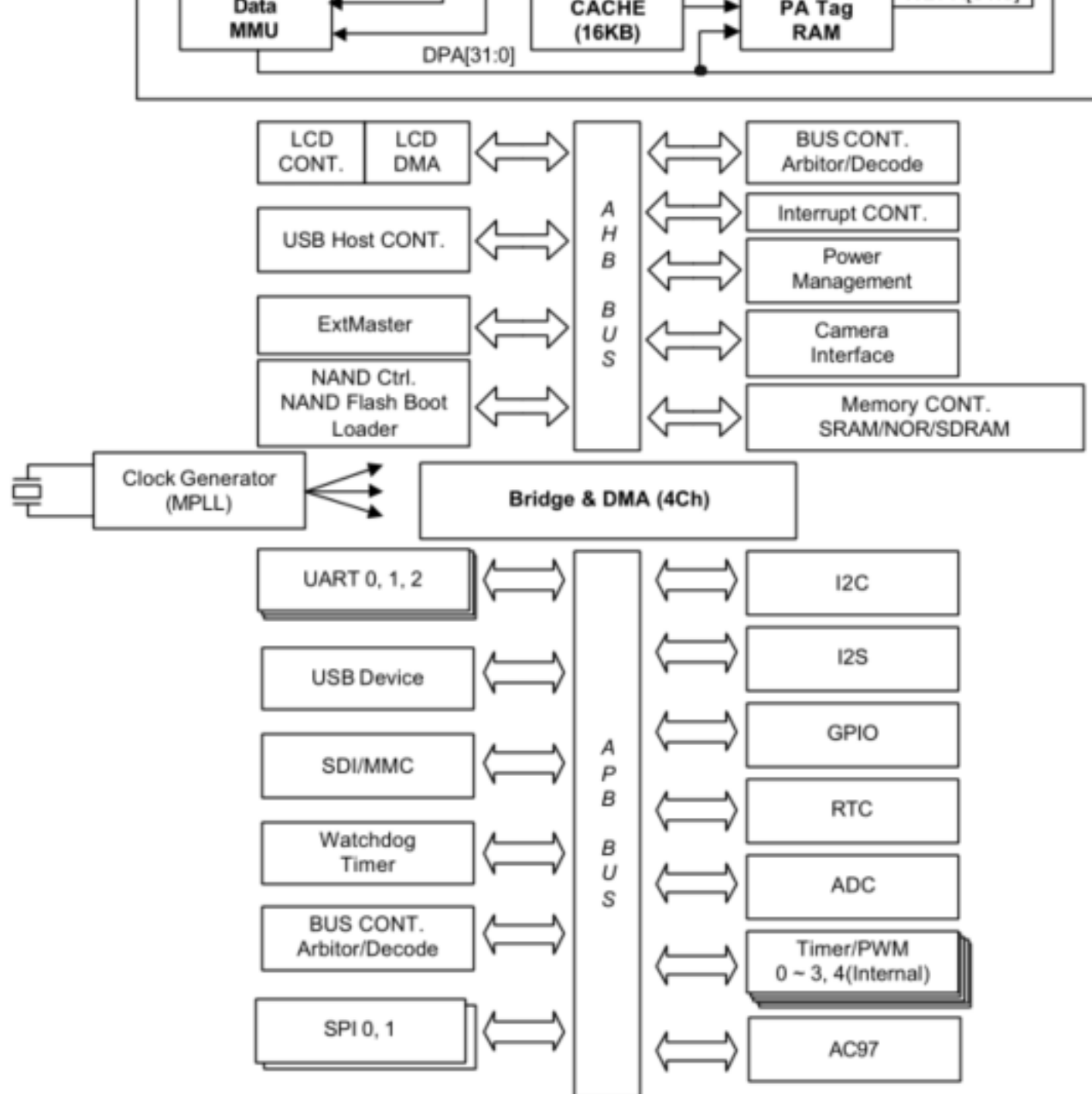
S3C2440 的 DMA 在以下 四种情况 可运行：



## 5.5 DMA

- (1) 源设备和目标都在系统总线AHB上；
- (2) 源设备和目标都在外围总线APB上；
- (3) 源设备在系统总线，而目标设备位于外围总线；
- (4) 源设备在外围总线，而目标设备位于系统总线。

DMA的主要优点是：可以不通过 CPU 的中断来实现数据的传输，DMA 的运行可以通过软件或者通过外围设备的中断和请求来初始化。



## 5.5.2 DMA工作原理

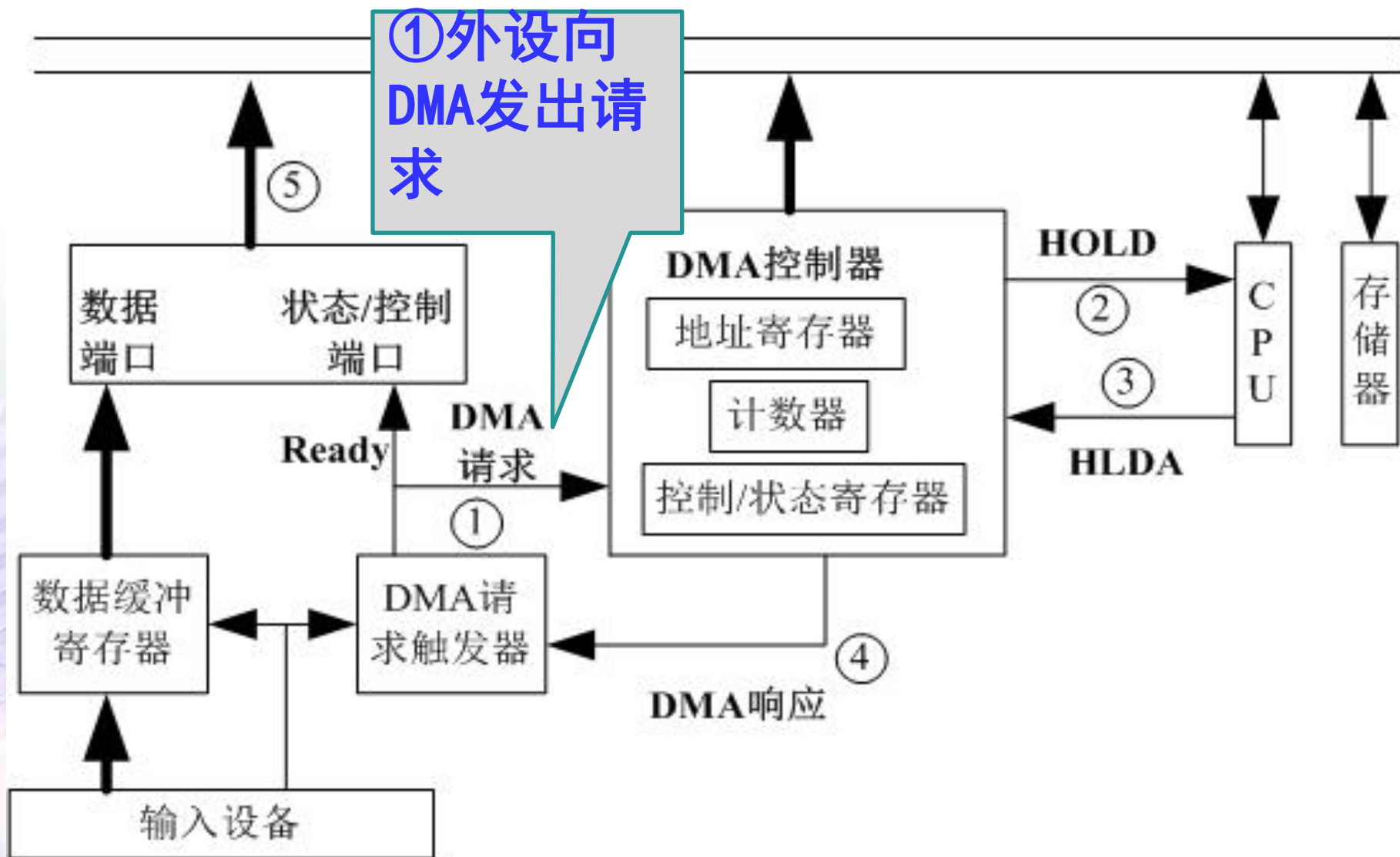
### (1) DMA的服务对象

每个 DMA通道 都有 多个DMA请求源，通过设置，可以从中挑选一个服务。每个通道的DMA 请求源如下表所示：

|      | Source0 | Source1 | Source2 | Source3 | Source4        | Source5 | Source6 |
|------|---------|---------|---------|---------|----------------|---------|---------|
| Ch-0 | nXDREQ0 | UART0   | SDI     | Timer   | USB device EP1 | I2SSDO  | PCMIN   |
| Ch-1 | nXDREQ1 | UART1   | I2SSDI  | SPI0    | USB device EP2 | PCMOUT  | SDI     |
| Ch-2 | I2SSDO  | I2SSDI  | SDI     | Timer   | USB device EP3 | PCMIN   | MICIN   |
| Ch-3 | UART2   | SDI     | SPI1    | Timer   | USB device EP4 | MICIN   | PCMOUT  |

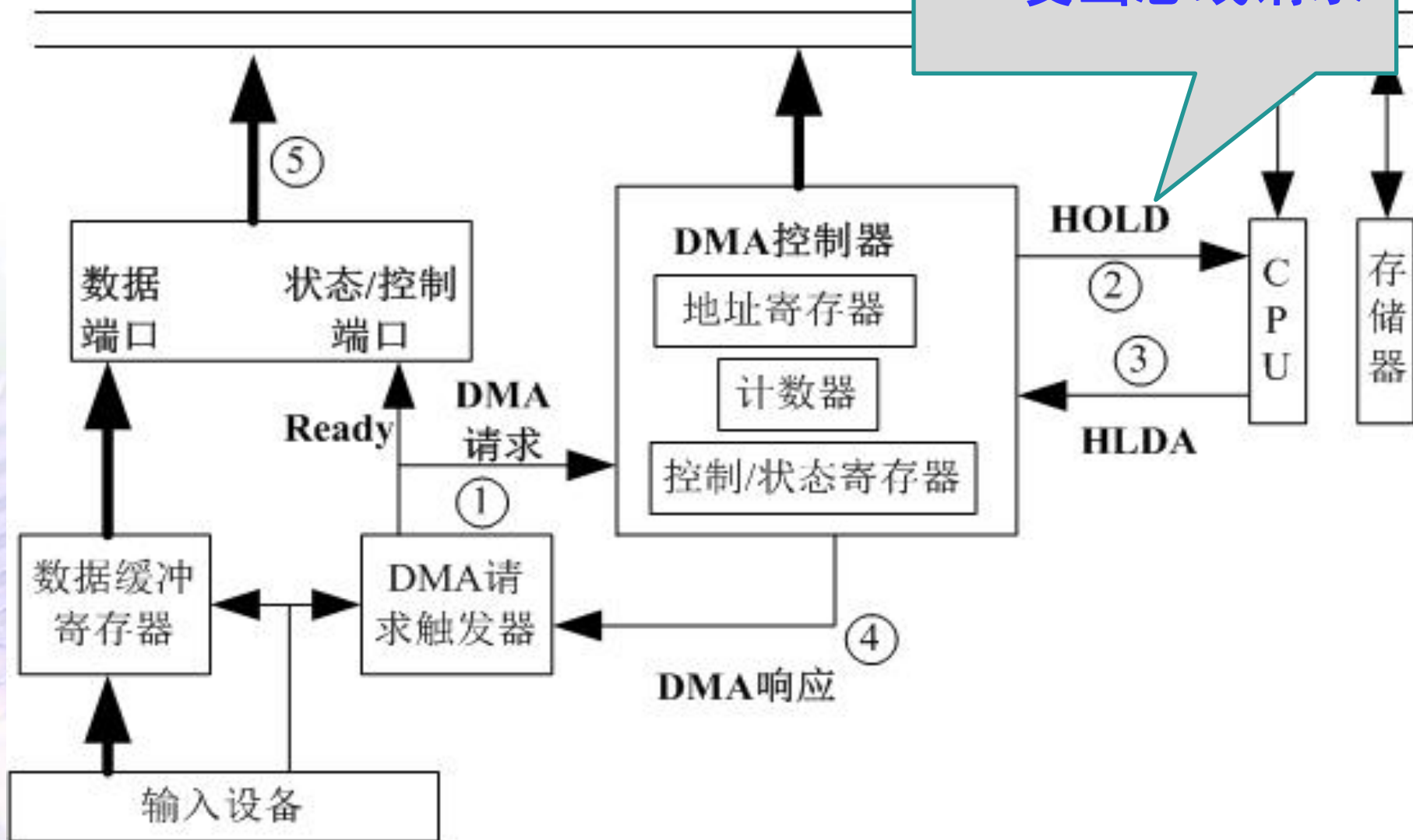
这里 nXDREQ0 和 nXDREQ1 表示两个外部源（外部设备），I2SSDO 和 I2SSDI 分别表示 IIS 传输和接收。

## (2) DMA的工作过程

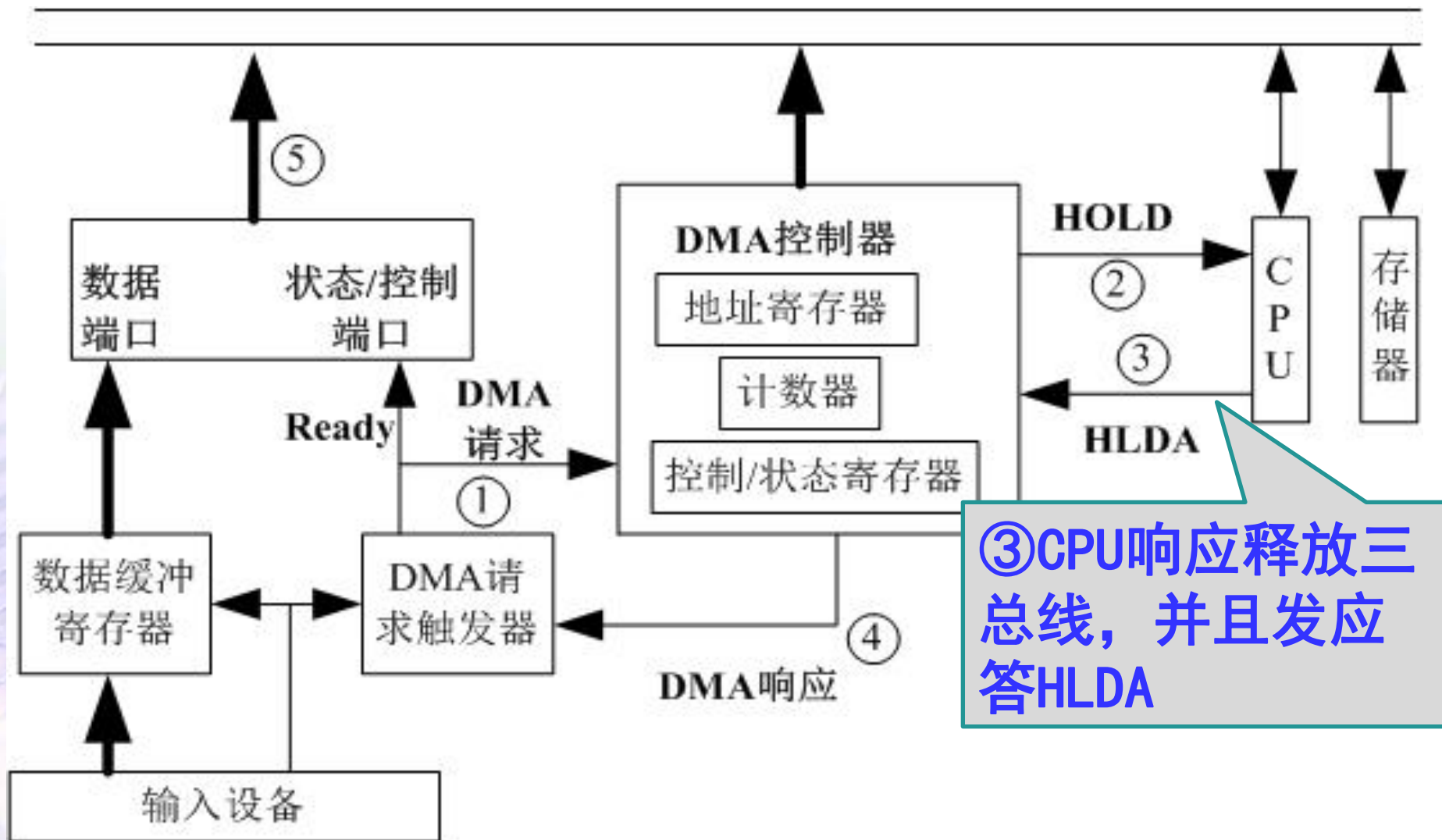


## (2) DMA的工作过程

②DMA通过HOLD向CPU发出总线请求



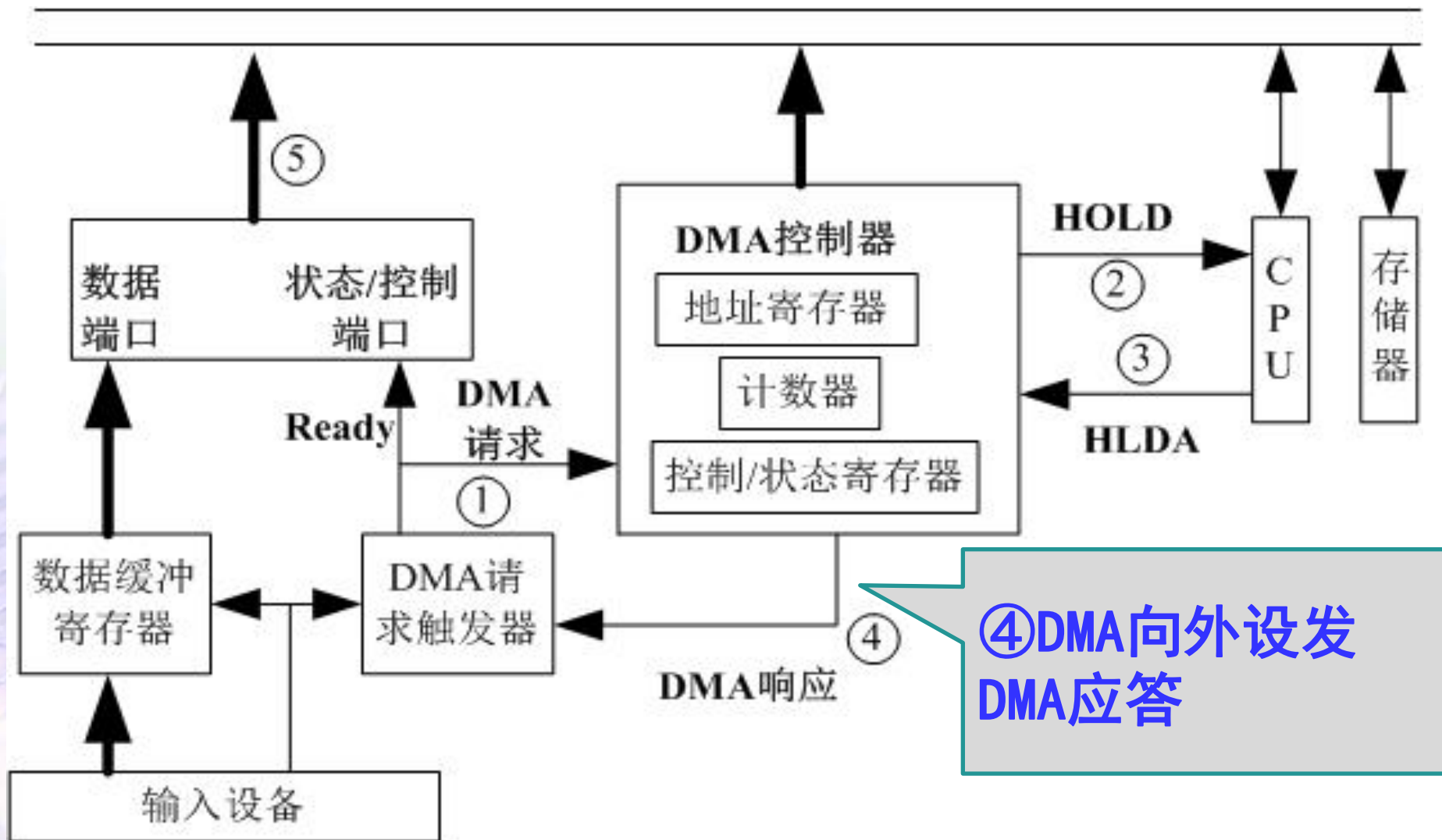
## (2) DMA的工作过程



③CPU响应释放三总线，并且发应答HLDA

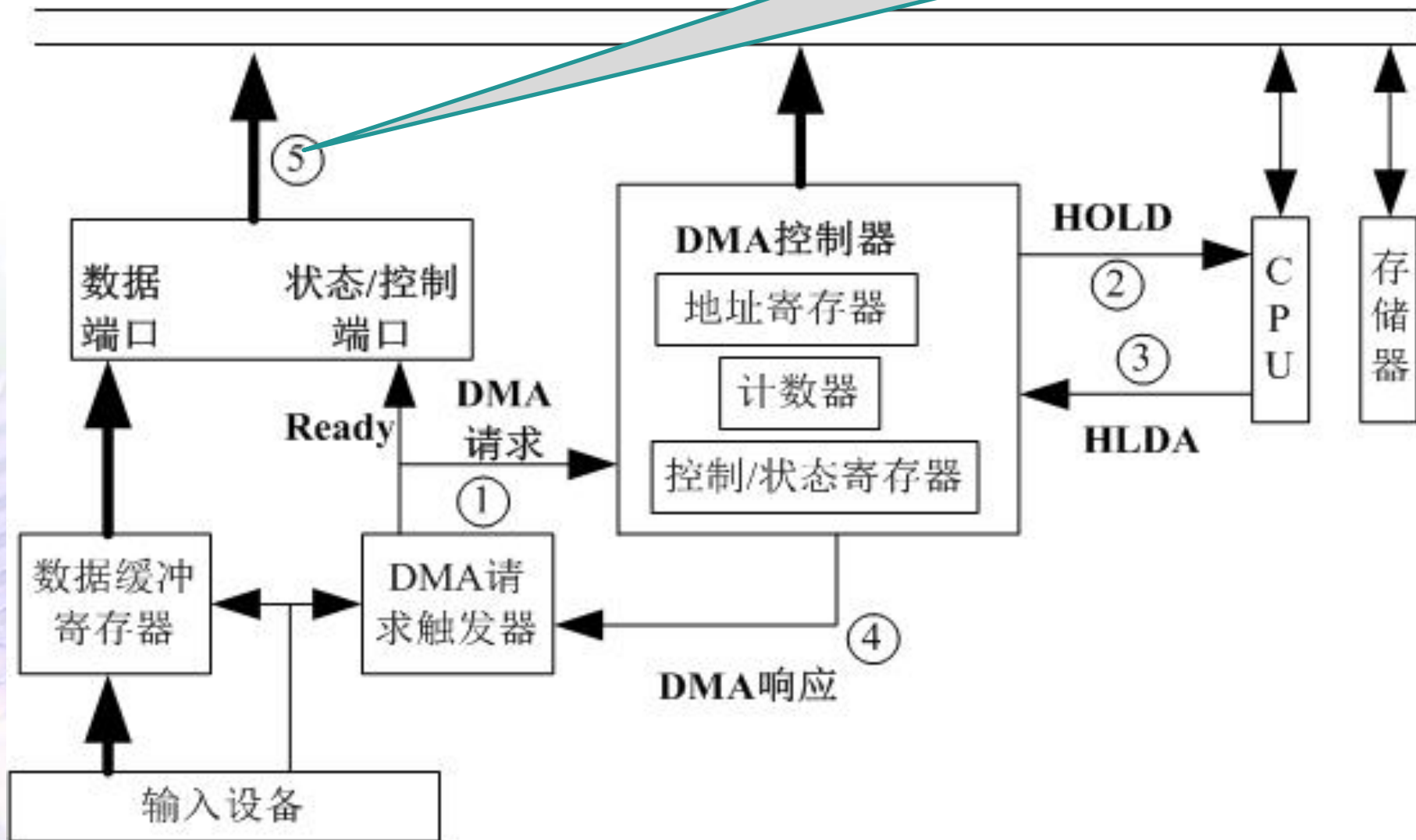


## (2) DMA的工作过程



## (2) DMA的工作过程

⑤DMA发出地址、控制信号，为外设传送数据



## 5.5.2 DMA工作原理

传送完规定的数据后，DMA 撤销 HOLD 信号，CPU 也撤销 HLDA 信号，并且恢复对三条总线的控制。

S3C2440 的 DMA 工作过程可以分为以下三个状态：

### ①状态1——等待状态

DMA 等待一个 DMA请求。如果有请求到来，将转移到 状态2。在这个状态下，DMA ACK 和 INT REQ 为 0。

## 5.5.2 DMA工作原理

### ②状态2——准备状态

DMA ACK 变为 1，计数器（CURR\_TC）从 DCON[19:0] 寄存器装载计数值。注意：DMA ACK保持为 1 直到它被清除。

### ③状态3——传输状态

DMA 控制器从 源地址 读入数据并将它写到 目的地址，每传输 一次，CURR\_TC 计数器减1，并且可能做以下操作：

## 5.5.2 DMA工作原理

- A. 重复传输：在全服务模式下，将重复传输，直到计数器 CURR\_TC变为0；在单服务模式下，仅传输一次；
- B. 设置中断请求信号：当 CURR\_TC 变为 0 时，而且 DCON[29] 即中断使能设定位被设为 1，则DMAC（DMA控制器）发出 INT REQ 信号；
- C. 清除 DMA ACK 信号：
  - 全服务模式 CURR\_TC 变为 0。
  - 对单服务模式，执行完该次传输操作。



## 5.5.2 DMA工作原理

**注意：**在单服务模式下，DMAC 的三个状态（等待、准备和传输）被执行一遍，然后停止，等待下一个 DMA REQ 的到来。如果 DMA REQ 到来，则这些状态被重复操作，直到 CURR\_TC 减为 0。

不管服务模式如何，一旦计数器（CURR\_TC）变成 0，中断请求信号 INT REQ 发出。



## 5.5.2 DMA工作原理

### (3) 外部DMA请求/响应协议

DMA有 三种类型 的 外部DMA请求/响应 规则：

①single service demand, 单服务请求（对应于需求模式）；

②single service handshake, 单服务握手（握手模式）；

③whole service handshake, 全服务握手（全服务模式）。

## 5.5.2 DMA工作原理

每种类型都定义了像DMA请求和DMA响应这些信号怎样与这些规则相联系。

请求/握手（demand与handshake）模式的比较如下：

在一次传输结束时，DMA检查xnxDREQ（DMA请求）信号的状态。

A. 在需求模式（单服务请求）下：如果DMA请求（xnxDREQ）信号仍然有效，则传输马上再次开始，否则等待；

## 5.5.2 DMA工作原理

**B. 在握手模式下：如果 DMA 请求信号无效，DMA 在2个时钟周期后将 DMA 响应（`xnxDACK`）信号变得无效，否则 DMA 等待直到 DMA 请求信号变得无效，每请求一次传输一次。**

## 5.6 中断控制器

**S3C2440X 中断控制器有 56个 中断源，对外提供 24个 外中断输入引脚，内部所有设备都有中断请求信号，例如：DMA控制器、UART、IIC等。**

**S3C2440X 的 ARM920T 内核有两个中断：IRQ中断 和 快速中断FIQ。**

**中断仲裁：当中断控制器接收到 多个中断请求时，其内的优先级仲裁器裁决后向 CPU 发出优先级最高的中断请求信号或快速中断请求信号。**

## 5.6.1 中断控制器的操作

### (1) 当前程序状态寄存器（CPSR）的 F位 和 I位

CPSR 指 ARM920T 处理器的程序状态寄存器。如果 CPSR 的 F位 被设置为 1，处理器将不接受来自中断控制器的FIQ（快速中断请求）；如果 PSR 的 I位 被设置为 1，处理器将不接受来自中断控制器的IRQ（中断请求）。因此，为了使能 FIQ 和 IRQ，PSR 的 F位 或 I位 必须被 清零，同时中断屏蔽寄存器 INTMASK 的相应位也必须被 清0。



## 5.6.1 中断控制器的操作

### (2) 中断模式

**ARM920T** 有两种类型的中断模式（**INTMOD**），即**FIQ**（快速中断请求）和**IRQ**（普通中断模式）。所有的中断源在中断请求时都要确定使用哪一种中断模式。

### (3) 中断挂起寄存器

**S3C2440** 有两个中断挂起寄存器：**源挂起寄存器（SRCPND）** 和 **中断挂起寄存器（INTPND）**。这些挂起寄存器指示了一个中断请求是否被挂起。



## 5.6.1 中断控制器的操作

- 当中断源请求中断服务时，SRCPND 相应位被置 1。
- 并且同时在仲裁后 INTPND 寄存器 仅有1位被置1。
- 如果中断被屏蔽，那么 SRCPND 相应位被置 1，但不能引起 INTPND 改变。
- 当 INTPND 的挂起位被设置时，只要相应的标志位I 或 标志位F 被清0，相应的中断服务程序都被执行。SRCPND 和 INTPND 可以被读写。

## 5.6.1 中断控制器的操作

### (4) 中断屏蔽寄存器 (INTMSK)

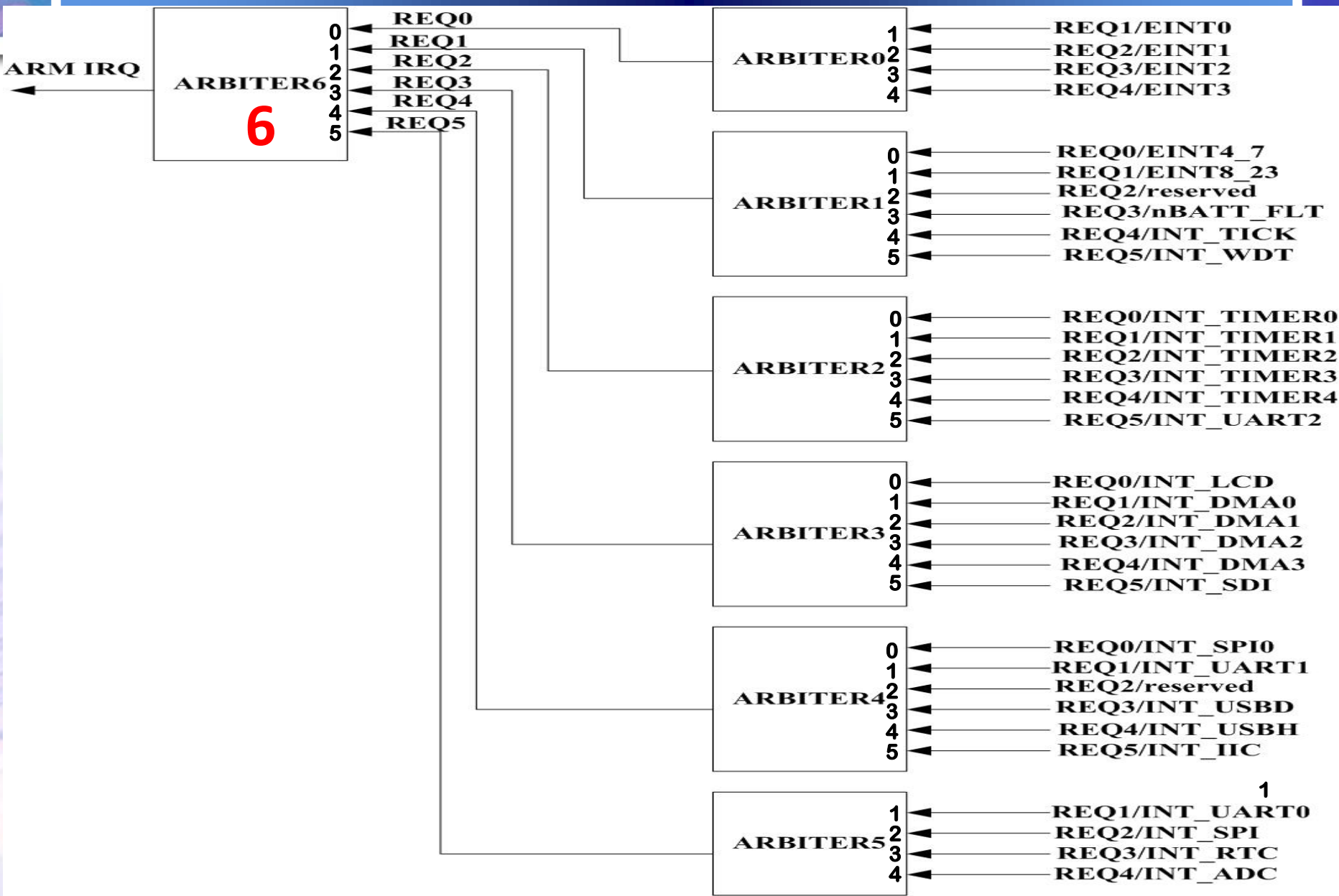
- 如果该寄存器的某一个位被 **置1**，则该位对应的中断响应被 **禁止** 了；
- 如果某个中断在 **INTMSK** 寄存器的对应位 **为0**，则这个中断发生时将会正常被响应。
- 如果某个中断在 **INTMSK** 的寄存器中的对应位 **为1**，但是这个中断发生了，它的**源挂起寄存器 (SRCPND)** 对应位还是会 **置1**。而对应**中断挂起寄存器 (INTPND)** 标志位，不会被置1。

## 5.6.2 S3C2440中断源

中断控制器提供 60 个 中断源，所以中断优先级非常重要。

### (1) 中断优先级仲裁器及工作原理

中断系统有 6个 分仲裁器 和 1个 总仲裁器，每一个仲裁器可以处理 6路 中断，优先级产生模块图如下：



## 5.6.2 S3C2440中断源

中断控制器提供60个中断源的管理。是如何实现的？

S3C2440将中断源分为两级：**中断源**和**子中断源（次级中断源）**：

- **中断源**就是上图32个中断源。而这32个中断源中包含单一中断源和复合中断源。
- **复合中断源**是**子中断源（次级中断源）**的复合信号。比如：INT\_UARTn。

这种设计，使得S3C2440中断控制器支持60个中断源。



## 5.6.2 S3C2440中断源

### (2) 中断优先级

每一个仲裁器可以控制 6个 中断请求，其基于由仲裁模式控制的 1位（ARB\_MODE） 和选择控制信号的2位（ARB\_SEL）。

①如果ARB\_SEL位为：00b，优先级次序为REQ0、REQ1、REQ2、REQ3、REQ4和REQ5；

②如果ARB\_SEL位为：01b，优先级次序为REQ0、REQ2、REQ3、REQ4、REQ1和REQ5；



## 5.6.2 S3C2440中断源

③如果ARB\_SEL位为：10b，优先级次序为REQ0、REQ3、REQ4、REQ1、REQ2和REQ5；

④如果ARB\_SEL位为：11b，优先级次序为REQ0、REQ4、REQ1、REQ2、REQ3和REQ5。

注意：仲裁器的 REQ0 为最高优先级，REQ5 为最低优先级；通过改变 ARB\_SEL 位的值，可以设定 REQ1~REQ4 优先级。

## 5.6.2 S3C2440中断源

- 这里，如果 ARB\_MODE位 设置为 0 ， ARB\_SEL 位不会自动改变，仲裁器会在固定的优先级模式中操作（即使在这个模式中，也可以通过手动改变 ARB\_SEL 位配置优先级）。
- 另一方面，如果 ARB\_MODE 位设置为 1， ARB\_SEL 位轮流改变。例如，如果 REQ1 被执行，为了把 REQ1 放在最低优先级， ARB\_SEL 位会被自动改为 01b。 ARB\_SEL 变化的详细规则如下：

## 5.6.2 S3C2440中断源

- ①如果REQ0或者REQ5被执行，ARB\_SEL不再改变；
- ②如果REQ1被执行，ARB\_SEL位改变为01b；
- ③如果REQ2被执行，ARB\_SEL位改变为10b；
- ④如果REQ3被执行，ARB\_SEL位改变为11b；
- ⑤如果REQ4被执行，ARB\_SEL位改变为00b。

## 5.6.3 中断控制器的特殊功能寄存器

中断控制器中包括 5个 控制寄存器：源挂起寄存器、中断模式寄存器、屏蔽寄存器、优先级寄存器、中断挂起寄存器。

来自中断的中断请求首先寄存在 源挂起寄存器。它们基于中断模式寄存器分为两组：FIQ中断请求 和 IRQ中断请求。对多重IRQ请求的仲裁过程使用优先级寄存器。

## 5.6.3 中断控制器的特殊功能寄存器

### (1) 源挂起寄存器

源挂起寄存器由 32位 组成，每一个比特位均与一个中断源相连。当中断源产生中断请求并且等待中断服务程序的执行时，其相应的比特位设置为 1 。

还有，源挂起寄存器指示此时哪一个中断源正在等待执行中断请求。注意：SRCPND寄存器的各比特位均为中断源自动设置，与屏蔽寄存器的相应位设置无关。



## 5.6.3 中断控制器的特殊功能寄存器

另外，SRCPND 寄存器不受中断控制寄存器的优先级逻辑的影响。

当中断服务被执行时，SRCPND 寄存器的相应位应该 **及时清0**，以便正确地接收相应中断源的中断请求。如果没有清除源挂起位，当中断服务返回时，中断控制寄存器会将之作为来自相同中断源的中断请求 **第二次执行**。也就是说，如果 SRCPND 寄存器的某位为 **1**，系统一般会当作一个有效的等待执行的中断请求。



## 5.6.3 中断控制器的特殊功能寄存器

清除相应位的时间依赖于用户的需求。如果要接收来自相同源的另一个有效中断请求，那么应该清除相应挂起位并且使能中断。

可以通过写入一个数据到此寄存器来清除SRCPND寄存器的指定位，这时只清除那些数据中被**设置为1**的相应位置的SRCPND位。那些数据中被设置为0的相应位置的位保持不变。

源挂起寄存器 SRCPND 的地址等信息如下所示：

| 寄存器    | 地址         | R/W | 描述                                  | 复位值 |
|--------|------------|-----|-------------------------------------|-----|
| SRCPND | 0X4A000000 | R/W | 指示中断请求状态：<br>0：没有中断请求；<br>1：产生中断请求。 | 0   |

## SRCPND 寄存器中各比特位的描述如下所示：

| SRCPND    | 位    | 描述         | 初始值 |
|-----------|------|------------|-----|
| INT_ADC   | [31] | 0：无请求；1：请求 | 0   |
| INT_RTC   | [30] | 0：无请求；1：请求 | 0   |
| INT_SPI1  | [29] | 0：无请求；1：请求 | 0   |
| INT_UART0 | [28] | 0：无请求；1：请求 | 0   |
| INT_IIC   | [27] | 0：无请求；1：请求 | 0   |
| INT_USBH  | [26] | 0：无请求；1：请求 | 0   |
| INT_USBD  | [25] | 0：无请求；1：请求 | 0   |
| INT_NFCON | [24] | 0：无请求；1：请求 | 0   |

| SRCPND     | 位    | 描述            | 初始值 |
|------------|------|---------------|-----|
| INT_UART1  | [23] | 0: 无请求; 1: 请求 | 0   |
| INT_SPI0   | [22] | 0: 无请求; 1: 请求 | 0   |
| INT_SDI    | [21] | 0: 无请求; 1: 请求 | 0   |
| INT_DMA3   | [20] | 0: 无请求; 1: 请求 | 0   |
| INT_DMA2   | [19] | 0: 无请求; 1: 请求 | 0   |
| INT_DMA1   | [18] | 0: 无请求; 1: 请求 | 0   |
| INT_DMA0   | [17] | 0: 无请求; 1: 请求 | 0   |
| INT_LCD    | [16] | 0: 无请求; 1: 请求 | 0   |
| INT_UART2  | [15] | 0: 无请求; 1: 请求 | 0   |
| INT_TIMER4 | [14] | 0: 无请求; 1: 请求 | 0   |
| INT_TIMER3 | [13] | 0: 无请求; 1: 请求 | 0   |
| INT_TIMER2 | [12] | 0: 无请求; 1: 请求 | 0   |
| INT_TIMER1 | [11] | 0: 无请求; 1: 请求 | 0   |
| INT_TIMER0 | [10] | 0: 无请求; 1: 请求 | 0   |
| INT_WDT    | [9]  | 0: 无请求; 1: 请求 | 0   |

| SRCPND    | 位   | 描述            | 初始值 |
|-----------|-----|---------------|-----|
| INT_TICK  | [8] | 0: 无请求; 1: 请求 | 0   |
| nBATT_FLT | [7] | 0: 无请求; 1: 请求 | 0   |
| INT_CAM   | [6] | 0: 无请求; 1: 请求 | 0   |
| EINT8_23  | [5] | 0: 无请求; 1: 请求 | 0   |
| EINT4_7   | [4] | 0: 无请求; 1: 请求 | 0   |
| EINT3     | [3] | 0: 无请求; 1: 请求 | 0   |
| EINT2     | [2] | 0: 无请求; 1: 请求 | 0   |
| EINT1     | [1] | 0: 无请求; 1: 请求 | 0   |
| EINT0     | [0] | 0: 无请求; 1: 请求 | 0   |

## 5.6.3 中断控制器的特殊功能寄存器

### (2) 中断模式寄存器

中断模式寄存器 **INTMOD** 的 **32个** 比特位分别与一个中断源相对应。如果某位被设置为 **1**，则相应的中断会以 **FIQ** 的模式来处理。相反，以 **IRQ** 的模式来处理。

**注意：**中断控制器中 仅有一个中断源 可以由 **FIQ** 模式来处理（只紧急中断使用**FIQ**中断），所以 **INTMOD** 中**只有一个比特位**可以被**设置为 1**。

## 5.6.3 中断控制器的特殊功能寄存器

中断模式寄存器 INTMOD 的地址等信息如下所示：

| 寄存器    | 地址         | R/W | 描述                               | 复位值 |
|--------|------------|-----|----------------------------------|-----|
| INTMOD | 0X4A000004 | R/W | 中断模式寄存器：<br>0：IRQ模式；<br>1：FIQ模式。 | 0   |

INTMOD 寄存器中各比特位的描述如下所示：



| INTMOD    | 位    | 描述             | 初始值 |
|-----------|------|----------------|-----|
| INT_ADC   | [31] | 0: IRQ; 1: FIQ | 0   |
| INT_RTC   | [30] | 0: IRQ; 1: FIQ | 0   |
| INT_SPI1  | [29] | 0: IRQ; 1: FIQ | 0   |
| INT_UART0 | [28] | 0: IRQ; 1: FIQ | 0   |
| INT_IIC   | [27] | 0: IRQ; 1: FIQ | 0   |
| INT_USBH  | [26] | 0: IRQ; 1: FIQ | 0   |
| INT_USBD  | [25] | 0: IRQ; 1: FIQ | 0   |
| INT_NFCON | [24] | 0: IRQ; 1: FIQ | 0   |
| INT_UART1 | [23] | 0: IRQ; 1: FIQ | 0   |
| INT_SPI0  | [22] | 0: IRQ; 1: FIQ | 0   |
| INT_SDI   | [21] | 0: IRQ; 1: FIQ | 0   |
| INT_DMA3  | [20] | 0: IRQ; 1: FIQ | 0   |
| INT_DMA2  | [19] | 0: IRQ; 1: FIQ | 0   |
| INT_DMA1  | [18] | 0: IRQ; 1: FIQ | 0   |
| INT_DMA0  | [17] | 0: IRQ; 1: FIQ | 0   |

| INTMOD     | 位    | 描述             | 初始值 |
|------------|------|----------------|-----|
| INT_LCD    | [16] | 0: IRQ; 1: FIQ | 0   |
| INT_UART2  | [15] | 0: IRQ; 1: FIQ | 0   |
| INT_TIMER4 | [14] | 0: IRQ; 1: FIQ | 0   |
| INT_TIMER3 | [13] | 0: IRQ; 1: FIQ | 0   |
| INT_TIMER2 | [12] | 0: IRQ; 1: FIQ | 0   |
| INT_TIMER1 | [11] | 0: IRQ; 1: FIQ | 0   |
| INT_TIMER0 | [10] | 0: IRQ; 1: FIQ | 0   |
| INT_WDT    | [9]  | 0: IRQ; 1: FIQ | 0   |
| INT_TICK   | [8]  | 0: IRQ; 1: FIQ | 0   |
| nBATT_FLT  | [7]  | 0: IRQ; 1: FIQ | 0   |
| INT_CAM    | [6]  | 0: IRQ; 1: FIQ | 0   |
| EINT8_23   | [5]  | 0: IRQ; 1: FIQ | 0   |
| EINT4_7    | [4]  | 0: IRQ; 1: FIQ | 0   |
| EINT3      | [3]  | 0: IRQ; 1: FIQ | 0   |
| EINT2      | [2]  | 0: IRQ; 1: FIQ | 0   |

| INTMOD | 位   | 描述             | 初始值 |
|--------|-----|----------------|-----|
| EINT1  | [1] | 0: IRQ; 1: FIQ | 0   |
| EINT0  | [0] | 0: IRQ; 1: FIQ | 0   |

注意：如果 **INTMOD** 寄存器中某中断模式设置为 **FIQ**，那么 **FIQ** 中断将不会影响 **INTPND** 和 **INTOFFSET** 寄存器，两者只对 **IRQ** 模式的中断源有效。

## 5.6.3 中断控制器的特殊功能寄存器

### (3) 中断屏蔽寄存器

中断屏蔽寄存器 INTMSK 也包括了32位，每一个比特位均与相应的一个中断源相对应。如果某位被设置为1，那么CPU不会执行相应中断源提出的中断请求（即使源挂起寄存器的相应位被设置为1）。如果屏蔽位为0，那么中断请求会被正常执行。

中断屏蔽寄存器 INTMSK 的地址等信息如下所示：

| 寄存器    | 地址         | R/W | 描述                                              | 复位值 |
|--------|------------|-----|-------------------------------------------------|-----|
| INTMSK | 0X4A000008 | R/W | 决定屏蔽的中断源，屏蔽的中断源将不会执行。<br>0：中断服务有效；<br>1：中断服务屏蔽。 | 0   |

## 5.6.3 中断控制器的特殊功能寄存器

**INTMSK** 寄存器中各个比特位的描述如下所示：

| INTMSK    | 位    | 描述        | 初始值 |
|-----------|------|-----------|-----|
| INT_ADC   | [31] | 0：有效；1：屏蔽 | 1   |
| INT_RTC   | [30] | 0：有效；1：屏蔽 | 1   |
| INT_SPI1  | [29] | 0：有效；1：屏蔽 | 1   |
| INT_UART0 | [28] | 0：有效；1：屏蔽 | 1   |
| INT_IIC   | [27] | 0：有效；1：屏蔽 | 1   |
| INT_USBH  | [26] | 0：有效；1：屏蔽 | 1   |
| INT_USBD  | [25] | 0：有效；1：屏蔽 | 1   |
| INT_NFCON | [24] | 0：有效；1：屏蔽 | 1   |
| INT_UART1 | [23] | 0：有效；1：屏蔽 | 1   |
| INT_SPI0  | [22] | 0：有效；1：屏蔽 | 1   |

| INTMSK     | 位    | 描述           | 初始值 |
|------------|------|--------------|-----|
| INT_SDI    | [21] | 0: 有效; 1: 屏蔽 | 1   |
| INT_DMA3   | [20] | 0: 有效; 1: 屏蔽 | 1   |
| INT_DMA2   | [19] | 0: 有效; 1: 屏蔽 | 1   |
| INT_DMA1   | [18] | 0: 有效; 1: 屏蔽 | 1   |
| INT_DMA0   | [17] | 0: 有效; 1: 屏蔽 | 1   |
| INT_LCD    | [16] | 0: 有效; 1: 屏蔽 | 1   |
| INT_UART2  | [15] | 0: 有效; 1: 屏蔽 | 1   |
| INT_TIMER4 | [14] | 0: 有效; 1: 屏蔽 | 1   |
| INT_TIMER3 | [13] | 0: 有效; 1: 屏蔽 | 1   |
| INT_TIMER2 | [12] | 0: 有效; 1: 屏蔽 | 1   |
| INT_TIMER1 | [11] | 0: 有效; 1: 屏蔽 | 1   |
| INT_TIMER0 | [10] | 0: 有效; 1: 屏蔽 | 1   |
| INT_WDT    | [9]  | 0: 有效; 1: 屏蔽 | 1   |
| INT_TICK   | [8]  | 0: 有效; 1: 屏蔽 | 1   |



| INTMSK    | 位   | 描述           | 初始值 |
|-----------|-----|--------------|-----|
| nBATT_FLT | [7] | 0: 有效; 1: 屏蔽 | 1   |
| INT_CAM   | [6] | 0: 有效; 1: 屏蔽 | 1   |
| EINT8_23  | [5] | 0: 有效; 1: 屏蔽 | 1   |
| EINT4_7   | [4] | 0: 有效; 1: 屏蔽 | 1   |
| EINT3     | [3] | 0: 有效; 1: 屏蔽 | 1   |
| EINT2     | [2] | 0: 有效; 1: 屏蔽 | 1   |
| EINT1     | [1] | 0: 有效; 1: 屏蔽 | 1   |
| EINT0     | [0] | 0: 有效; 1: 屏蔽 | 1   |

## 5.6.3 中断控制器的特殊功能寄存器

### (4) 优先级控制寄存器

优先级控制寄存器 **PRIORITY** 的地址等信息如下所示：

| 寄存器      | 地址         | R/W | 描述          | 复位值  |
|----------|------------|-----|-------------|------|
| PRIORITY | 0X4A00000C | R/W | IRQ优先级控制寄存器 | 0X7F |

**PRIORITY** 寄存器中各个比特位的描述如下所示：

| PRIORITY         | 位       | 描述                                                                                                     | 初始值 |
|------------------|---------|--------------------------------------------------------------------------------------------------------|-----|
| ARB_SEL6<br>总仲裁器 | [20:19] | Arbiter6优先级顺序设置：<br>00: REQ 0-1-2-3-4-5; 01: REQ 0-2-3-4-1-5; 10: REQ 0-3-4-1-2-5; 11: REQ 0-4-1-2-3-5 | 0   |
| ARB_SEL5         | [18:17] | Arbiter5优先级顺序设置：<br>00: REQ 1-2-3-4; 01: REQ 2-3-4-1; 10: REQ 3-4-1-2; 11: REQ 4-1-2-3                 | 0   |
| ARB_SEL4         | [16:15] | Arbiter4优先级顺序设置：<br>00: REQ 0-1-2-3-4-5; 01: REQ 0-2-3-4-1-5; 10: REQ 0-3-4-1-2-5; 11: REQ 0-4-1-2-3-5 | 0   |
| ARB_SEL3         | [14:13] | Arbiter3优先级顺序设置：<br>00: REQ 0-1-2-3-4-5; 01: REQ 0-2-3-4-1-5; 10: REQ 0-3-4-1-2-5; 11: REQ 0-4-1-2-3-5 | 0   |

| PRIORITY  | 位       | 描述                                                                                                     | 初始值 |
|-----------|---------|--------------------------------------------------------------------------------------------------------|-----|
| ARB_SEL2  | [12:11] | Arbiter2优先级顺序设置：<br>00: REQ 0-1-2-3-4-5; 01: REQ 0-2-3-4-1-5; 10: REQ 0-3-4-1-2-5; 11: REQ 0-4-1-2-3-5 | 0   |
| ARB_SEL1  | [10:9]  | Arbiter1优先级顺序设置：<br>00: REQ 0-1-2-3-4-5; 01: REQ 0-2-3-4-1-5; 10: REQ 0-3-4-1-2-5; 11: REQ 0-4-1-2-3-5 | 0   |
| ARB_SEL0  | [8:7]   | Arbiter0优先级顺序设置：<br>00: REQ 1-2-3-4; 01: REQ 2-3-4-1; 10: REQ 3-4-1-2; 11: REQ 4-1-2-3                 | 0   |
| ARB_MODE6 | [6]     | Arbiter6优先级转换使能：<br>0: 优先级不自动转换; 1: 优先级自动转换                                                            | 1   |
| ARB_MODE5 | [5]     | Arbiter5优先级转换使能：<br>0: 优先级不自动转换; 1: 优先级自动转换                                                            | 1   |

| PRIORITY  | 位   | 描述                                       | 初始值 |
|-----------|-----|------------------------------------------|-----|
| ARB_MODE4 | [4] | Arbiter4优先级转换使能：<br>0：优先级不自动转换；1：优先级自动转换 | 1   |
| ARB_MODE3 | [3] | Arbiter3优先级转换使能：<br>0：优先级不自动转换；1：优先级自动转换 | 1   |
| ARB_MODE2 | [2] | Arbiter2优先级转换使能：<br>0：优先级不自动转换；1：优先级自动转换 | 1   |
| ARB_MODE1 | [1] | Arbiter1优先级转换使能：<br>0：优先级不自动转换；1：优先级自动转换 | 1   |
| ARB_MODE0 | [0] | Arbiter0优先级转换使能：<br>0：优先级不自动转换；1：优先级自动转换 | 1   |

## 5.6.3 中断控制器的特殊功能寄存器

### (5) 中断挂起寄存器

中断挂起寄存器 **INTPND** 的32位，显示具有**最高优先级**，且**未被屏蔽**的中断请求，在等待**中断服务**。因为INTPND寄存器位于优先级逻辑之后，仅1位可以被置1。该中断请求将生成对CPU的IRQ。在对于IRQ的中断服务程序中，我们可以读取该寄存器，来查看哪个中断源被服务。



## 5.6.3 中断控制器的特殊功能寄存器

- 同 SRCPND 一样，INTPND 也需要在中断服务程序中加入 清0 操作，位于 SRCPND 寄存器 清0 操作之后。
- 可以通过向 INTPND 寄存器写入一个数据对相应位 清0 。仅将1对应位清除，其余保持原值不变。

中断挂起寄存器 INTPND 的地址等信息如下表所示：

| 寄存器    | 地址         | R/W | 描述                               | 复位值 |
|--------|------------|-----|----------------------------------|-----|
| INTPND | 0X4A000010 | R/W | 指示中断请求状态：<br>0：无中断请求<br>1：产生中断请求 | 0X0 |

## 5.6.3 中断控制器的特殊功能寄存器

**INTPND** 寄存器中各个比特位的描述如下所示：

| INTPND    | 位    | 描述         | 初始值 |
|-----------|------|------------|-----|
| INT_ADC   | [31] | 0：无请求；1：请求 | 0   |
| INT_RTC   | [30] | 0：无请求；1：请求 | 0   |
| INT_SPI1  | [29] | 0：无请求；1：请求 | 0   |
| INT_UART0 | [28] | 0：无请求；1：请求 | 0   |
| INT_IIC   | [27] | 0：无请求；1：请求 | 0   |
| INT_USBH  | [26] | 0：无请求；1：请求 | 0   |
| INT_USBD  | [25] | 0：无请求；1：请求 | 0   |
| INT_NFCON | [24] | 0：无请求；1：请求 | 0   |
| INT_UART1 | [23] | 0：无请求；1：请求 | 0   |
| INT_SPI0  | [22] | 0：无请求；1：请求 | 0   |

| INTPND     | 位    | 描述            | 初始值 |
|------------|------|---------------|-----|
| INT_SDI    | [21] | 0: 无请求; 1: 请求 | 0   |
| INT_DMA3   | [20] | 0: 无请求; 1: 请求 | 0   |
| INT_DMA2   | [19] | 0: 无请求; 1: 请求 | 0   |
| INT_DMA1   | [18] | 0: 无请求; 1: 请求 | 0   |
| INT_DMA0   | [17] | 0: 无请求; 1: 请求 | 0   |
| INT_LCD    | [16] | 0: 无请求; 1: 请求 | 0   |
| INT_UART2  | [15] | 0: 无请求; 1: 请求 | 0   |
| INT_TIMER4 | [14] | 0: 无请求; 1: 请求 | 0   |
| INT_TIMER3 | [13] | 0: 无请求; 1: 请求 | 0   |
| INT_TIMER2 | [12] | 0: 无请求; 1: 请求 | 0   |
| INT_TIMER1 | [11] | 0: 无请求; 1: 请求 | 0   |
| INT_TIMER0 | [10] | 0: 无请求; 1: 请求 | 0   |
| INT_WDT    | [9]  | 0: 无请求; 1: 请求 | 0   |

| INTPND    | 位   | 描述            | 初始值 |
|-----------|-----|---------------|-----|
| INT_TICK  | [8] | 0: 无请求; 1: 请求 | 0   |
| nBATT_FLT | [7] | 0: 无请求; 1: 请求 | 0   |
| INT_CAM   | [6] | 0: 无请求; 1: 请求 | 0   |
| EINT8_23  | [5] | 0: 无请求; 1: 请求 | 0   |
| EINT4_7   | [4] | 0: 无请求; 1: 请求 | 0   |
| EINT3     | [3] | 0: 无请求; 1: 请求 | 0   |
| EINT2     | [2] | 0: 无请求; 1: 请求 | 0   |
| EINT1     | [1] | 0: 无请求; 1: 请求 | 0   |
| EINT0     | [0] | 0: 无请求; 1: 请求 | 0   |

## 5.6.3 中断控制器的特殊功能寄存器

**注意：**

如果 **FIQ** 模式中断发生，那么在 **INTPND** 寄存器相应位将不被改变，因为 **INTPND** 寄存器仅设置为 **IRQ** 模式中断有效。

### (6) INTOFFSET寄存器

**INTOFFSET** 寄存器的值说明了 **INTPND** 寄存器中哪一个 **IRQ** 模式的中断请求有效。这个比特位可以通过清除 **SRCPND** 和 **INTPND** 寄存器来自动清除。

## 5.6.3 中断控制器的特殊功能寄存器

**INTOFFSET** 寄存器的地址等信息如下所示：

| 寄存器       | 地址         | R/W | 描述         | 复位值 |
|-----------|------------|-----|------------|-----|
| INTOFFSET | 0X4A000014 | R/W | 指示IRQ中断请求源 | 0X0 |

**INTOFFSET** 寄存器中各个比特位的描述如下所示：



| 中断源       | OFFSET值 | 中断源       | OFFSET值 |
|-----------|---------|-----------|---------|
| INT_ADC   | 31      | INT_UART1 | 23      |
| INT_RTC   | 30      | INT_SPI0  | 22      |
| INT_SPI1  | 29      | INT_SDI   | 21      |
| INT_UART0 | 28      | INT_DMA3  | 20      |
| INT_IIC   | 27      | INT_DMA2  | 19      |
| INT_USBH  | 26      | INT_DMA1  | 18      |
| INT_USBD  | 25      | INT_DMA0  | 17      |
| INT_NFCON | 24      | INT_LCD   | 16      |

| 中断源        | OFFSET值 | 中断源       | OFFSET值 |
|------------|---------|-----------|---------|
| INT_UART2  | 15      | nBATT_FLT | 7       |
| INT_TIMER4 | 14      | INT_CAM   | 6       |
| INT_TIMER3 | 13      | EINT8_23  | 5       |
| INT_TIMER2 | 12      | EINT4_7   | 4       |
| INT_TIMER1 | 11      | EINT3     | 3       |
| INT_TIMER0 | 10      | EINT2     | 2       |
| INT_WDT    | 9       | EINT1     | 1       |
| INT_TICK   | 8       | EINT0     | 0       |

注意：FIQ 模式中断不影响 INTOFFSET 寄存器，因为 INTOFFSET 寄存器仅对 IRQ 模式中断有效。

## 5.6.3 中断控制器的特殊功能寄存器

### (7) 次级源挂起寄存器

可以通过向 **SUBSRCPND** 寄存器某一位写入1来清除该位。此数据仅清除数据对应的比特位，其它的比特位不变。次级源挂起寄存器 **SUBSRCPND** 的地址等信息如下所示：

| 寄存器       | 地址         | R/W | 描述                                 | 复位值 |
|-----------|------------|-----|------------------------------------|-----|
| SUBSRCPND | 0X4A000018 | R/W | 指示中断请求的状态：<br>0：无中断发生；<br>1：有中断发生。 | 0X0 |

## 5.6.3 中断控制器的特殊功能寄存器

**SUBSRCPND** 寄存器中各个比特位的描述如下所示：

| SUBSRCPND | 位       | 描述         | 初始值 |
|-----------|---------|------------|-----|
| Reserved  | [31:11] | 0：无请求；1：请求 | 0   |
| INT_ADC_S | [10]    | 0：无请求；1：请求 | 0   |
| INT_TC    | [9]     | 0：无请求；1：请求 | 0   |
| INT_ERR2  | [8]     | 0：无请求；1：请求 | 0   |
| INT_TXD2  | [7]     | 0：无请求；1：请求 | 0   |
| INT_RXD2  | [6]     | 0：无请求；1：请求 | 0   |
| INT_ERR1  | [5]     | 0：无请求；1：请求 | 0   |
| INT_TXD1  | [4]     | 0：无请求；1：请求 | 0   |
| INT_RXD1  | [3]     | 0：无请求；1：请求 | 0   |
| INT_ERR0  | [2]     | 0：无请求；1：请求 | 0   |
| INT_TXD0  | [1]     | 0：无请求；1：请求 | 0   |
| INT_RXD0  | [0]     | 0：无请求；1：请求 | 0   |

## 5.6.3 中断控制器的特殊功能寄存器

### (8) 中断次级屏蔽寄存器

中断次级屏蔽寄存器 **INTSUBMSK** 中的 **11 位** 和响应的中断源有联系，如果某一比特位设置为 **1**，那么来自相应中断源的中断请求便不会被 **CPU** 执行（注意此种情况下，**SUBSRCPND** 寄存器相应位会被设置为 **1**）。如果屏蔽位为 **0**，那么中断请求将会被执行。

中断次级屏蔽寄存器 **INTSUBMSK** 的地址等信息如下：

## 5.6.3 中断控制器的特殊功能寄存器

| 寄存器       | 地址         | R/W | 描述                                                  | 复位值   |
|-----------|------------|-----|-----------------------------------------------------|-------|
| INTSUBMSK | 0X4A00001C | R/W | 决定哪一个中断源被屏蔽，被屏蔽的中断源将不会执行。<br>0：中断服务有效；<br>1：中断服务屏蔽。 | 0X7FF |

**INTSUBMSK** 寄存器中各个比特位的描述如下所示：



## 5.6.3 中断控制器的特殊功能寄存器

| INTSUBMSK | 位       | 描述           | 初始值 |
|-----------|---------|--------------|-----|
| Reserved  | [31:11] | 保留           | 0   |
| INT_ADC_S | [10]    | 0: 有效; 1: 屏蔽 | 1   |
| INT_TC    | [9]     | 0: 有效; 1: 屏蔽 | 1   |
| INT_ERR2  | [8]     | 0: 有效; 1: 屏蔽 | 1   |
| INT_TXD2  | [7]     | 0: 有效; 1: 屏蔽 | 1   |
| INT_RXD2  | [6]     | 0: 有效; 1: 屏蔽 | 1   |
| INT_ERR1  | [5]     | 0: 有效; 1: 屏蔽 | 1   |
| INT_TXD1  | [4]     | 0: 有效; 1: 屏蔽 | 1   |
| INT_RXD1  | [3]     | 0: 有效; 1: 屏蔽 | 1   |
| INT_ERR0  | [2]     | 0: 有效; 1: 屏蔽 | 1   |
| INT_TXD0  | [1]     | 0: 有效; 1: 屏蔽 | 1   |
| INT_RXD0  | [0]     | 0: 有效; 1: 屏蔽 | 1   |

## 5.6.3 中断控制器的特殊功能寄存器

各中断相关寄存器中，相关中断顺序表

| 中断        | 位    | 中断        | 位    | 中断         | 位    | 中断        | 位   |
|-----------|------|-----------|------|------------|------|-----------|-----|
| INT_ADC   | [31] | INT_UART1 | [23] | INT_UART2  | [15] | nBATT_FLT | [7] |
| INT_RTC   | [30] | INT_SPI0  | [22] | INT_TIMER4 | [14] | INT_CAM   | [6] |
| INT_SPI1  | [29] | INT_SDI   | [21] | INT_TIMER3 | [13] | EINT8_23  | [5] |
| INT_UART0 | [28] | INT_DMA3  | [20] | INT_TIMER2 | [12] | EINT4_7   | [4] |
| INT_IIC   | [27] | INT_DMA2  | [19] | INT_TIMER1 | [11] | EINT3     | [3] |
| INT_USBH  | [26] | INT_DMA1  | [18] | INT_TIMER0 | [10] | EINT2     | [2] |
| INT_USBD  | [25] | INT_DMA0  | [17] | INT_WDT    | [9]  | EINT1     | [1] |
| INT_NFCON | [24] | INT_LCD   | [16] | INT_TICK   | [8]  | EINT0     | [0] |

源挂起寄存器(SRCPND)、中断模式寄存器(INTMOD)、中断屏蔽寄存器(INTMSK)、优先级控制寄存器(PRIORITY)、中断挂起寄存器(INTPND)中,各位所对应的中断如表2所示。不同寄存器每位不同取值所对应的含义如下:

□源挂起寄存器(SRCPND):

**0: 对应中断无请求; 1: 对应中断发出请求**

□中断模式寄存器(INTMOD):

**0: 对应中断采用IRQ方式; 1: 对应中断采用FIQ方式**

□中断屏蔽寄存器(INTMSK):

**0: 对应中断有效; 1: 对应中断屏蔽**

□优先级控制寄存器 PRIORITY (ARB\_MODE<sub>n</sub>和ARB\_SEL<sub>n</sub>)

ARB\_MODE<sub>n</sub> = 0: 优先级不能转换; 1: 优先级转换使能。

ARB\_SEL<sub>n</sub> (n取0-6的值, 分别对应7个优先级控制器) 这两位, 决定对应的优先级控制器优先级的设置顺序。其对应的顺序为:

00: =REQ 0-1-2-3-4-5、 01: =REQ 0-2-3-4-1-5、

10: =REQ 0-3-4-1-2-5、 11: =REQ 0-4-1-2-3-5。

□中断挂起寄存器(INTPND):

**0: 对应中断无请求; 1: 对应中断发出请求**

## 5.6.3 中断控制器的特殊功能寄存器

例题：

请描述INT\_TIMER0、INT\_TIMER2和INT\_UART0三个中断的处理过程。有以下几个假设：

假设1：这三个中断的屏蔽被取消，即没有被屏蔽。

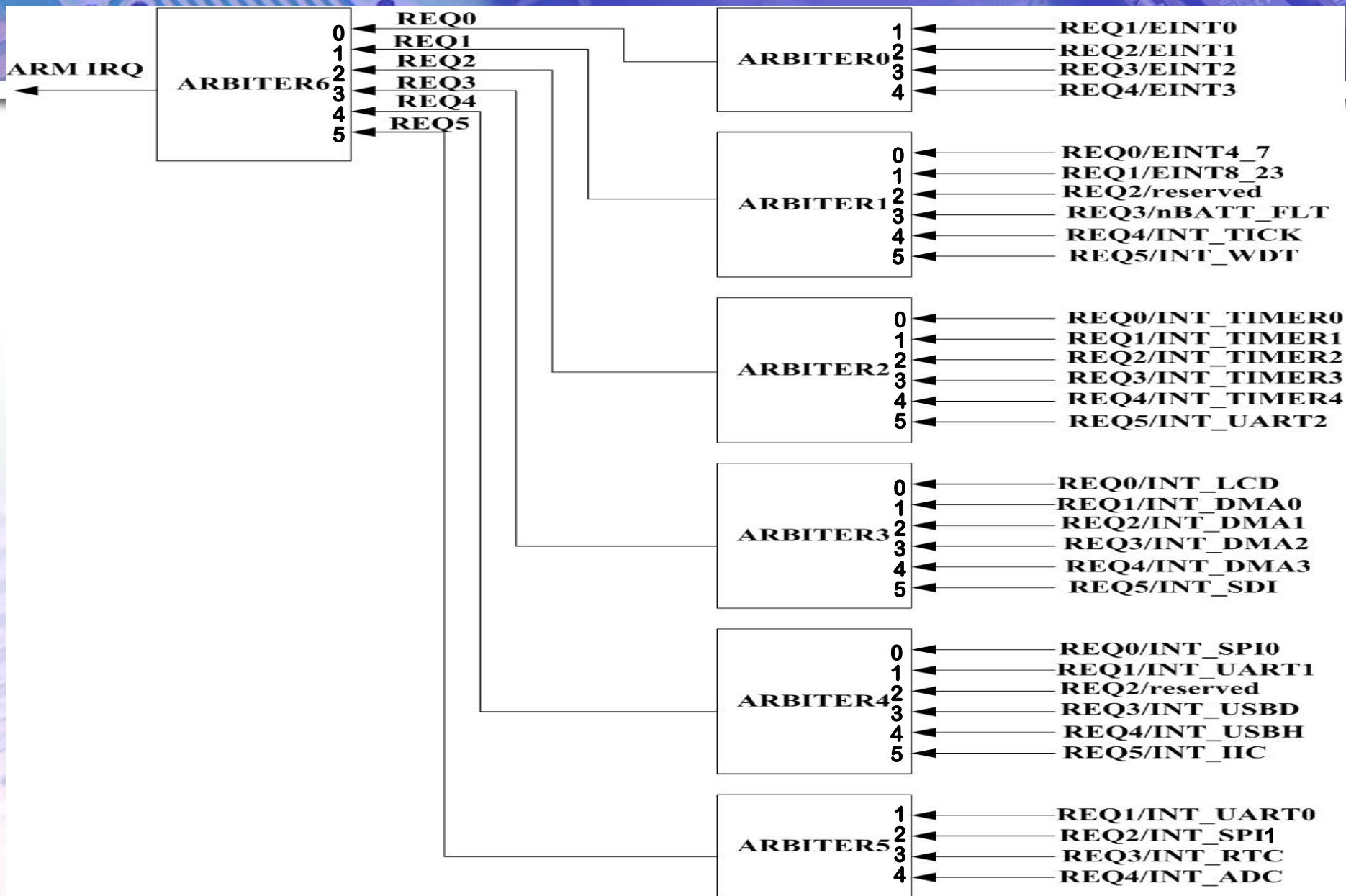
假设2：PRIORITY寄存器中ARB\_MODE2，ARB\_MODE6皆为0，ARB\_SEL2和ARB\_SEL6皆为00。

假设3：这三个中断皆为IRQ类型。

假设4：这三个中断同时被触发。

问：假设2意味着什么？

即不进行优先级的自动旋转排序，任何时候ARBITER2和ARBITER6（总控）控制的中断组优先级次序分别为0-1-2-3-4-5。



- **SRCPND** : INT\_TIMER0、NT\_TIMER2和INT\_UART0三个中断被同时触发, 此时三个中断信号流向SRCPND寄存器, 使该寄存器中的第10位, 12位, 28位被置为1。
- 信号进一步流经INTMODE寄存器, 这三个中断皆为IRQ类型; 中断信号继续向前流经INTMASK寄存器, 这三个中断都没有被屏蔽, 于是中断信号继续向前流向PRIORITY寄存器。
- **INTPND 和 INTOFFSET** : 经过优先级判断, INT\_TIMER0中断信号使INTPND寄存器的第10位置1 (INT\_TIMER0优先级最高), 此时INTOFFSET寄存器的值为10。



- CPU转向相应的中断服务例程进行处理。
- 中断服务器程序中，我们的程序将INTPND和SRCPND的第10清0，然后进行中断服务程序的处理，直到INT\_TIMER0中断处理完毕。
- 此时SRCPND的第12位，28位仍为1（这两个中断请求未被处理），故他们会继续被CPU以刚才描述的方式进行处理。

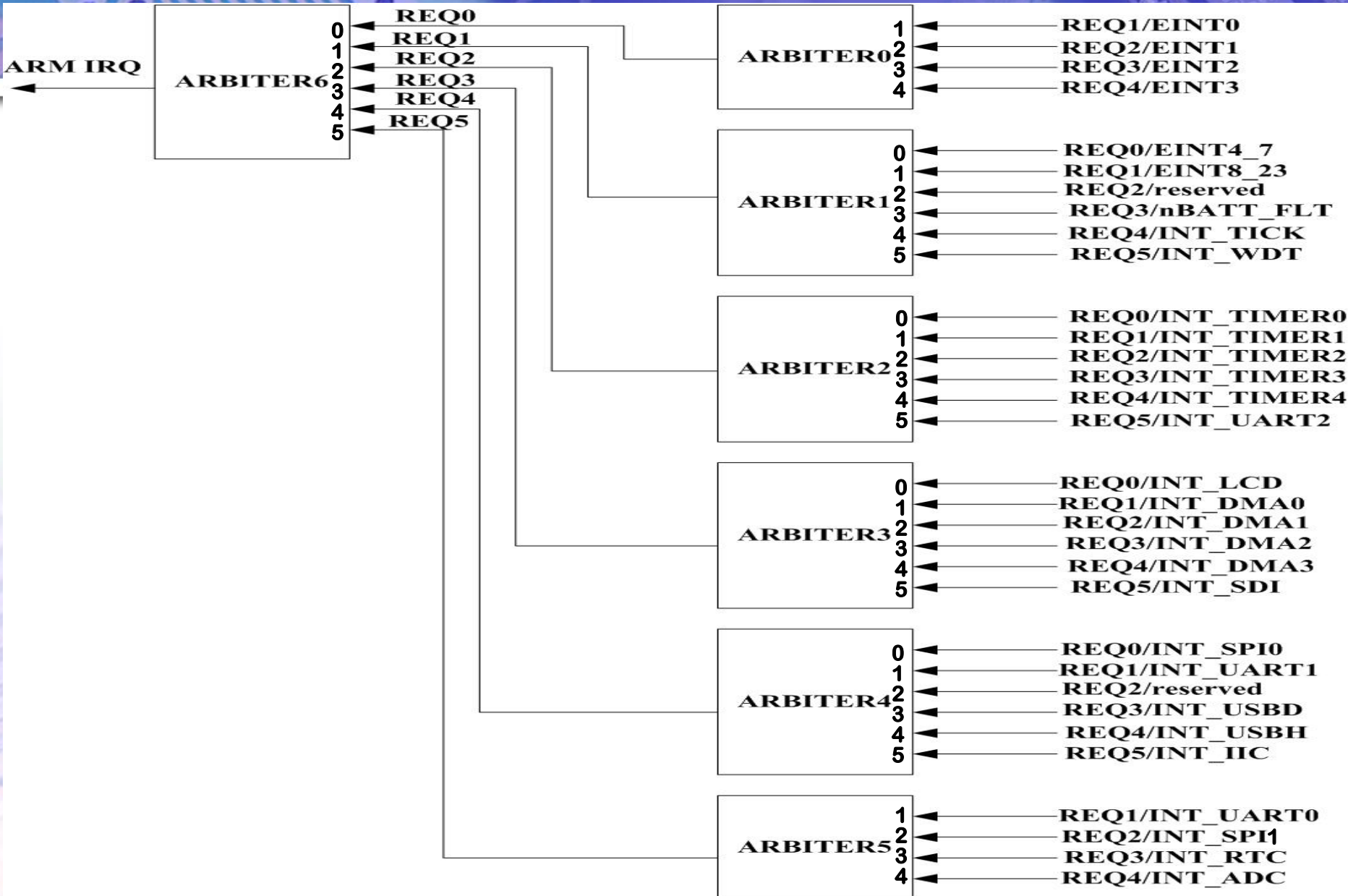
在一个采用S3C2440作为处理器的嵌入式系统中，假设中断INT\_TIMER0、INT\_TIMER3、INT\_SPI1、EINT0这四个中断同时产生，这四个中断产生之前，CPSR寄存器的低8位的值为0x1F。相关寄存器的值如下：

| 寄存器           | 寄存器地址      | 中断发生时对应寄存器的二进制值                         |
|---------------|------------|-----------------------------------------|
| <b>INTMOD</b> | 0X4A000004 | 0000 0000 0000 0000 0000 0000 0000 0000 |
| <b>INTMSK</b> | 0X4A000008 | 0000 0000 1111 1111 0000 0000 1111 1111 |
| <b>INTPND</b> | 0X4A000010 | 0000 0000 0000 0000 0000 0000 0000 0000 |
| <b>SRCPND</b> | 0X4A000000 | 0000 0000 0000 0000 0000 0000 0000 0000 |

且优先级控制寄存器 PRIORITY 中的 ARB\_MODE2，ARB\_MODE5皆为1。

PS: INT\_TIMER0[10]、INT\_TIMER3[13]、INT\_SPI1[29]、EINT0[0]





试回答一下问题：

- ① 当INT\_TIMER0、INT\_TIMER3、INT\_SPI1、EINT0这四个中断同时产生后，SRCPND寄存器中的值（使用32位二进制来表示）为：

0010 0000 0000 0000 0010 0100 0000 0001

- ② INTPND寄存器中的值（使用32位二进制来表示）为：

0000 0000 0000 0000 0000 0100 0000 0000

- ③ 此时INTOFFSET寄存器的值为10（使用10进制数表示）。

- ④ CPU将进行INT\_TIMER0中断的中断服务例程进行处理（从INT\_TIMER0、INT\_TIMER3、INT\_SPI1、EINT0中选填一个）。

- a) 则当第一个中断服务程序结束后，SRCPND寄存器中的值（使用32位二进制来表示）为：

0010 0000 0000 0000 0010 0000 0000 0001.

- b) INTPND寄存器中的值（使用32位二进制来表示）为：

0000 0000 0000 0000 0010 0000 0000 0000.

- c) 如果程序要屏蔽INT\_TIMER0中断，请写出相关程序代码（注：只需写出相关指令代码，不需要写出完整的汇编程序结构）。（3分）

LDR R1, =0x4A000008

LDR R2, [R1]

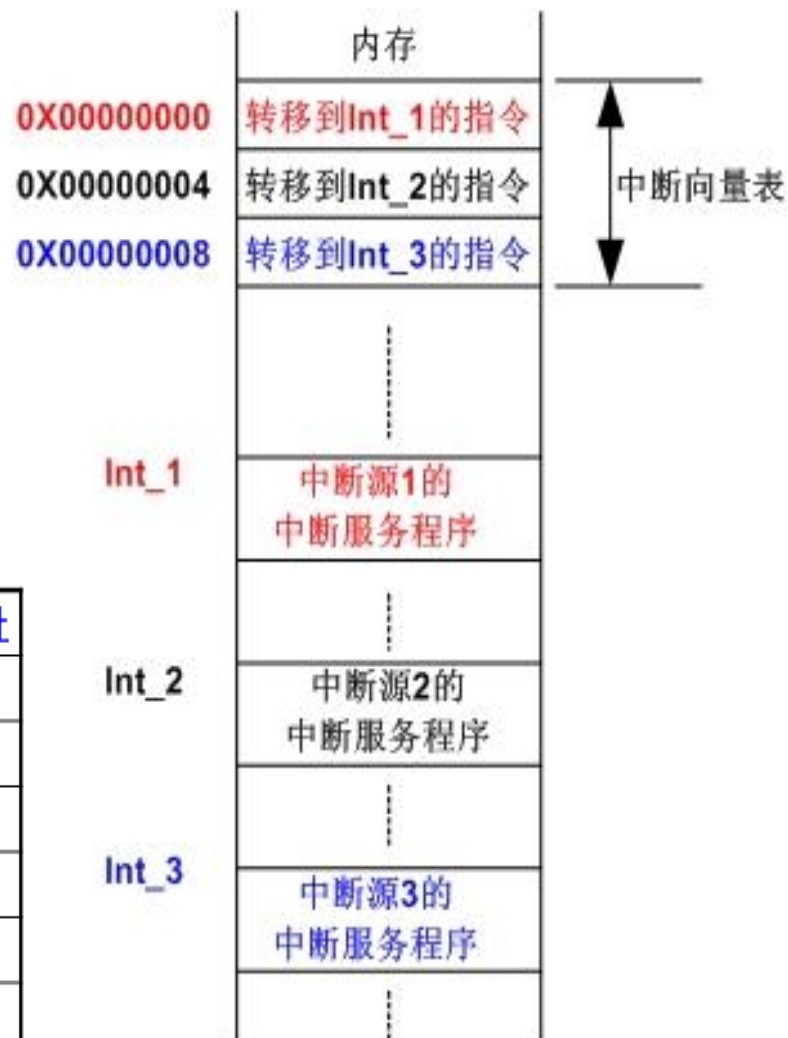
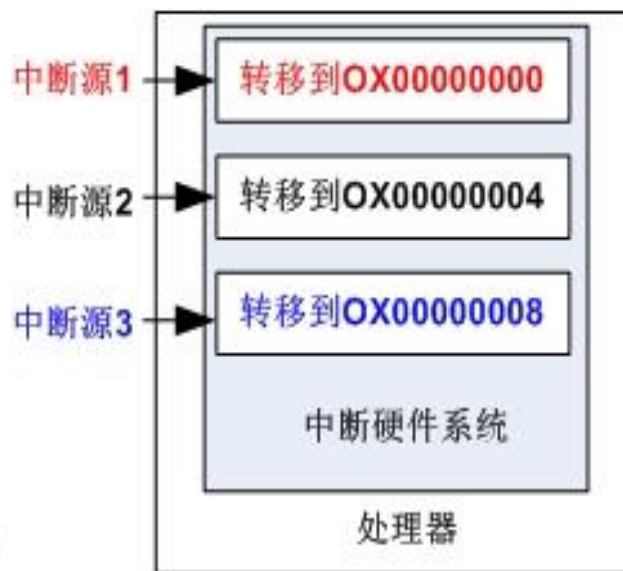
ORR R2, R2, #0x400

STR R2, [R1]





# 编写中断服务程序



| 中断 (异常)       | 向量在低端向量表的地址 | 向量在高端向量表的地址 |
|---------------|-------------|-------------|
| 复位 (RESET)    | 0x00000000  | 0xFFFF0000  |
| 未定义指令 (UNDEF) | 0x00000004  | 0xFFFF0004  |
| 软中断 (SWI)     | 0x00000008  | 0xFFFF0008  |
| 预取指令中止 (PABT) | 0x0000000C  | 0xFFFF000C  |
| 数据中止 (DABT)   | 0x00000010  | 0xFFFF0010  |
| 保留            | 0x00000014  | 0xFFFF0014  |
| 中断 (IRQ)      | 0x00000018  | 0xFFFF0018  |
| 快中断 (FIQ)     | 0x0000001C  | 0xFFFF001C  |



## 编写中断服务程序（方法一）

s3c2440.s中中断向量以及中断服务程序相关的程序段如下（简化版）：

```
PRESERVE8  
AREA      RESET, CODE, READONLY
```

Vectors

```
LDR      PC, Reset_Addr  
LDR      PC, Undef_Addr  
LDR      PC, SWI_Addr  
LDR      PC, PAbt_Addr  
LDR      PC, DAbt_Addr  
DCD      0x4000  
LDR      PC, IRQ_Addr  
LDR      PC, FIQ_Addr
```

## 编写中断服务程序

s3c2440. s中中断向量以及中断服务程序相关的程序段如下（简化版）：

|            |     |                      |
|------------|-----|----------------------|
| Reset_Addr | DCD | Reset_Handler        |
| Undef_Addr | DCD | Undef_Handler        |
| SWI_Addr   | DCD | SWI_Handler          |
| PAbt_Addr  | DCD | PAbt_Handler         |
| DAbt_Addr  | DCD | DAbt_Handler         |
|            | DCD | 0 ; Reserved Address |
| IRQ_Addr   | DCD | IRQ_Handler          |
| FIQ_Addr   | DCD | FIQ_Handler          |

## 编写中断服务程序

以上程序段，编译后在存储单元中的内容如下：

| 地址单元       | 存放的指令                     | 说明                                                                   |
|------------|---------------------------|----------------------------------------------------------------------|
| 0x00000000 | LDR PC, <u>Reset_Addr</u> | 编译后为: LDR PC,[PC, 0x18]                                              |
| 0x00000004 | LDR PC, <u>Undef_Addr</u> | 编译后为: LDR PC,[PC, 0x18]                                              |
| 0x00000008 | LDR PC, <u>SWI_Addr</u>   | 编译后为: LDR PC,[PC, 0x18]                                              |
| 0x0000000C | LDR PC, <u>PAbt_Addr</u>  | 编译后为: LDR PC,[PC, 0x18]                                              |
| 0x00000010 | LDR PC, <u>DAbt_Addr</u>  | 编译后为: LDR PC,[PC, 0x18]                                              |
| 0x00000014 | 0x4000                    | DCD 0x4000                                                           |
| 0x00000018 | LDR PC, <u>IRQ_Addr</u>   | 编译后为: LDR PC,[PC, 0x18]                                              |
| 0x0000001C | LDR PC, <u>FIQ_Addr</u>   | 编译后为: LDR PC,[PC, 0x18]                                              |
| 0x00000020 | <u>Reset_Handler</u>      | Reset 中断服务服务程序入口地址<br>由 <u>Reset_Add</u> DCD <u>Reset_Handler</u> 定义 |
| 0x00000024 | <u>Undef_Handler</u>      |                                                                      |
| 0x00000028 | <u>SWI_Handler</u>        |                                                                      |
| 0x0000002C | <u>PAbt_Handler</u>       |                                                                      |
| 0x00000030 | <u>DAbt_Handler</u>       |                                                                      |
| 0x00000034 | 0                         |                                                                      |
| 0x00000038 | <u>IRQ_Handler</u>        |                                                                      |
| 0x0000003C | <u>FIQ_Handler</u>        |                                                                      |

# 编写中断服务程序

s3c2440. s中中断向量以及中断服务程序相关的程序段如下（简化版）：

```
Undef_Handler    B    Undef_Handler
SWI_Handler      B    SWI_Handler
PAbt_Handler     B    PAbt_Handler
DAbt_Handler     B    DAbt_Handler
IRQ_Handler      PROC
EXPORT            IRQ_Handler    [WEAK]
B                .
ENDP
FIQ_Handler      B    FIQ_Handler

EXPORT            Reset_Handler
Reset_Handler
:
:
```

# 编写中断服务程序

重新定义的IRQ\_Handler中断服务程序。（IRQ\_ISR.s）

```
PRESERVE8
GET 2440addr.inc
AREA IRQ_ISR, CODE, READONLY
IRQ_Handler PROC
EXPORT IRQ_Handler

IsrIRQ

    sub    sp, sp, #4                ;reserved
    stmfd  sp!, {r8-r9}
    :
    :
    ldmfd  sp!, {r8-r9, pc}

ENDP
```

# 编写中断服务程序

## 问题：

1. EINT0、EINT1……TIMER0、UART0等中断服务程序如何编写？
2. 如何用C语言实现上述中断服务程序的编写？



## 重新定义的IRQ\_Handler中断服务程序。（IRQ\_ISR.s）

本实验定义的RAM是从0x3000 0000开始, size为: 0x20 0000, 把自定义中断向量放在RAM的高端, 从0x301f ff00开始。

```
PRESERVE8
GET 2440addr.inc
```

```
_ISR_STARTADDRESS EQU 0x301fff00
```

```
AREA IRQ_ISR, CODE, READONLY
```

```
IRQ_Handler PROC
```

```
EXPORT IRQ_Handler
```

```
IsrIRQ
```

```
sub sp, sp, #4 ;reserved for PC
```

;保留一个堆栈单元, 用于存放跳转地址, 为出栈到PC做准备

```
stmfd sp!, {r8-r9}
```

```
ldr r9, =INTOFFSET
```

;获取INTOFFSET 寄存器的地址, INTOFFSET 寄存器的值

;说明了INTPND寄存器中哪一个IRQ模式的中断请求有效

```
ldr r9, [r9]
```

```
ldr r8, =HandleEINT0
```

;获取自定义IRQ向量表的首地址, 即EINT0对应的地址

```
add r8, r8, r9, lsl #2
```

;自定义IRQ向量表的首地址+对应的偏移量

## INTOFFSET寄存器

| INTOFFSET | 偏移量 | INTOFFSET    | 偏移量 |
|-----------|-----|--------------|-----|
| INT_ADC   | 31  | INT_UART2    | 15  |
| INT_RTC   | 30  | INT_TIMER4   | 14  |
| INT_SPI1  | 29  | INT_TIMER3   | 13  |
| INT_UART0 | 28  | INT_TIMER2   | 12  |
| INT_IIC   | 27  | INT_TIMER1   | 11  |
| INT_USBH  | 26  | INT_TIMER0   | 10  |
| INT_USBD  | 25  | INT_WDT_AC97 | 9   |
| INT_NFCON | 24  | INT_TICK     | 8   |
| INT_UART1 | 23  | nBATT_FLT    | 7   |
| INT_SPI0  | 22  | INT_CAM      | 6   |
| INT_SDI   | 21  | EINT8_23     | 5   |
| INT_DMA3  | 20  | EINT4_7      | 4   |
| INT_DMA2  | 19  | EINT3        | 3   |
| INT_DMA1  | 18  | EINT2        | 2   |
| INT_DMA0  | 17  | EINT1        | 1   |
| INT_LCD   | 16  | EINT0        | 0   |



重新定义的IRQ\_Handler中断服务程序。（IRQ\_ISR.s）

```
ldr    r8, [r8]
```

;从对应的IRQ向量表中取得中断服务程序入口地址

```
str    r8, [sp, #8]
```

;把获得的中断服务程序入口地址存入预留的堆栈单元中

```
ldmfd sp!, {r8-r9, pc}
```

;出栈操作，恢复r8和r9的原始值，并把中断服务程序入口地址

;弹入到PC中，转到对应中断服务程序执行。

```
ENDP
```

;=====用于自定义IRQ中断向量设置=====

```
ALIGN
```

```
AREA RamData, DATA, READWRITE
```

```
MAP    _ISR_STARTADDRESS
```

;IRQ中断向量从**\_ISR\_STARTADDRESS**开始存放

```
HandleEINT0      #    4
```

```
HandleEINT1      #    4
```

```
HandleEINT2      #    4
```

重新定义的IRQ\_Handler中断服务程序。（IRQ\_ISR.s）

```
HandleEINT3          # 4
HandleEINT4_7        # 4
HandleEINT8_23       # 4
HandleCAM            # 4
HandleBATFLT         # 4
HandleTICK           # 4
HandleWDT            # 4
HandleTIMER0         # 4
HandleTIMER1         # 4
HandleTIMER2         # 4
HandleTIMER3         # 4
HandleTIMER4         # 4
HandleUART2         # 4
:
:
```

; Added for 2440.

这里定义了一个数据段RamData，专门用于存放中断服务程序的跳转地址。

HandleEINT0存储单元中存放的就是EINT0中断服务程序的地址。

相关地址可以由开发人员自行填充。

C语言中的定义。（2440addr.h）



```
// Exception vector
```

```
#define _ISR_STARTADDRESS 0x301fff00
```

```
//这个和_ISR_STARTADDRESS EQU 0x301fff00相对应
```

```
#define pISR_EINT0      (*(unsigned *)(_ISR_STARTADDRESS+0x0))
```

```
#define pISR_EINT1      (*(unsigned *)(_ISR_STARTADDRESS+0x4))
```

```
#define pISR_EINT2      (*(unsigned *)(_ISR_STARTADDRESS+0x8))
```

```
#define pISR_EINT3      (*(unsigned *)(_ISR_STARTADDRESS+0xc))
```

```
#define pISR_EINT4_7    (*(unsigned *)(_ISR_STARTADDRESS+0x10))
```

```
#define pISR_EINT8_23   (*(unsigned *)(_ISR_STARTADDRESS+0x14))
```

```
#define pISR_CAM        (*(unsigned *)(_ISR_STARTADDRESS+0x18))
```

```
:
```

```
:
```

C语言中的定义。（main.c）

```
void IRQ_init(void)
{
    :
    : //下面语句用于把bit8和bit9被清0，其余位保持不变
    rEINTMASK &= ~((1<<8) | (1<<9)); //使中断不被屏蔽
    pISR_EINT8_23 = (U32)Key_ISR; //中断服务程序
    EnableIrq(BIT_EINT8_23); //使能中断
}
```

<2440addr.h>中定义

```
#define EnableIrq(bit) rINTMSK &= ~(bit)
```

//上面语句用于把INTMSK寄存器中的对应bit清0，使能中断。



## C语言中的定义。（main.c）

[illegible]

“\_irq” 专门用来声明IRQ中断服务程序。

如果用 “\_irq” 来声明一个函数，那么该函数表示一个IRQ中断服务程序，编译器便会自动在该函数内部增加中断现场保护的代码。

同样一个函数，如果将关键字 “\_irq” 去掉，那么编译器便不会增加现场保护的代码，而只是作为一个普通函数来处理。

用 “\_irq” 声明后，

进入中断服务器程序时，会使用以下语句保护现场：

```
STMDB R13!, {Rxx, R14}
```

离开程序时，会使用以下语句恢复现场：

```
LDMIA R13!, {Rxx, R14}
```

```
SUBS PC, R14, #0x00000004
```

## 编写中断服务程序 一自己构建初始化程序实现方式（方法二）

初始化程序中断相关的完整的程序构架如下（注释见后面描述）：

```
MACRO
$HandlerLabel HANDLER $HandleLabel

$HandlerLabel
    sub    sp, sp, #4
    stmfd  sp!, {r0}
    ldr     r0, =$HandleLabel
    ldr     r0, [r0]
    str     r0, [sp, #4]
    ldmfd  sp!, {r0, pc}
MEND

AREA     RESET, CODE, READONLY
ENTRY

b        ResetHandler
b        HandlerUndef    ;handler for Undefined mode
b        HandlerSWI      ;handler for SWI interrupt
```

# 编写中断服务程序 - 自己构建初始化程序实现方式

```
b      HandlerPabort ;handler for PAbort
b      HandlerDabort ;handler for DAbort
b      .              ;reserved
b      HandlerIRQ    ;handler for IRQ interrupt
b      HandlerFIQ    ;handler for FIQ interrupt
```

```
HandlerFIQ      HANDLER HandleFIQ    ;汇编时宏将展开为程序段
HandlerIRQ      HANDLER HandleIRQ
HandlerUndef    HANDLER HandleUndef
HandlerSWI      HANDLER HandleSWI
HandlerDabort   HANDLER HandleDabort
HandlerPabort   HANDLER HandlePabort
```

IsrIRQ

```
sub    sp, sp, #4          ;reserved for PC
stmfd  sp!, {r8-r9}
```

# 编写中断服务程序 - 自己构建初始化程序实现方式

```
ldr    r9, =INT0OFFSET
ldr    r9, [r9]
ldr    r8, =HandleEINT0
add    r8, r8, r9, lsl #2
ldr    r8, [r8]
str    r8, [sp, #8]
ldmfd  sp!, {r8-r9, pc}
```

LTORG

ResetHandler

:  
:

; =====Setup IRQ handler

```
ldr    r0, =HandleIRQ
ldr    r1, =IsrIRQ
str    r1, [r0]
```

:  
:

;This routine is needed

# 编写中断服务程序 - 自己构建初始化程序实现方式

ALIGN

AREA RamData, DATA, READWRITE

^     \_ISR\_STARTADDRESS

HandleReset       #    4

HandleUndef       #    4

HandleSWI          #    4

HandlePabort       #    4

HandleDabort       #    4

HandleReserved     #    4

HandleIRQ          #    4

HandleFIQ          #    4

HandleEINT0        #    4

HandleEINT1        #    4

HandleEINT2        #    4

HandleEINT3        #    4

HandleEINT4\_7       #    4

HandleEINT8\_23     #    4

:

:



# 编写中断服务程序 - 自己构建初始化程序实现方式

MACRO ;定义一个宏

\$HandlerLabel HANDLER \$HandlerLabel

; 主标号: \$HandlerLabel 宏名: HANDLER 带有一个参数:

\$HandlerLabel

\$HandlerLabel

sub sp, sp, #4

; 保留一个堆栈单元, 用于存放跳转地址, 为出栈到PC做准备

stmfd sp!, {r0} ;PUSH r0寄存器

ldr r0, =\$HandlerLabel

;把存储单元HandleXXX的地址加载到r0

ldr r0, [r0]

; 把存储单元HandleXXX的内容(中断服务程序的跳转地址)读出到r0

str r0, [sp, #4] ;把中断服务程序的跳转地址压栈

ldmfd sp!, {r0, pc}

;恢复r0中的值, 然后跳转到中断服务程序

MEND

宏定义后, 如何使用?

## 编写中断服务程序 - 自己构建初始化程序实现方式

|               |                      |
|---------------|----------------------|
| HandlerFIQ    | HANDLER HandleFIQ    |
| HandlerIRQ    | HANDLER HandleIRQ    |
| HandlerUndef  | HANDLER HandleUndef  |
| HandlerSWI    | HANDLER HandleSWI    |
| HandlerDabort | HANDLER HandleDabort |
| HandlerPabort | HANDLER HandlePabort |

以HandlerIRQ为例，程序编译时，宏展开后如下：

HandlerIRQ

```
sub    sp, sp, #4
stmfd  sp!, {r0}
ldr    r0, =HandleIRQ
ldr    r0, [r0]
str    r0, [sp, #4]
ldmfd  sp!, {r0, pc} ; 程序将跳转到HandleIRQ存
```

储单元所指向的地址执行，本例为： IsrIRQ

# 编写中断服务程序 - 自己构建初始化程序实现方式

ALIGN

AREA RamData, DATA, READWRITE

^ \_ISR\_STARTADDRESS

HandleReset # 4

HandleUndef # 4

HandleSWI # 4

HandlePabort # 4

HandleDabort # 4

HandleReserved # 4

HandleIRQ # 4

HandleFIQ # 4

HandleEINT0 # 4

HandleEINT1 # 4

:

:

这里定义了一个数据段  
RamData，专门用于存放中  
断服务程序的跳转地址。

HandleIRQ存储单元中存放  
的就是IRQ中断服务程序的  
地址。

相关地址可以由开发人员自  
行填充。

## 编写中断服务程序 - 自己构建初始化程序实现方式

在复位中断处理程序中，有如下对IRQ中断服务程序进行初始化的程序段：

ResetHandler

:

:

; =====Setup IRQ handler

ldr r0, =HandleIRQ

; 获取存储单元HandleIRQ的地址

ldr r1, =IsrIRQ

; 获取IRQ中断服务程序IsrIRQ的入口地址

str r1, [r0]

; 把IRQ中断服务程序入口地址填充到存储单元HandleIRQ中

:

:

# 编写中断服务程序 - 自己构建初始化程序实现方式

IsrIRQ

```
sub    sp, sp, #4           ;reserved for PC
stmfd  sp!, {r8-r9}

ldr     r9, =INTOFFSET
ldr     r9, [r9]
ldr     r8, =HandleEINT0
add     r8, r8, r9, lsl #2
ldr     r8, [r8]
str     r8, [sp, #8]
ldmfd  sp!, {r8-r9, pc}
```

这个程序段，和方法一中的IsrIRQ程序段是一样的，用于根据不同的中断源，跳转到不同的中断服务程序执行。

后续的在C程序中的处理方式和方法一一样。



# 作业

作业见群文件《平时作业要求》中第5章的内容。

其中第五题需要在学习了后面的内容后再做。



## 5.7 I/O端口

处理器通过 I/O端口 和 外围硬件 连接。ARM芯片的 I/O端口 一般是和其它引脚 复用的。如果要熟悉 ARM芯片I/O端口 的配置方法，就要熟悉 S3C2440芯片的I/O端口 功能配置和 特殊功能寄存器的配置。

S3C2440 有 130个 通用可编程多功能 输入/输出引脚，可分为以下几类：

## 5.7 I/O端口

- (1) Port A (GPA) : 25位输出端口;
- (2) Port B (GPB) : 11位输入/输出端口;
- (3) Port C (GPC) : 16位输入/输出端口;
- (4) Port D (GPD) : 16位输入/输出端口;
- (5) Port E (GPE) : 16位输入/输出端口;
- (6) Port F (GPF) : 8位输入/输出端口;
- (7) Port G (GPG) : 16位输入/输出端口;

## 5.7 I/O端口

(8) Port H (GPH) : 9位输入/输出端口;

(9) Port J (GPJ) : 13位输入/输出端口。

这些端口都具有 多功能，通过引脚配置寄存器，可以将其设置为所需要的功能，如：I/O功能、中断功能等。

## 5.7 I/O端口

每一个端口 都可以通过 软件设置来 满足 各种各样的系统设置 和 设计要求。每个端口的功能通常要在 主程序开始前被定义。如果一个引脚的多功能没有使用，那么这个引脚将被设置为 I/O端口。在引脚配置之前，需要对引脚的初始化状态进行设置，以避免一些问题的出现。

## 5.7.1 S3C2440 端口控制描述

**I/O端口** 的各种功能主要通过通过对**端口各个寄存器** 进行设置而实现的，下面通过对各个寄存器的说明来分别介绍 **I/O端口** 所能实现的功能。

## 5.7.1 S3C2440 端口控制描述

### (1) 端口配置寄存器 (GPACON~GPJCON)

在 S3C2440中，大多数的引脚 都是 多功能引脚，所以应该为 每一个引脚 选择功能。端口控制字寄存器 (GPnCON) 决定了 每一个引脚的功能。

如果 GPF0~GPF7 和 GPG0~GPG7 在 掉电的模式下 被用作唤醒信号，则这些端口必须被配置，必须被配置为外中断功能输入口。



## 5.7.1 S3C2440 端口控制描述

### (2) 端口数据寄存器 (GPADAT~GPJDAT)

如果这些端口被配置为 **输出端口**，则输出数据可以被写入 **GPnDAT的相应位**。  
如果被设定为 **输入端口**，则输入数据可以从 **GPnDAT的相应位**读取。

## 5.7.1 S3C2440 端口控制描述

### (3) 端口上拉寄存器 (GPBUP~GPJUP)

**端口上拉寄存器控制** 每一个端口组的上拉电阻的使能端。当相应的位被设为 0 时，引脚接上拉电阻；当相应的位被设置为 1 时，引脚不接上拉电阻。如果引脚设置接上拉电阻，那么上拉电阻与引脚的功能设置无关（例如 input、output、DATAn、EINTn 等）。

## 5.7.1 S3C2440 端口控制描述

### (4) 杂项控制寄存器 MISCCR

这个寄存器控制 **DATA**端口的 **上拉电阻**、**hi-z**（高阻）状态、**USB** 和 **CLKOUT** 选择。

## 5.7.1 S3C2440 端口控制描述

### (5) 外部中断控制寄存器 (EXTINTN)

24个 **外部中断源** 可用各种信号来请求。外部中断寄存器为外部中断请求设置了**信号触发方法**，包括：**低电平触发、高电平触发、上升沿触发、下降沿触发和双边沿触发**。8个外部中断引脚 **有一个** 数字滤波器（见 EINTFLTn），16个引脚EINT[15:0]用作唤醒源。

## 5.7.1 S3C2440 端口控制描述

### (6) 掉电模式 和 I/O口

所有的 **GPIO寄存器** 的值在掉电模式下被保存。这在时钟功率管理模块中的掉电模式下提到。

**EINTMSK** 不能禁止从掉电模式唤醒，但是如果 **EINTMSK** 屏蔽了 **EINT[15:4]** 中的对应位，系统可以被唤醒，但是 **SRCPND** 中的 **EINT4\_7位** 和 **EINT8\_23 位** 不会在唤醒后被置1。

## 5.7.2 I/O端口 寄存器及引脚配置

### (1) 端口A寄存器（GPACON/GPADAT）

#### 25位输出端口

端口A 控制寄存器包括：端口A配置寄存器GPACON 和 数据寄存器GPADAT，如下表所示：

| 寄存器      | 地址         | R/W | 描述       | 复位值     |
|----------|------------|-----|----------|---------|
| GPACON   | 0X56000000 | R/W | 配置端口A的引脚 | 0X7FFFF |
| GPADAT   | 0X56000004 | R/W | 端口A数据寄存器 | 不确定     |
| Reserved | 0X56000008 | —   | 保留       | 不确定     |
| Reserved | 0X5600000C | —   | 保留       | 不确定     |



## GPACON 各位的描述如下：

| GPACON | 位    | 描述                  |
|--------|------|---------------------|
| GPA24  | [24] | 保留                  |
| GPA23  | [23] | 保留                  |
| GPA22  | [22] | 0 = 输出, 1 = nFCE    |
| GPA21  | [21] | 0 = 输出, 1 = nRSTOUT |
| GPA20  | [20] | 0 = 输出, 1 = nFRE    |
| GPA19  | [19] | 0 = 输出, 1 = nFWE    |
| GPA18  | [18] | 0 = 输出, 1 = ALE     |
| GPA17  | [17] | 0 = 输出, 1 = CLE     |
| GPA16  | [16] | 0 = 输出, 1 = nGCS[5] |
| GPA15  | [15] | 0 = 输出, 1 = nGCS[4] |
| GPA14  | [14] | 0 = 输出, 1 = nGCS[3] |

## GPACON 各位的描述如下（续）：

| GPACON | 位    | 描述                  |
|--------|------|---------------------|
| GPA13  | [13] | 0 = 输出, 1 = nGCS[2] |
| GPA12  | [12] | 0 = 输出, 1 = nGCS[1] |
| GPA11  | [11] | 0 = 输出, 1 = ADDR26  |
| GPA10  | [10] | 0 = 输出, 1 = ADDR25  |
| GPA9   | [9]  | 0 = 输出, 1 = ADDR24  |
| GPA8   | [8]  | 0 = 输出, 1 = ADDR23  |
| GPA7   | [7]  | 0 = 输出, 1 = ADDR22  |
| GPA6   | [6]  | 0 = 输出, 1 = ADDR21  |
| GPA5   | [5]  | 0 = 输出, 1 = ADDR20  |
| GPA4   | [4]  | 0 = 输出, 1 = ADDR19  |
| GPA3   | [3]  | 0 = 输出, 1 = ADDR18  |

## GPACON 各位的描述如下（续）：

| GPACON | 位   | 描述                 |
|--------|-----|--------------------|
| GPA2   | [2] | 0 = 输出, 1 = ADDR17 |
| GPA1   | [1] | 0 = 输出, 1 = ADDR16 |
| GPA0   | [0] | 0 = 输出, 1 = ADDR0  |

GPADAT 为准备输出的数据，其值为 23 位[22:0]。

## 5.7.2 I/O端口 寄存器及引脚配置

注意：

①当 A口 引脚配置为 非输出功能时，其输出无意义；

②从引脚输入 没有意义。

端口A 数据寄存器 GPADAT 中各位描述  
如下所示：

| GPADAT    | 位      | 描述                                                           |
|-----------|--------|--------------------------------------------------------------|
| GPA[22:0] | [22:0] | 当端口配置为输出端口时，对应引脚的状态和该位的值相同；<br>当端口配置为功能引脚时，如果读该位的值将是一个不确定的值。 |

## 5.7.2 I/O端口 寄存器及引脚配置

### (2) 端口B寄存器（GPBCON、GPBDAT、GPBUP） 11位 输入/输出端口

端口B 控制寄存器包括：端口B配置寄存器GPBCON、数据寄存器GPBDAT 和 上拉寄存器GPBUP，具体如下所示：

| 寄存器     | 地址         | R/W | 描述        | 复位值 |
|---------|------------|-----|-----------|-----|
| GPBCON  | 0X56000010 | R/W | 配置端口B的引脚  | 0X0 |
| GPBDAT  | 0X56000014 | R/W | 端口B的数据寄存器 | 不确定 |
| GPBUP   | 0X56000018 | R/W | 端口B的上拉寄存器 | 0X0 |
| Reserve | 0X5600001C | —   | 保留        | 不确定 |

## GPBCON 各位的描述如下:

| GPBCON | 位       | 描述                                                |
|--------|---------|---------------------------------------------------|
| GPB10  | [21:20] | 00 = Input, 10 = nXDREQ0,<br>01 = Output, 11 = 保留 |
| GPB9   | [19:18] | 00 = Input, 10 = nXDACK0,<br>01 = Output, 11 = 保留 |
| GPB8   | [17:16] | 00 = Input, 10 = nXDREQ1,<br>01 = Output, 11 = 保留 |
| GPB7   | [15:14] | 00 = Input, 10 = nXDACK1,<br>01 = Output, 11 = 保留 |
| GPB6   | [13:12] | 00 = Input, 10 = nXBREQ,<br>01 = Output, 11 = 保留  |
| GPB5   | [11:10] | 00 = Input, 10 = nXBACK ,<br>01 = Output, 11 = 保留 |



## GPBCON 各位的描述如下（续）：

| GPBCON | 位     | 描述                                                 |
|--------|-------|----------------------------------------------------|
| GPB4   | [9:8] | 00 = Input, 10 = TCLK [0],<br>01 = Output, 11 = 保留 |
| GPB3   | [7:6] | 00 = Input, 10 = TOUT3,<br>01 = Output, 11 = 保留    |
| GPB2   | [5:4] | 00 = Input, 10 = TOUT2,<br>01 = Output, 11 = 保留    |
| GPB1   | [3:2] | 00 = Input, 10 = TOUT1,<br>01 = Output, 11 = 保留    |
| GPB0   | [1:0] | 00 = Input, 10 = TOUT0,<br>01 = Output, 11 = 保留    |

## 5.7.2 I/O端口 寄存器及引脚配置

**GPBDAT** 为准备输出或输入的数据，其值为 **11位[10:0]**。

**注意：**当端口B引脚配置为 **非输入/输出功能**时，其寄存器中的值没有意义。

**GPBUP** 为 端口B **上拉寄存器**，位 **[10:0]** 有意义。

## 5.7.2 I/O端口 寄存器及引脚配置

**端口B 数据寄存器 GPBDAT 中各位的描述如下：**

| GPBDAT    | 位      | 描述                                                                                                               |
|-----------|--------|------------------------------------------------------------------------------------------------------------------|
| GPB[10:0] | [10:0] | <p>当端口配置为输入端口时，外部源的数据可以从对应引脚读出；</p> <p>当端口配置为输出端口时，写入寄存器的数据会被发送到对应的引脚上；</p> <p>当端口配置为功能引脚时，如果读该位的值将是一个不确定的值。</p> |

**端口B 上拉寄存器 GPBUP 中各位的描述如下：**

| GPBUP     | 位      | 描述                                             |
|-----------|--------|------------------------------------------------|
| GPB[10:0] | [10:0] | <p>0：允许上拉电阻连接到对应脚；</p> <p>1：不允许上拉电阻连接到对应脚。</p> |

## 5.7.2 I/O端口 寄存器及引脚配置

### (3) 端口C寄存器（GPCCON、GPCDAT、GPCUP） 16位输入/输出端口

端口C 控制寄存器包括：端口C配置寄存器GPCCON、数据寄存器GPCDAT 和 上拉寄存器GPCUP，具体如下所示：

| 寄存器     | 地址         | R/W | 描述        | 复位值 |
|---------|------------|-----|-----------|-----|
| GPCCON  | 0X56000020 | R/W | 配置端口C的引脚  | 0X0 |
| GPCDAT  | 0X56000024 | R/W | 端口C的数据寄存器 | 不确定 |
| GPCUP   | 0X56000028 | R/W | 端口C的上拉寄存器 | 0X0 |
| Reserve | 0X5600002C | —   | 保留        | 不确定 |

## GPCCON 各位的描述如下：

| GPCCON | 位       | 描述                                              |
|--------|---------|-------------------------------------------------|
| GPC15  | [31:30] | 00 = Input, 10 = VD[7],<br>01 = Output, 11 = 保留 |
| GPC14  | [29:28] | 00 = Input, 10 = VD[6],<br>01 = Output, 11 = 保留 |
| GPC13  | [27:26] | 00 = Input, 10 = VD[5],<br>01 = Output, 11 = 保留 |
| GPC12  | [25:24] | 00 = Input, 10 = VD[4],<br>01 = Output, 11 = 保留 |
| GPC11  | [23:22] | 00 = Input, 10 = VD[3],<br>01 = Output, 11 = 保留 |
| GPC10  | [21:20] | 00 = Input, 10 = VD[2],<br>01 = Output, 11 = 保留 |

## GPCCON 各位的描述如下（续）：

| GPCCON | 位       | 描述                                                    |
|--------|---------|-------------------------------------------------------|
| GPC9   | [19:18] | 00 = Input, 10 = VD[1],<br>01 = Output, 11 = 保留       |
| GPC8   | [17:16] | 00 = Input, 10 = VD[0],<br>01 = Output, 11 = 保留       |
| GPC7   | [15:14] | 00 = Input, 10 = LCD_LPCREVB,<br>01 = Output, 11 = 保留 |
| GPC6   | [13:12] | 00 = Input, 10 = LCD_LPCREV,<br>01 = Output, 11 = 保留  |
| GPC5   | [11:10] | 00 = Input, 10 = LCD_LPCOE,<br>01 = Output, 11 = 保留   |
| GPC4   | [9:8]   | 00 = Input, 10 = VM,<br>01 = Output, 11 = I2SSDI      |



## GPCCON 各位的描述如下（续）：

| GPCCON | 位     | 描述                                               |
|--------|-------|--------------------------------------------------|
| GPC3   | [7:6] | 00 = Input, 10 = VFRAME,<br>01 = Output, 11 = 保留 |
| GPC2   | [5:4] | 00 = Input, 10 = VLINE,<br>01 = Output, 11 = 保留  |
| GPC1   | [3:2] | 00 = Input, 10 = VCLK,<br>01 = Output, 11 = 保留   |
| GPC0   | [1:0] | 00 = Input, 10 = LEND,<br>01 = Output, 11 = 保留   |

## 5.7.2 I/O端口 寄存器及引脚配置

**GPCDAT** 为准备输出或输入的数据，其值为 16位[15:0]。

**注意：**当端口C 引脚配置为非输入/输出功能时，其寄存器中的值没有意义。

**GPCUP** 为端口C上拉寄存器，位 [15:0] 有意义。

**端口C 数据寄存器 GPCDAT** 中各位的描述如下：

## 5.7.2 I/O端口 寄存器及引脚配置

| GPCDAT    | 位      | 描述                                                                                                               |
|-----------|--------|------------------------------------------------------------------------------------------------------------------|
| GPC[15:0] | [15:0] | <p>当端口配置为输入端口时，外部源的数据可以从对应引脚读出；</p> <p>当端口配置为输出端口时，写入寄存器的数据会被发送到对应的引脚上；</p> <p>当端口配置为功能引脚时，如果读该位的值将是一个不确定的值。</p> |

**端口C 上拉寄存器 GPCUP 中各位的描述如下：**

| GPCUP     | 位      | 描述                                             |
|-----------|--------|------------------------------------------------|
| GPC[15:0] | [15:0] | <p>0：允许上拉电阻连接到对应脚；</p> <p>1：不允许上拉电阻连接到对应脚。</p> |

## 5.7.2 I/O端口 寄存器及引脚配置

(4) 端口D寄存器 (GPDCON、GPDDAT、GPDUP) 16位输入/输出端口

端口D 控制寄存器包括：端口D配置寄存器GPDCON、数据寄存器GPDDAT 和 上拉寄存器GPDUP，具体如下所示：

| 寄存器     | 地址         | R/W | 描述        | 复位值    |
|---------|------------|-----|-----------|--------|
| GPDCON  | 0X56000030 | R/W | 配置端口D的引脚  | 0X0    |
| GPDDAT  | 0X56000034 | R/W | 端口D的数据寄存器 | 不确定    |
| GPDUP   | 0X56000038 | R/W | 端口D的上拉寄存器 | 0Xf000 |
| Reserve | 0X5600003C | —   | 保留        | 不确定    |

## GPDCON 各位的描述如下：

| GPDCON | 位       | 描述                                                    |
|--------|---------|-------------------------------------------------------|
| GPD15  | [31:30] | 00 = Input, 10 = VD[23],<br>01 = Output, 11 = nSS0    |
| GPD14  | [29:28] | 00 = Input, 10 = VD[22],<br>01 = Output, 11 = nSS1    |
| GPD13  | [27:26] | 00 = Input, 10 = VD[21],<br>01 = Output, 11 = 保留      |
| GPD12  | [25:24] | 00 = Input, 10 = VD[20],<br>01 = Output, 11 = 保留      |
| GPD11  | [23:22] | 00 = Input, 10 = VD[19],<br>01 = Output, 11 = 保留      |
| GPD10  | [21:20] | 00 = Input, 10 = VD[18],<br>01 = Output, 11 = SPICLK1 |

## GPDCON 各位的描述如下（续）：

| GPDCON | 位       | 描述                                                     |
|--------|---------|--------------------------------------------------------|
| GPD9   | [19:18] | 00 = Input, 10 = VD[17],<br>01 = Output, 11 = SPIMOSI1 |
| GPD8   | [17:16] | 00 = Input, 10 = VD[16],<br>01 = Output, 11 = SPIMISO1 |
| GPD7   | [15:14] | 00 = Input, 10 = VD[15],<br>01 = Output, 11 = 保留       |
| GPD6   | [13:12] | 00 = Input, 10 = VD[14],<br>01 = Output, 11 = 保留       |
| GPD5   | [11:10] | 00 = Input, 10 = VD[13],<br>01 = Output, 11 = 保留       |
| GPD4   | [9:8]   | 00 = Input, 10 = VD[12],<br>01 = Output, 11 = 保留       |



## GPDCON 各位的描述如下（续）：

| GPDCON | 位     | 描述                                               |
|--------|-------|--------------------------------------------------|
| GPD3   | [7:6] | 00 = Input, 10 = VD[11],<br>01 = Output, 11 = 保留 |
| GPD2   | [5:4] | 00 = Input, 10 = VD[10],<br>01 = Output, 11 = 保留 |
| GPD1   | [3:2] | 00 = Input, 10 = VD[9],<br>01 = Output, 11 = 保留  |
| GPD0   | [1:0] | 00 = Input, 10 = VD[8],<br>01 = Output, 11 = 保留  |

## 5.7.2 I/O端口 寄存器及引脚配置

**GPDDAT** 为准备输出或输入的数据，其值为 16位[15:0]。

**注意：**当端口D引脚配置为非输入/输出功能时，其寄存器中的值没有意义。

**GPDUP** 为端口D上拉寄存器，位 [15:0] 有意义。初始化时，[15:12]无上拉功能，而 [11:0]有上拉。

## 5.7.2 I/O端口 寄存器及引脚配置

**端口D 数据寄存器 GPDDAT 中各位的描述如下：**

| GPDDAT    | 位      | 描述                                                                                                               |
|-----------|--------|------------------------------------------------------------------------------------------------------------------|
| GPD[15:0] | [15:0] | <p>当端口配置为输入端口时，外部源的数据可以从对应引脚读出；</p> <p>当端口配置为输出端口时，写入寄存器的数据会被发送到对应的引脚上；</p> <p>当端口配置为功能引脚时，如果读该位的值将是一个不确定的值。</p> |

**端口D 上拉寄存器 GPDUP 中各位的描述如下：**

| GPDUP     | 位      | 描述                                             |
|-----------|--------|------------------------------------------------|
| GPD[15:0] | [15:0] | <p>0：允许上拉电阻连接到对应脚；</p> <p>1：不允许上拉电阻连接到对应脚。</p> |

## 5.7.2 I/O端口 寄存器及引脚配置

### (5) 端口E寄存器（GPECON、GPEDAT、GPEUP） 16位输入/输出端口

端口E 控制寄存器包括：端口E配置寄存器GPECON、数据寄存器GPEDAT 和 上拉寄存器GPEUP，具体如下所示：

| 寄存器     | 地址         | R/W | 描述        | 复位值 |
|---------|------------|-----|-----------|-----|
| GPECON  | 0X56000040 | R/W | 配置端口E的引脚  | 0X0 |
| GPEDAT  | 0X56000044 | R/W | 端口E的数据寄存器 | 不确定 |
| GPEUP   | 0X56000048 | R/W | 端口E的上拉寄存器 | 0X0 |
| Reserve | 0X5600004C | —   | 保留        | 不确定 |

# GPECON 各位的描述如下：

| GPECON | 位       | 描述                                                              |
|--------|---------|-----------------------------------------------------------------|
| GPE15  | [31:30] | 00 = Input, 01 = Output, 10 = IICSDA,<br>11 = 保留, 此引脚开路, 无上拉电阻  |
| GPE14  | [29:28] | 00 = Input, 01 = Output, 10 = IIC_SCL,<br>11 = 保留, 此引脚开路, 无上拉电阻 |
| GPE13  | [27:26] | 00 = Input, 01 = Output,<br>10 = SPICLK0, 11 = 保留               |
| GPE12  | [25:24] | 00 = Input, 01 = Output,<br>10 = SPIMOSI0, 11 = 保留              |
| GPE11  | [23:22] | 00 = Input, 01 = Output,<br>10 = SPIMISO0, 11 = 保留              |
| GPE10  | [21:20] | 00 = Input, 01 = Output,<br>10 = SDDAT3, 11 = 保留                |

## GPECON 各位的描述如下（续）：

| GPECON | 位       | 描述                                                        |
|--------|---------|-----------------------------------------------------------|
| GPE9   | [19:18] | 00 = Input, 01 = Output,<br>10 = SDDAT2, 11 = 保留          |
| GPE8   | [17:16] | 00 = Input, 01 = Output,<br>10 = SDDAT1, 11 = 保留          |
| GPE7   | [15:14] | 00 = Input, 01 = Output,<br>10 = SDDAT0, 11 = 保留          |
| GPE6   | [13:12] | 00 = Input, 01 = Output,<br>10 = SDCMD, 11 = 保留           |
| GPE5   | [11:10] | 00 = Input, 01 = Output,<br>10 = SDCLK, 11 = 保留           |
| GPE4   | [9:8]   | 00 = Input, 01 = Output,<br>10 = I2SD0, 11 = AC_SDATA_OUT |



## GPECON 各位的描述如下（续）：

| GPECON | 位     | 描述                                                        |
|--------|-------|-----------------------------------------------------------|
| GPE3   | [7:6] | 00 = Input, 01 = Output,<br>10 = I2SDI, 11 = AC_SDATA_IN  |
| GPE2   | [5:4] | 00 = Input, 01 = Output,<br>10 = CDCLK, 11 = AC_nRESET    |
| GPE1   | [3:2] | 00 = Input, 01 = Output,<br>10 = I2SSCLK, 11 = AC_BIT_CLK |
| GPE0   | [1:0] | 00 = Input, 01 = Output,<br>10 = I2SLRCK, 11 = AC_SYNC    |

## 5.7.2 I/O端口 寄存器及引脚配置

**GPEDAT** 为准备输出或输入的数据，其值为 16位[15:0]。

**注意：**当端口E引脚配置为非输入/输出功能时，其寄存器中的值没有意义。

**GPEUP** 为端口E上拉寄存器，位 [15:0] 有意义。初始化时各个引脚都有上拉功能。

## 5.7.2 I/O端口 寄存器及引脚配置

**端口E 数据寄存器 GPEDAT 中各位的描述如下：**

| GPEDAT    | 位      | 描述                                                                                                               |
|-----------|--------|------------------------------------------------------------------------------------------------------------------|
| GPE[15:0] | [15:0] | <p>当端口配置为输入端口时，外部源的数据可以从对应引脚读出；</p> <p>当端口配置为输出端口时，写入寄存器的数据会被发送到对应的引脚上；</p> <p>当端口配置为功能引脚时，如果读该位的值将是一个不确定的值。</p> |

**端口E 上拉寄存器 GPEUP 中各位的描述如下：**

| GPEUP     | 位      | 描述                                             |
|-----------|--------|------------------------------------------------|
| GPE[15:0] | [15:0] | <p>0：允许上拉电阻连接到对应脚；</p> <p>1：不允许上拉电阻连接到对应脚。</p> |

## 5.7.2 I/O端口 寄存器及引脚配置

### (6) 端口F寄存器（GPFCON、GPFDAT、GPFUP）

8位输入/输出端口

GPFO~GPF7 用作 POWER\_OFF 模式中的唤醒信号，那么必须要将其配置为外部中断（中断模式中设置）。

端口F 控制寄存器包括：端口F配置寄存器GPFCON、数据寄存器GPFDAT 和 上拉寄存器GPFUP，具体如下所示：

| 寄存器     | 地址         | R/W | 描述        | 复位值 |
|---------|------------|-----|-----------|-----|
| GPFCON  | 0X56000050 | R/W | 配置端口F的引脚  | 0X0 |
| GPFDAT  | 0X56000054 | R/W | 端口F的数据寄存器 | 不确定 |
| GPFUP   | 0X56000058 | R/W | 端口F的上拉寄存器 | 0X0 |
| Reserve | 0X5600005C | —   | 保留        | 不确定 |

## GPFCON 各位的描述如下：

| GPFCON | 位       | 描述                                                |
|--------|---------|---------------------------------------------------|
| GPF7   | [15:14] | 00 = Input, 10 = EINT[7],<br>01 = Output, 11 = 保留 |
| GPF6   | [13:12] | 00 = Input, 10 = EINT[6],<br>01 = Output, 11 = 保留 |
| GPF5   | [11:10] | 00 = Input, 10 = EINT[5],<br>01 = Output, 11 = 保留 |
| GPF4   | [9:8]   | 00 = Input, 10 = EINT[4],<br>01 = Output, 11 = 保留 |
| GPF3   | [7:6]   | 00 = Input, 10 = EINT[3],<br>01 = Output, 11 = 保留 |
| GPF2   | [5:4]   | 00 = Input, 10 = EINT[2],<br>01 = Output, 11 = 保留 |

## GPFCON 各位的描述如下（续）：

| GPFCON | 位     | 描述                                                |
|--------|-------|---------------------------------------------------|
| GPF1   | [3:2] | 00 = Input, 10 = EINT[1],<br>01 = Output, 11 = 保留 |
| GPF0   | [1:0] | 00 = Input, 10 = EINT[0],<br>01 = Output, 11 = 保留 |



## 5.7.2 I/O端口 寄存器及引脚配置

**GPFDAT** 为准备输出或输入的数据，其值为 8位[7:0]。

**注意：**当端口F引脚配置为非输入/输出功能时，其寄存器中的值没有意义。

**GPFUP** 为端口F上拉寄存器，位 [7:0] 有意义。初始化时各个引脚都有上拉功能。

## 5.7.2 I/O端口 寄存器及引脚配置

端口F数据寄存器GPFDAT中各位的描述如下：

| GPFDAT    | 位     | 描述                                                                                                               |
|-----------|-------|------------------------------------------------------------------------------------------------------------------|
| GPF [7:0] | [7:0] | <p>当端口配置为输入端口时，外部源的数据可以从对应引脚读出；</p> <p>当端口配置为输出端口时，写入寄存器的数据会被发送到对应的引脚上；</p> <p>当端口配置为功能引脚时，如果读该位的值将是一个不确定的值。</p> |

端口F上拉寄存器GPFUP中各位的描述如下：

| GPFUP     | 位     | 描述                                             |
|-----------|-------|------------------------------------------------|
| GPF [7:0] | [7:0] | <p>0：允许上拉电阻连接到对应脚；</p> <p>1：不允许上拉电阻连接到对应脚。</p> |

## 5.7.2 I/O端口 寄存器及引脚配置

### (7) 端口G寄存器 (GPGCON、GPGDAT、GPGUP) 16位输入/输出端口

GPG0~GPG7 用作 POWER\_OFF 模式中的 唤醒信号，那么必须要将其配置为外部中断（中断模式中设置）。

端口G 控制寄存器包括：端口G配置寄存器GPGCON、数据寄存器GPGDAT 和 上拉寄存器GPGUP，具体如下所示：

## 5.7.2 I/O端口 寄存器及引脚配置

| 寄存器     | 地址         | R/W | 描述        | 复位值    |
|---------|------------|-----|-----------|--------|
| GPGCON  | 0X56000060 | R/W | 配置端口G的引脚  | 0X0    |
| GPGDAT  | 0X56000064 | R/W | 端口G的数据寄存器 | 不确定    |
| GPGUP   | 0X56000068 | R/W | 端口G的上拉寄存器 | 0Xfc00 |
| Reserve | 0X5600006C | —   | 保留        | 不确定    |

## GPGCON 各位的描述如下：

| GPGCON | 位       | 描述                                                      |
|--------|---------|---------------------------------------------------------|
| GPG15* | [31:30] | 00 = Input, 10 = EINT[23],<br>01 = Output, 11 = 保留      |
| GPG14* | [29:28] | 00 = Input, 10 = EINT[22],<br>01 = Output, 11 = 保留      |
| GPG13* | [27:26] | 00 = Input, 10 = EINT[21],<br>01 = Output, 11 = 保留      |
| GPG12  | [25:24] | 00 = Input, 10 = EINT[20],<br>01 = Output, 11 = 保留      |
| GPG11  | [23:22] | 00 = Input, 10 = EINT[19],<br>01 = Output, 11 = TCLK[1] |
| GPG10  | [21:20] | 00 = Input, 10 = EINT[18],<br>01 = Output, 11 = nCTS1   |

## GPGCON 各位的描述如下（续）：

| GPGCON | 位       | 描述                                                        |
|--------|---------|-----------------------------------------------------------|
| GPG9   | [19:18] | 00 = Input, 10 = EINT[17],<br>01 = Output, 11 = nRTS1     |
| GPG8   | [17:16] | 00 = Input, 10 = EINT[16],<br>01 = Output, 11 = 保留        |
| GPG7   | [15:14] | 00 = Input, 10 = EINT[15],<br>01 = Output, 11 = SPICLK1   |
| GPG6   | [13:12] | 00 = Input, 10 = EINT[14],<br>01 = Output, 11 = SPIMOSI1  |
| GPG5   | [11:10] | 00 = Input, 10 = EINT[13],<br>01 = Output, 11 = SPIMISO1  |
| GPG4   | [9:8]   | 00 = Input, 10 = EINT[12],<br>01 = Output, 11 = LCD_PWRDN |



## GPGCON 各位的描述如下（续）：

| GPGCON | 位     | 描述                                                   |
|--------|-------|------------------------------------------------------|
| GPG3   | [7:6] | 00 = Input, 10 = EINT[11],<br>01 = Output, 11 = nSS1 |
| GPG2   | [5:4] | 00 = Input, 10 = EINT[10],<br>01 = Output, 11 = nSS0 |
| GPG1   | [3:2] | 00 = Input, 10 = EINT[9],<br>01 = Output, 11 = 保留    |
| GPG0   | [1:0] | 00 = Input, 10 = EINT[8],<br>01 = Output, 11 = 保留    |

**注：在NAND引导方式时，GPG[15: 13]必须选择输入。**

## 5.7.2 I/O端口 寄存器及引脚配置

**GPGDAT** 为准备输出或输入的数据，其值为 **16位[15:0]**。

**GPGUP** 为端口G上拉寄存器，位 **[15:0]** 有意义。初始化时 **[15:10]** 引脚无上拉功能（对应bit位值为1），其它bit位值为0。

**注意：**当端口G引脚配置为非输入/输出功能时，其寄存器中的值没有意义。

## 5.7.2 I/O端口 寄存器及引脚配置

**端口G 数据寄存器 GPGDAT 中各位的描述如下：**

| GPGDAT    | 位      | 描述                                                                                                               |
|-----------|--------|------------------------------------------------------------------------------------------------------------------|
| GPG[15:0] | [15:0] | <p>当端口配置为输入端口时，外部源的数据可以从对应引脚读出；</p> <p>当端口配置为输出端口时，写入寄存器的数据会被发送到对应的引脚上；</p> <p>当端口配置为功能引脚时，如果读该位的值将是一个不确定的值。</p> |

**端口G 上拉寄存器 GPGUP 中各位的描述如下：**

| GPGUP     | 位      | 描述                                             |
|-----------|--------|------------------------------------------------|
| GPG[15:0] | [15:0] | <p>0：允许上拉电阻连接到对应脚；</p> <p>1：不允许上拉电阻连接到对应脚。</p> |

## 5.7.2 I/O端口 寄存器及引脚配置

### (8) 端口H寄存器（GPHCON、GPHDAT、GPHUP） 11位输入/输出端口

端口H 控制寄存器包括：端口H配置寄存器GPHCON、数据寄存器GPHDAT 和 上拉寄存器GPHUP，具体如下所示：

| 寄存器     | 地址         | R/W | 描述        | 复位值 |
|---------|------------|-----|-----------|-----|
| GPHCON  | 0X56000070 | R/W | 配置端口H的引脚  | 0X0 |
| GPHDAT  | 0X56000074 | R/W | 端口H的数据寄存器 | 不确定 |
| GPHUP   | 0X56000078 | R/W | 端口H的上拉寄存器 | 0X0 |
| Reserve | 0X5600007C | —   | 保留        | 不确定 |

## GPHCON 各位的描述如下：

| GPHCON | 位       | 描述                                                  |
|--------|---------|-----------------------------------------------------|
| GPH10  | [21:20] | 00 = Input, 10 = CLKOUT1,<br>01 = Output, 11 = 保留   |
| GPH9   | [19:18] | 00 = Input, 10 = CLKOUT0,<br>01 = Output, 11 = 保留   |
| GPH8   | [17:16] | 00 = Input, 10 = UEXTCLK,<br>01 = Output, 11 = 保留   |
| GPH7   | [15:14] | 00 = Input, 10 = RXD[2],<br>01 = Output, 11 = nCTS1 |
| GPH6   | [13:12] | 00 = Input, 10 = TXD[2],<br>01 = Output, 11 = nRTS1 |
| GPH5   | [11:10] | 00 = Input, 10 = RXD[1],<br>01 = Output, 11 = 保留    |

## GPHCON 各位的描述如下（续）：

| GPHCON | 位     | 描述                                               |
|--------|-------|--------------------------------------------------|
| GPH4   | [9:8] | 00 = Input, 10 = TXD[1],<br>01 = Output, 11 = 保留 |
| GPH3   | [7:6] | 00 = Input, 10 = RXD[0],<br>01 = Output, 11 = 保留 |
| GPH2   | [5:4] | 00 = Input, 10 = TXD[0],<br>01 = Output, 11 = 保留 |
| GPH1   | [3:2] | 00 = Input, 10 = nRTS0,<br>01 = Output, 11 = 保留  |
| GPH0   | [1:0] | 00 = Input, 10 = nCTS0,<br>01 = Output, 11 = 保留  |



## 5.7.2 I/O端口 寄存器及引脚配置

**GPHDAT** 为准备输出或输入的数据，其值为 11位[10:0]。

**注意：**当端口H引脚配置为非输入/输出功能时，其寄存器中的值没有意义。

**GPHUP** 为端口H上拉寄存器，位[10:0]有意义。

## 5.7.2 I/O端口 寄存器及引脚配置

**端口H 数据寄存器 GPHDAT 中各位的描述如下：**

| GPHDAT    | 位      | 描述                                                                                                               |
|-----------|--------|------------------------------------------------------------------------------------------------------------------|
| GPH[10:0] | [10:0] | <p>当端口配置为输入端口时，外部源的数据可以从对应引脚读出；</p> <p>当端口配置为输出端口时，写入寄存器的数据会被发送到对应的引脚上；</p> <p>当端口配置为功能引脚时，如果读该位的值将是一个不确定的值。</p> |

**端口H 上拉寄存器 GPHUP 中各位的描述如下：**

| GPHUP     | 位      | 描述                                             |
|-----------|--------|------------------------------------------------|
| GPH[10:0] | [10:0] | <p>0：允许上拉电阻连接到对应脚；</p> <p>1：不允许上拉电阻连接到对应脚。</p> |

## 5.7.2 I/O端口 寄存器及引脚配置

### (9) 端口J寄存器（GPJCON、GPJDAT、GPJUP） 13位输入/输出端口

端口J 控制寄存器包括：端口J配置寄存器GPJCON、数据寄存器GPJDAT 和 上拉寄存器GPJUP，具体如下所示：

| 寄存器     | 地址         | R/W | 描述        | 复位值 |
|---------|------------|-----|-----------|-----|
| GPJCON  | 0X560000D0 | R/W | 配置端口J的引脚  | 0X0 |
| GPJDAT  | 0X560000D4 | R/W | 端口J的数据寄存器 | 不确定 |
| GPJUP   | 0X560000D8 | R/W | 端口J的上拉寄存器 | 0X0 |
| Reserve | 0X560000DC | —   | 保留        | 不确定 |

# GPJCON 各位的描述如下:

| GPJCON | 位       | 描述                                                   |
|--------|---------|------------------------------------------------------|
| GPJ12  | [25:24] | 00 = Input, 10 = CAMRESET,<br>01 = Output, 11 = 保留   |
| GPJ11  | [23:22] | 00 = Input, 10 = CAMCLKOUT,<br>01 = Output, 11 = 保留  |
| GPJ10  | [21:20] | 00 = Input, 10 = CAMHREF,<br>01 = Output, 11 = 保留    |
| GPJ9   | [19:18] | 00 = Input, 10 = CAMVSYNC,<br>01 = Output, 11 = 保留   |
| GPJ8   | [17:16] | 00 = Input, 10 = CAMPCLK,<br>01 = Output, 11 = 保留    |
| GPJ7   | [15:14] | 00 = Input, 10 = CAMDATA[7],<br>01 = Output, 11 = 保留 |

## GPJCON 各位的描述如下（续）：

|      |         |                                                      |
|------|---------|------------------------------------------------------|
| GPJ6 | [13:12] | 00 = Input, 10 = CAMDATA[6],<br>01 = Output, 11 = 保留 |
| GPJ5 | [11:10] | 00 = Input, 10 = CAMDATA[5],<br>01 = Output, 11 = 保留 |
| GPJ4 | [9:8]   | 00 = Input, 10 = CAMDATA[4],<br>01 = Output, 11 = 保留 |
| GPJ3 | [7:6]   | 00 = Input, 10 = CAMDATA[3],<br>01 = Output, 11 = 保留 |
| GPJ2 | [5:4]   | 00 = Input, 10 = CAMDATA[2],<br>01 = Output, 11 = 保留 |
| GPJ1 | [3:2]   | 00 = Input, 10 = CAMDATA[1],<br>01 = Output, 11 = 保留 |
| GPJ0 | [1:0]   | 00 = Input, 10 = CAMDATA[0],<br>01 = Output, 11 = 保留 |

## 5.7.2 I/O端口 寄存器及引脚配置

**端口J 数据寄存器 GPJDAT 中各位的描述如下：**

| GPJDAT    | 位      | 描述                                                                                                               |
|-----------|--------|------------------------------------------------------------------------------------------------------------------|
| GPJ[12:0] | [12:0] | <p>当端口配置为输入端口时，外部源的数据可以从对应引脚读出；</p> <p>当端口配置为输出端口时，写入寄存器的数据会被发送到对应的引脚上；</p> <p>当端口配置为功能引脚时，如果读该位的值将是一个不确定的值。</p> |

**端口J 上拉寄存器 GPJUP 中各位的描述如下：**

| GPJUP     | 位      | 描述                                             |
|-----------|--------|------------------------------------------------|
| GPJ[12:0] | [12:0] | <p>0：允许上拉电阻连接到对应脚；</p> <p>1：不允许上拉电阻连接到对应脚。</p> |



## 5.7.2 I/O端口 寄存器及引脚配置

### (12) 外部中断控制寄存器 (EXTINTn)

**24个外部中断** 可以有不同的信号方式请求。**EXTINTn** 为外部中断请求，配置信号的方式为：**电平触发** 或者 **边沿触发**，也可以配置信号极性。

**为了能检测出电平触发中断，不受噪声滤波器的影响，EXTINTn引脚的有效逻辑电平至少 要维持 40ns。外部中断控制器 EXTINTn 如下所示。**

| 寄存器     | 地址         | R/W | 描述         | 复位值 |
|---------|------------|-----|------------|-----|
| EXTINT0 | 0X56000088 | R/W | 外部中断控制寄存器0 | 0X0 |
| EXTINT1 | 0X5600008C | W/R | 外部中断控制寄存器1 | 0X0 |
| EXTINT2 | 0X56000090 | W/R | 外部中断控制寄存器2 | 0X0 |

## EXTINT0 中各位的描述如下：

| EXTINT0 | 位       | 描述                                                                  |
|---------|---------|---------------------------------------------------------------------|
| EINT7   | [30:28] | 设置EINT7的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| EINT6   | [26:24] | 设置EINT6的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| EINT5   | [22:20] | 设置EINT5的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |

# EXTINT0 中各位的描述如下（续）：

| EXTINT0 | 位       | 描述                                                                  |
|---------|---------|---------------------------------------------------------------------|
| EINT4   | [18:16] | 设置EINT4的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| EINT3   | [14:12] | 设置EINT3的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| EINT2   | [10:8]  | 设置EINT2的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| EINT1   | [6:4]   | 设置EINT1的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| EINT0   | [2:0]   | 设置EINT0的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |

# EXTINT1 中各位的描述如下:

| EXTINT1 | 位       | 描述                                                                        |
|---------|---------|---------------------------------------------------------------------------|
| FLTEN15 | [31]    | EINT15滤波器使能: 0 = 无滤波器, 1 = 滤波器使能                                          |
| EINT15  | [30:28] | 设置EINT15的信号方式: 000 = 低电平, 001 = 高电平, 01x = 下降沿触发, 10x = 上升沿触发, 11x = 边沿触发 |
| FLTEN14 | [27]    | EINT14滤波器使能: 0 = 无滤波器, 1 = 滤波器使能                                          |
| EINT14  | [26:24] | 设置EINT14的信号方式: 000 = 低电平, 001 = 高电平, 01x = 下降沿触发, 10x = 上升沿触发, 11x = 边沿触发 |
| FLTEN13 | [23]    | EINT13滤波器使能: 0 = 无滤波器, 1 = 滤波器使能                                          |
| EINT13  | [22:20] | 设置EINT13的信号方式: 000 = 低电平, 001 = 高电平, 01x = 下降沿触发, 10x = 上升沿触发, 11x = 边沿触发 |

## EXTINT1 中各位的描述如下（续）：

| EXTINT1 | 位       | 描述                                                                   |
|---------|---------|----------------------------------------------------------------------|
| FLTEN12 | [19]    | EINT12滤波器使能：0 = 无滤波器，1 = 滤波器使能                                       |
| EINT12  | [18:16] | 设置EINT12的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| FLTEN11 | [15]    | EINT11滤波器使能：0 = 无滤波器，1 = 滤波器使能                                       |
| EINT11  | [14:12] | 设置EINT11的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| FLTEN10 | [11]    | EINT10滤波器使能：0 = 无滤波器，1 = 滤波器使能                                       |
| EINT10  | [10:8]  | 设置EINT10的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |

## EXTINT1 中各位的描述如下（续）：

| EXTINT1 | 位     | 描述                                                                  |
|---------|-------|---------------------------------------------------------------------|
| FLTEN9  | [7]   | EINT9滤波器使能：0 = 无滤波器，1 = 滤波器使能                                       |
| EINT9   | [6:4] | 设置EINT9的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| FLTEN8  | [3]   | EINT8滤波器使能：0 = 无滤波器，1 = 滤波器使能                                       |
| EINT8   | [2:0] | 设置EINT8的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |



# EXTINT2 中各位的描述如下:

| EXTINT2 | 位       | 描述                                                                        |
|---------|---------|---------------------------------------------------------------------------|
| FLTEN23 | [31]    | EINT23滤波器使能: 0 = 无滤波器, 1 = 滤波器使能                                          |
| EINT23  | [30:28] | 设置EINT23的信号方式: 000 = 低电平, 001 = 高电平, 01x = 下降沿触发, 10x = 上升沿触发, 11x = 边沿触发 |
| FLTEN22 | [27]    | EINT22滤波器使能: 0 = 无滤波器, 1 = 滤波器使能                                          |
| EINT22  | [26:24] | 设置EINT22的信号方式: 000 = 低电平, 001 = 高电平, 01x = 下降沿触发, 10x = 上升沿触发, 11x = 边沿触发 |
| FLTEN21 | [23]    | EINT21滤波器使能: 0 = 无滤波器, 1 = 滤波器使能                                          |
| EINT21  | [22:20] | 设置EINT21的信号方式: 000 = 低电平, 001 = 高电平, 01x = 下降沿触发, 10x = 上升沿触发, 11x = 边沿触发 |

## EXTINT2 中各位的描述如下（续）：

| EXTINT2 | 位       | 描述                                                                   |
|---------|---------|----------------------------------------------------------------------|
| FLTEN20 | [19]    | EINT20滤波器使能：0 = 无滤波器，1 = 滤波器使能                                       |
| EINT20  | [18:16] | 设置EINT20的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| FLTEN19 | [15]    | EINT19滤波器使能：0 = 无滤波器，1 = 滤波器使能                                       |
| EINT19  | [14:12] | 设置EINT19的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| FLTEN18 | [11]    | EINT18滤波器使能：0 = 无滤波器，1 = 滤波器使能                                       |
| EINT18  | [10:8]  | 设置EINT18的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |

## EXTINT2 中各位的描述如下（续）：

| EXTINT2 | 位     | 描述                                                                   |
|---------|-------|----------------------------------------------------------------------|
| FLTEN17 | [7]   | EINT17滤波器使能：0 = 无滤波器，1 = 滤波器使能                                       |
| EINT17  | [6:4] | 设置EINT17的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |
| FLTEN16 | [3]   | EINT16滤波器使能：0 = 无滤波器，1 = 滤波器使能                                       |
| EINT16  | [2:0] | 设置EINT16的信号方式：000 = 低电平，001 = 高电平，01x = 下降沿触发，10x = 上升沿触发，11x = 边沿触发 |

## 5.7.2 I/O端口 寄存器及引脚配置

### (13) 外部中断过滤寄存器 (EINTFLTn) (了解)

要确认中断等级，在 EXTINTn 引脚上正确的逻辑电平必须保留 40ns 噪音滤波。

外部中断过滤寄存器的地址如下：

| 寄存器      | 地址         | R/W | 描述         | 复位值 |
|----------|------------|-----|------------|-----|
| EINTFLT0 | 0X56000094 | R/W | 保留         |     |
| EINTFLT1 | 0X56000098 | R/W | 保留         |     |
| EINTFLT2 | 0X5600009C | R/W | 外部中断过滤寄存器2 | 0X0 |
| EINTFLT3 | 0X560000A0 | R/W | 外部中断过滤寄存器3 | 0X0 |

## 5.7.2 I/O端口 寄存器及引脚配置

外部中断过滤寄存器各位的描述如下：

| EINTFLT2  | Bit     | 描述                                              |
|-----------|---------|-------------------------------------------------|
| EINTFLT19 | [30:24] | EINT19滤波宽度                                      |
| FLTCLK18  | [23]    | EINT18滤波时钟(通过OM设置)：0 = PCLK, 1= EXTCLK/OSC_CLK  |
| EINTFLT18 | [22:16] | EINT18滤波宽度                                      |
| FLTCLK17  | [15]    | EINT17滤波时钟(通过OM设置)：0 = PCLK, 1= EXTCLK/OSC_CLK  |
| EINTFLT17 | [14:8]  | EINT17滤波宽度                                      |
| FLTCLK16  | [7]     | 滤波时钟EINT16 (通过OM设置)：0 = PCLK, 1= EXTCLK/OSC_CLK |
| EINTFLT16 | [6:0]   | 滤波宽度EINT16                                      |

## 5.7.2 I/O端口 寄存器及引脚配置

外部中断过滤寄存器各位的描述如下：

| EINTFLT3  | Bit     | 描述                                             |
|-----------|---------|------------------------------------------------|
| FLTCLK23  | [31]    | 滤波时钟EINT23（通过OM设置）：0 = PCLK, 1= EXTCLK/OSC_CLK |
| EINTFLT23 | [30:24] | 滤波宽度EINT23                                     |
| FLTCLK22  | [23]    | 滤波时钟EINT22（通过OM设置）：0 = PCLK, 1= EXTCLK/OSC_CLK |
| EINTFLT22 | [22:16] | 滤波宽度EINT22                                     |
| FLTCLK21  | [15]    | 滤波时钟EINT21（通过OM设置）：0 = PCLK, 1= EXTCLK/OSC_CLK |
| EINTFLT21 | [14:8]  | 滤波宽度EINT21                                     |
| FLTCLK20  | [7]     | 滤波时钟EINT20（通过OM设置）：0 = PCLK, 1= EXTCLK/OSC_CLK |
| EINTFLT20 | [6:0]   | 滤波宽度EINT20                                     |



## 5.7.2 I/O端口 寄存器及引脚配置

### (14) 外部中断屏蔽寄存器 (EINTMASK)

针对 20个 外部中断 (EINT[23:4]) 的中断屏蔽寄存器 (即EINTMASK)，具体信息如下：

| 寄存器      | 地址         | R/W | 描述        | 复位值         |
|----------|------------|-----|-----------|-------------|
| EINTMASK | 0X560000A4 | R/W | 外部中断屏蔽寄存器 | 0X00FFFFFF0 |

外部中断屏蔽寄存器各位说明如下：

| EINTMASK | 位    | 描述             |
|----------|------|----------------|
| EINT23   | [23] | 0=允许中断, 1=屏蔽中断 |
| EINT22   | [22] | 0=允许中断, 1=屏蔽中断 |
| EINT21   | [21] | 0=允许中断, 1=屏蔽中断 |
| EINT20   | [20] | 0=允许中断, 1=屏蔽中断 |
| EINT19   | [19] | 0=允许中断, 1=屏蔽中断 |
| EINT18   | [18] | 0=允许中断, 1=屏蔽中断 |
| EINT17   | [17] | 0=允许中断, 1=屏蔽中断 |
| EINT16   | [16] | 0=允许中断, 1=屏蔽中断 |
| EINT15   | [15] | 0=允许中断, 1=屏蔽中断 |
| EINT14   | [14] | 0=允许中断, 1=屏蔽中断 |
| EINT13   | [13] | 0=允许中断, 1=屏蔽中断 |
| EINT12   | [12] | 0=允许中断, 1=屏蔽中断 |
| EINT11   | [11] | 0=允许中断, 1=屏蔽中断 |
| EINT10   | [10] | 0=允许中断, 1=屏蔽中断 |

| EINTMASK | 位   | 描述             |
|----------|-----|----------------|
| EINT9    | [9] | 0=允许中断, 1=屏蔽中断 |
| EINT8    | [8] | 0=允许中断, 1=屏蔽中断 |
| EINT7    | [7] | 0=允许中断, 1=屏蔽中断 |
| EINT6    | [6] | 0=允许中断, 1=屏蔽中断 |
| EINT5    | [5] | 0=允许中断, 1=屏蔽中断 |
| EINT4    | [4] | 0=允许中断, 1=屏蔽中断 |
| EINT3    | [3] | 保留             |
| EINT2    | [2] | 保留             |
| EINT1    | [1] | 保留             |
| EINT0    | [0] | 保留             |

## 5.7.2 I/O端口 寄存器及引脚配置

### (15) 外部中断挂起寄存器 (EINTPENDn)

外部中断挂起寄存器 **EINTPENDn** 是针对 **20个** 外部中断 (**EINT[23:4]**) 的, 可以通过向此 **EINTPEND** 寄存器相应位 **写1来清除比特位**。

| 寄存器      | 地址         | R/W | 描述        | 复位值  |
|----------|------------|-----|-----------|------|
| EINTPEND | 0X560000A8 | R/W | 外部中断挂起寄存器 | 0X00 |

外部中断挂起寄存器各位说明如下:

| EINTPEND | 位    | 描述                      |
|----------|------|-------------------------|
| EINT23   | [23] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT22   | [22] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT21   | [21] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT20   | [20] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT19   | [19] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT18   | [18] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT17   | [17] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT16   | [16] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT15   | [15] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT14   | [14] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT13   | [13] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT12   | [12] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT11   | [11] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT10   | [10] | 通过写“1”清除，0：没发生中断，1：发生中断 |

| EINTPEND | 位   | 描述                      |
|----------|-----|-------------------------|
| EINT9    | [9] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT8    | [8] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT7    | [7] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT6    | [6] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT5    | [5] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT4    | [4] | 通过写“1”清除，0：没发生中断，1：发生中断 |
| EINT3    | [3] | 保留                      |
| EINT2    | [2] | 保留                      |
| EINT1    | [1] | 保留                      |
| EINT0    | [0] | 保留                      |



## 5.8 PWM定时器

**S3C2440** 定时器的主要特性如下：

- (1) 5个16 位定时器；
- (2) 2个8位**预分频器** 和 2个4位**分频器**  
(时钟除法器) ；
- (3) 输出波形的占空比可编程控制（PWM） ；
- (4) 自动加载模式 或 单触发脉冲模式；
- (5) 死区产生器。

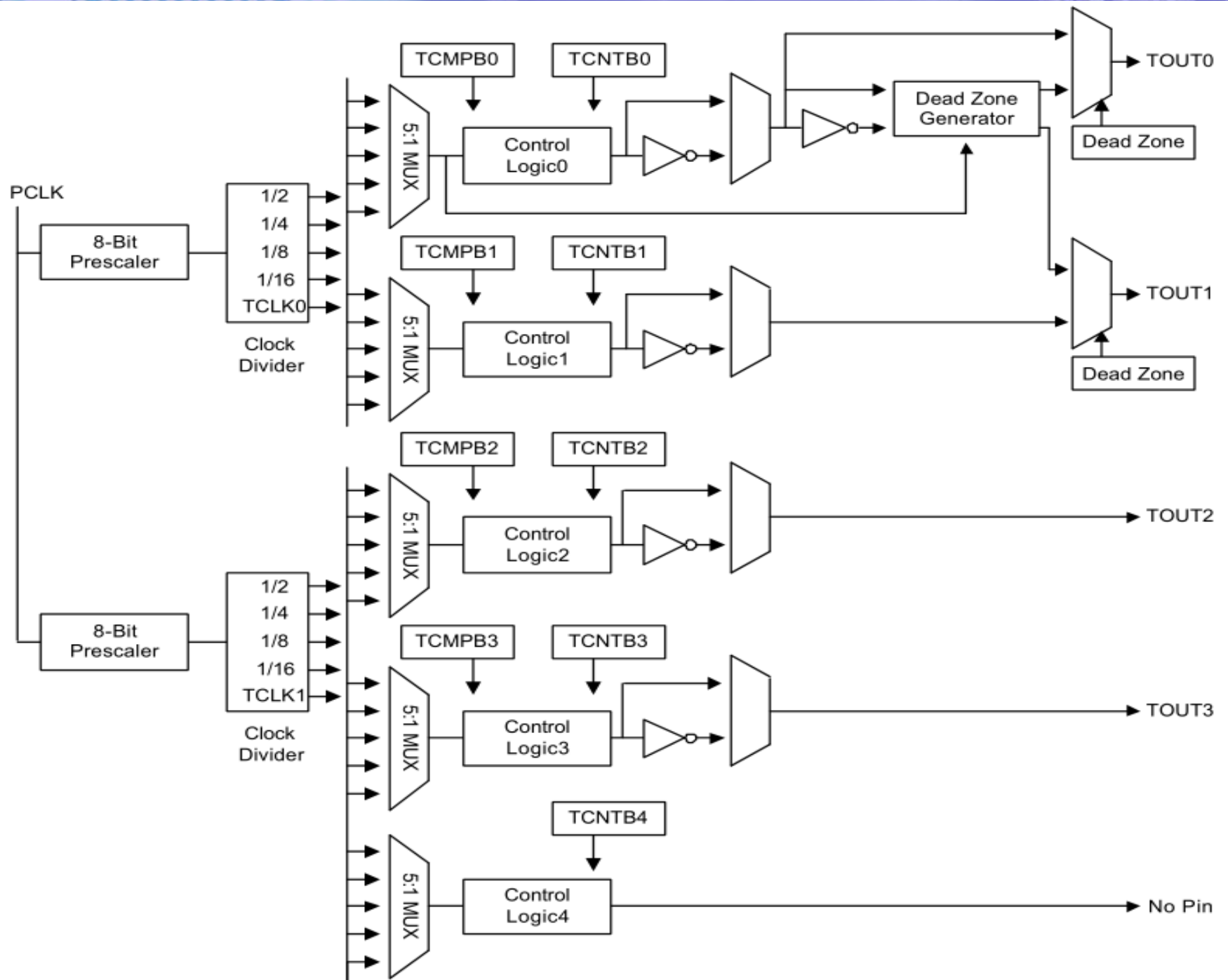
## 5.8 PWM定时器

S3C2440 具有 5个16位定时器，定时器0、1、2、和3 具有PWM功能（脉宽调制），定时器4 是一个内部定时器，不具有对外输出口线。定时器0 具有死区发生器，通常用于大电流设备应用。

定时器0和1 公用 一个8位预分频器，定时器2、3和4 公用 另一个8位预分频器。每一个定时器通过除法器可以得到5种不同的信号（ $1/2$ 、 $1/4$ 、 $1/8$ 、 $1/16$ 和TCLK）。

## 5.8 PWM定时器

每一个时钟模块 从其时钟除法器中 接收时钟信号，时钟除法器 从其相应的 8位预分频器 中接收时钟信号。8位预分频器 是可编程的，并根据存储在 TCFG0 和 TCFG1 的值来对 PCLK信号 进行分频。



## 5.8.1 PWM 概念

**PWM 概念（脉宽调制）就是只对 一方波序列信号的占空比 按要求进行调制，而不是 改变方波信号的其它参数，即不改变 幅度和周期，因此脉宽调制信号的产生和传输，都是数字式的。**

## 5.8.1 PWM 概念

用脉宽调制技术 可以实现模拟信号：如果调制信号的 频率远远大于 信号接受者的分辨率，则接受者获得的是信号平均效果，不能感知数字信号的 0和1 ，其信号大小的平均值与信号的占空比有关，信号的占空比越大，平均信号越强，其平均值与占空比成正比。只要带宽足够（频率足够高或周期足够短），任何模拟信号都可以使用 PWM 来实现。



## 5.8.1 PWM 概念

通过对一系列脉冲的宽度进行调制，等效出所需要的波形（含形状和幅值）。

### 理论基础

**面积等效原理**——冲量相等而形状不同的窄脉冲加在具有惯性的环节上时，其效果基本相同。

冲量



窄脉冲的面积

效果基本相同



环节的输出响应波形基本相同

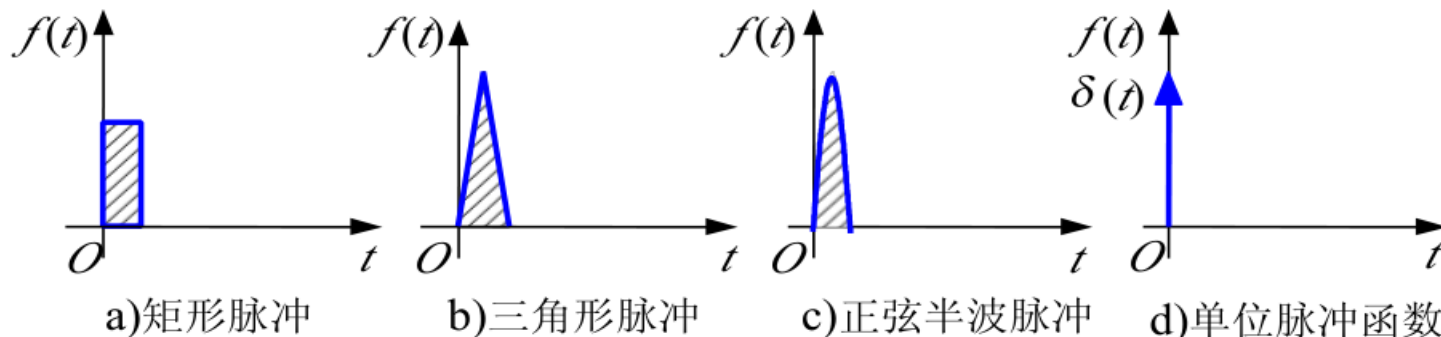
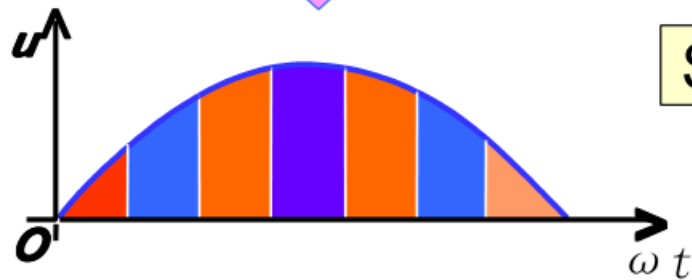
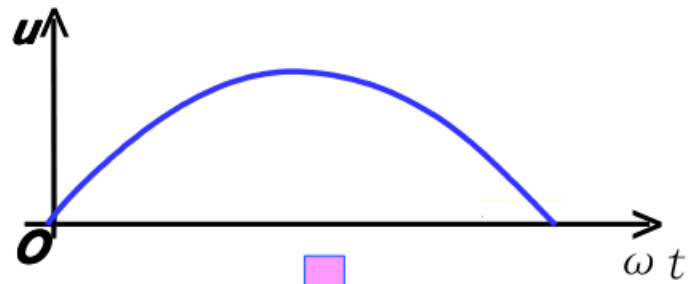


图6-1 形状不同而冲量相同的各种窄脉冲

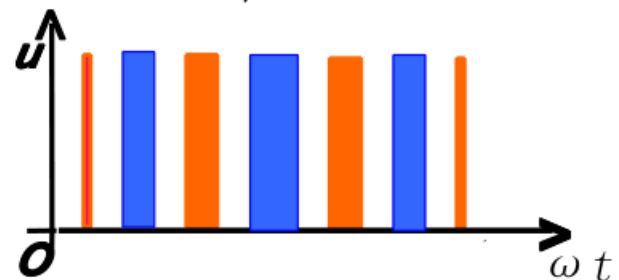
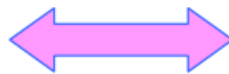
## 5.8.1 PWM 概念

### PWM的调制思想

如何用一系列脉冲来代替一个正弦半波？



SPWM波



固定幅值，使波形调制方案更具可实现性

## 5.8.1 PWM 概念

**PWM 技术的应用：**借助于微处理器，使用脉宽调制方法实现模拟信号是一种非常有效的技术，广泛应用于：从测量、通信到功率控制与变换的许多领域中。

## 5.8.2 PWM 定时器结构

- 定时器计数缓冲寄存器 (TCNTBn) 的值 是当定时器使能时装载到减法计数器的 初值; (周期)
- 定时器比较缓冲器 (TCMPBn) 的值 将被装载到比较寄存器并与减法计数器的值进行比较; (占空比)
- TCNTBn 和 TCMPBn 双重缓冲器的特性使定时器频率 和 占空比改变时, 确保产生稳定的输出。

## 5.8.2 PWM 定时器结构

每个计数器都有自己的 16位减法器，由定时器时钟驱动。当定时器计数器值达到0时，定时器发出中断请求，通知 CPU 定时工作已经完成，相应的 TCNTB<sub>n</sub> 将自动装入计数器，以继续下一个操作。但是，如果，定时器控制寄存器 TCON<sub>n</sub> 中的自动重载开/关被清除，则 TCNTB<sub>n</sub> 中的值将不会被装载到计数器。

## 5.8.2 PWM 定时器结构

**TCMPBn** 的值用于脉宽调节，当该计数器值与定时器控制逻辑中的比较寄存器值相等时，定时控制逻辑改变输出电平。因此，比较寄存器决定 **PWM** 输出的高电平时间（或低电平时间）。



## 5.8.3 PWM 定时器操作

### (1) 预分频器 和 分频器（时钟除法器）

一个 8位 的预分频器和一个 4位 分频器作用下的输出频率如下所示：

| 4位分频器的设置            | 最小分辨率<br>(预分频器=0)      | 最大分辨率<br>(预分频器=255)    | 最大间隔时间<br>(TCNTBn=65535) |
|---------------------|------------------------|------------------------|--------------------------|
| 1/2 (PCLK=66.5MHZ)  | 0.0300us (33.2500MHZ)  | 7.6992us (129.8828kHz) | 0.5045s                  |
| 1/4 (PCLK=66.5MHZ)  | 0.06001us (16.6250MHZ) | 15.3984us (64.9414kHz) | 1.0091s                  |
| 1/8 (PCLK=66.5MHZ)  | 0.1203us (8.3125MHZ)   | 30.7968 (32.4707kHz)   | 2.0182s                  |
| 1/16 (PCLK=66.5MHZ) | 0.2406us (4.1562MHZ)   | 61.5936us (16.2353kHz) | 4.0365s                  |

## 5.8.2 PWM 定时器结构

### (2) 定时器基本操作

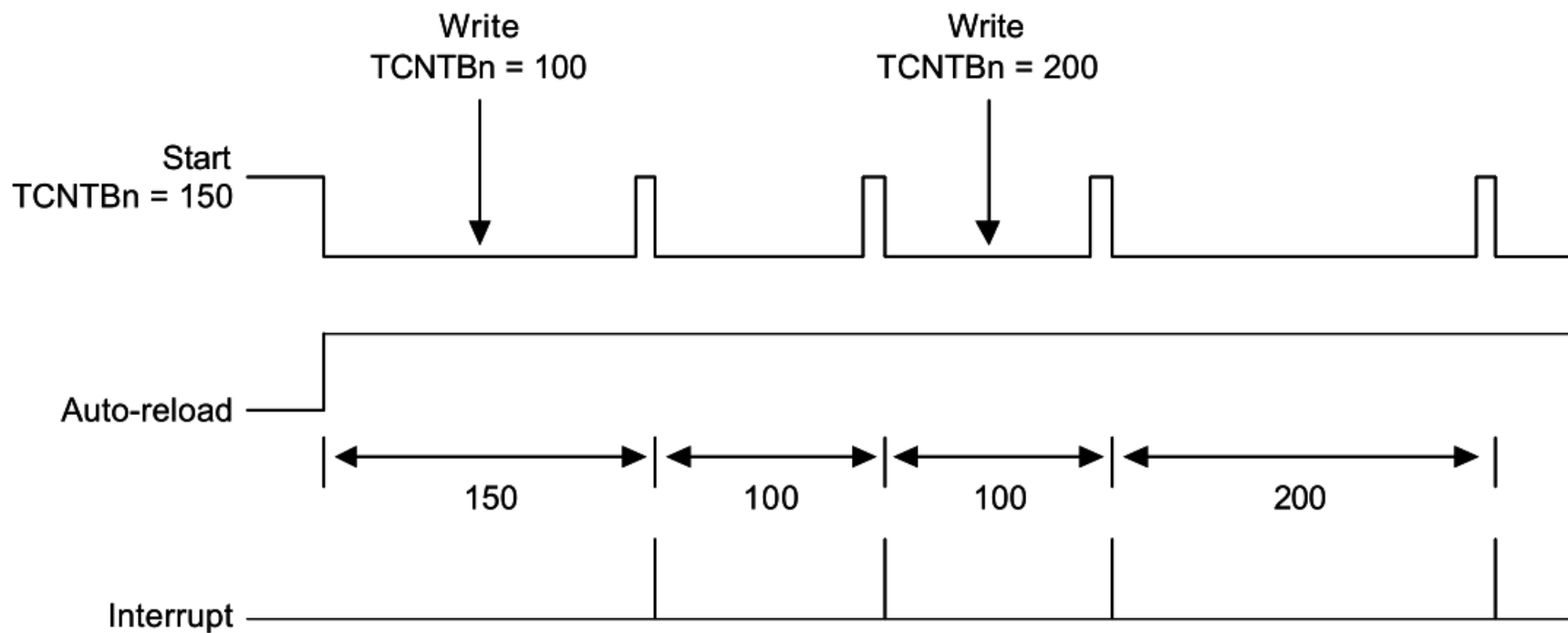
一个定时器（定时器4除外）包含：TCNTBn、TCNTn、TCMPBn和TCMPn几个寄存器（TCNTn和TCMPn是减法计数器内部寄存器的名称，TCNTn的值可以通过读TCNT0n得到）。当定时器达到0时，TCNTBn 和 TCMPBn的值将自动加载到TCNTn 和 TCMPn 中。当 TCNTn到0且中断使能时，定时器将产生一个中断请求。

## 5.8.2 PWM 定时器结构

### (3) 自动加载 和 双缓冲模式

脉宽调制定时器 **有一个** 双缓冲功能，在这种情况下，改变下次加载值的同时 **不影响** 当前定时周期。因此，尽管设置一个 **新的定时器值**，当前定时器的操作将会继续完成而不受影响。双缓冲功能时序图如下所示：

## 5.8.2 PWM 定时器结构



## 5.8.2 PWM 定时器结构

定时器的值可以写入定时器计数值缓冲寄存器（TCNTB<sub>n</sub>）中，而当前计数器的值可以通过读定时器计数值观测寄存器（TCNT0<sub>n</sub>）得到。

当 TCNT<sub>n</sub> 的值到 0 时，自动加载操作复制 TCNTB<sub>n</sub> 的值到 TCNT<sub>n</sub> 中。但是如果自动加载模式没有使能，TCNT<sub>n</sub> 将不进行任何操作。

## 5.8.2 PWM 定时器结构

(4) 用手动更新位和反相器位对定时器进行初始化

当递减计数器的值 到0时，自动加载操作才能进行。所以，用户必须预先对 TCNTn 定时一个起始值。因此，起始值必须由手动更新位载入。以下步骤描述了怎么起始一个定时器。

- ①将初始值写入到 TCNTBn 和 TCMPBn中；
- ②设置相应定时器的手动更新位。推荐同时配置反相器位开关位（不管反相器用与否）；
- ③设置相应定时器的起始位，从而启动一个定时器（同时清除手动更新位）。



## 5.8.2 PWM 定时器结构

如果定时器被迫停止，TCNT<sub>n</sub>将保留计数器的值且不重载TCNTB<sub>n</sub>。如果用户需要设置一个新值，必须执行手动更新。

需要注意的是，无论何时 TOUT 反相器开关位的值改变。TOUT<sub>n</sub> 的逻辑值将随之改变。因此，推荐反相器开关位的配置与手动更新位同时进行。

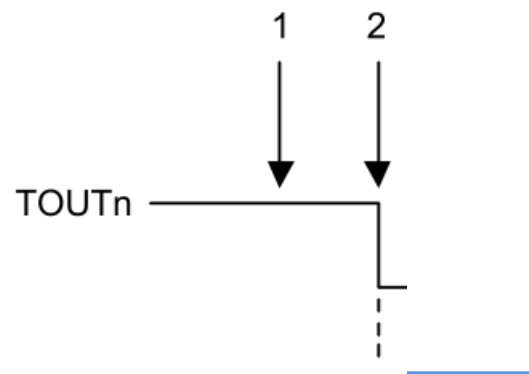
## 5.8.2 PWM 定时器结构

### (5) 定时器操作步骤举例

#### ①使能自动加载功能

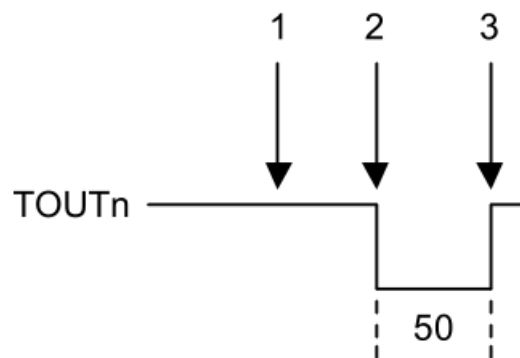
设置  $TCNTB_n$  为 160,  $TCMPB_n$  为 110。  
设置手动更新位并配置反相器位（关）。手动更新位设置  $TCNT_n$  和  $TCMP_n$  的值与  $TCNTB_n$  和  $TCMPB_n$  相同。手动更新位清0。然后设置  $TCNTB_n$  和  $TCMPB_n$  的值分别为 80 和 40, 确定下一个周期的值;

②自动加载 开, 设置起始位, 在定时器的延迟时间后定时器开始递减计数;



## 5.8.2 PWM 定时器结构

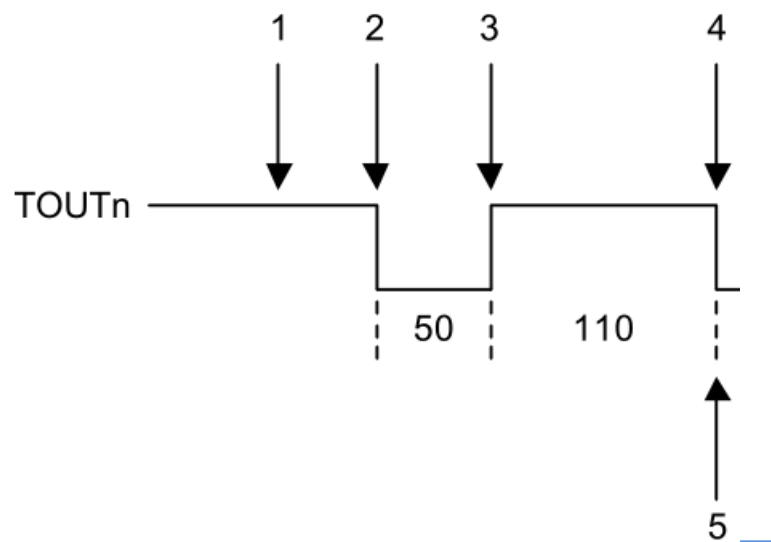
③当  $TCNT_n$  的值和  $TCMP_n$  相等时，则  $TOUT_n$  的逻辑电平将发生改变，由低到高；



## 5.8.2 PWM 定时器结构

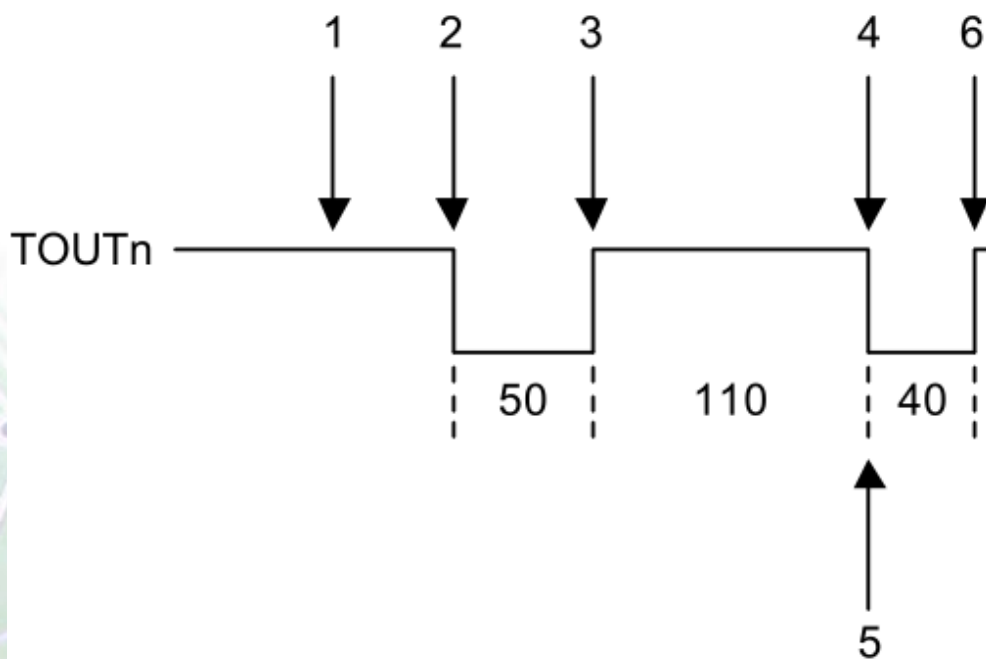
④当  $TCNTn$  的值到0时，产生一个中断并且将  $TCNTBn$  的值加载到一个临时寄存器。在下一个时钟周期， $TCNTBn$  由临时寄存器加载到  $TCNTn$  中；

⑤在中断服务程序中， $TCNTBn$  和  $TCMPBn$  分别设置成 80 和 60；



## 5.8.2 PWM 定时器结构

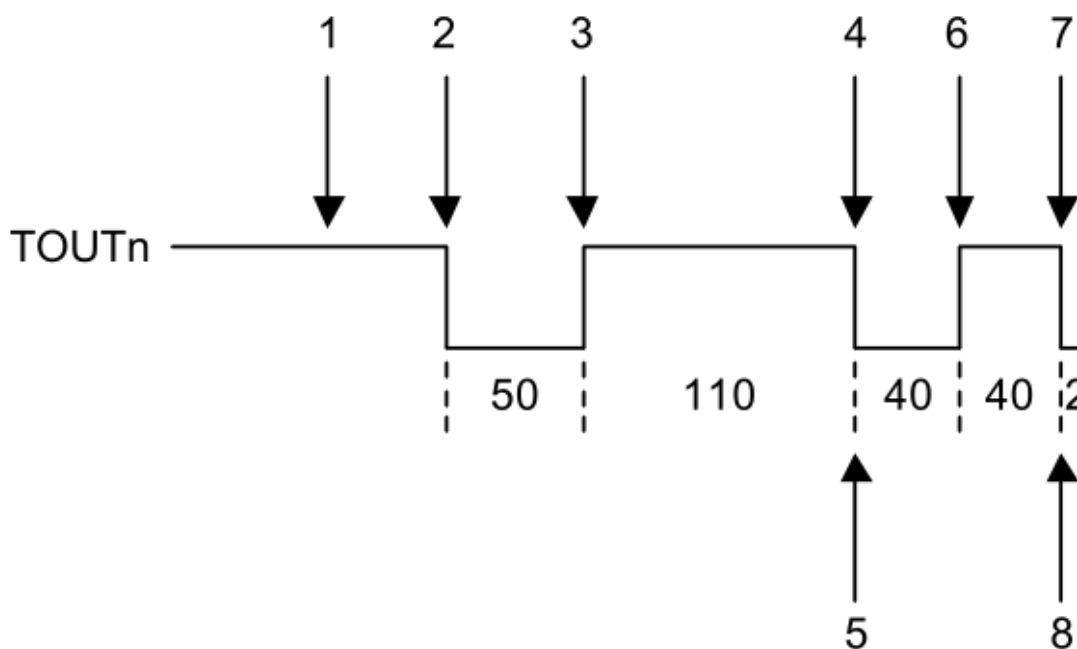
⑥当  $TCNT_n$  的值和  $TCMP_n$  (40) 相等时, 则  $TOUT_n$  的逻辑电平发生改变, 由低到高;



## 5.8.2 PWM 定时器结构

⑦当  $TCNT_n$  到0时， $TCNT_n$  自动重新加载，并发出一个中断请求；

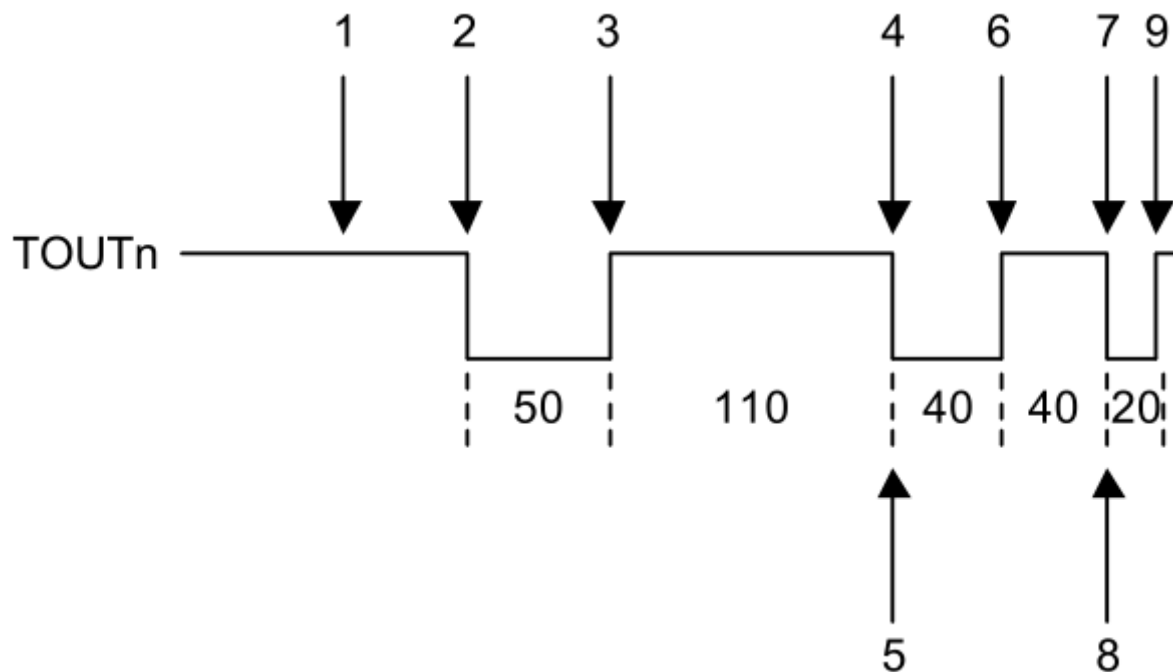
⑧在中断服务子程序中，自动加载和中断请求都被禁止，从而将停止定时器；





## 5.8.2 PWM 定时器结构

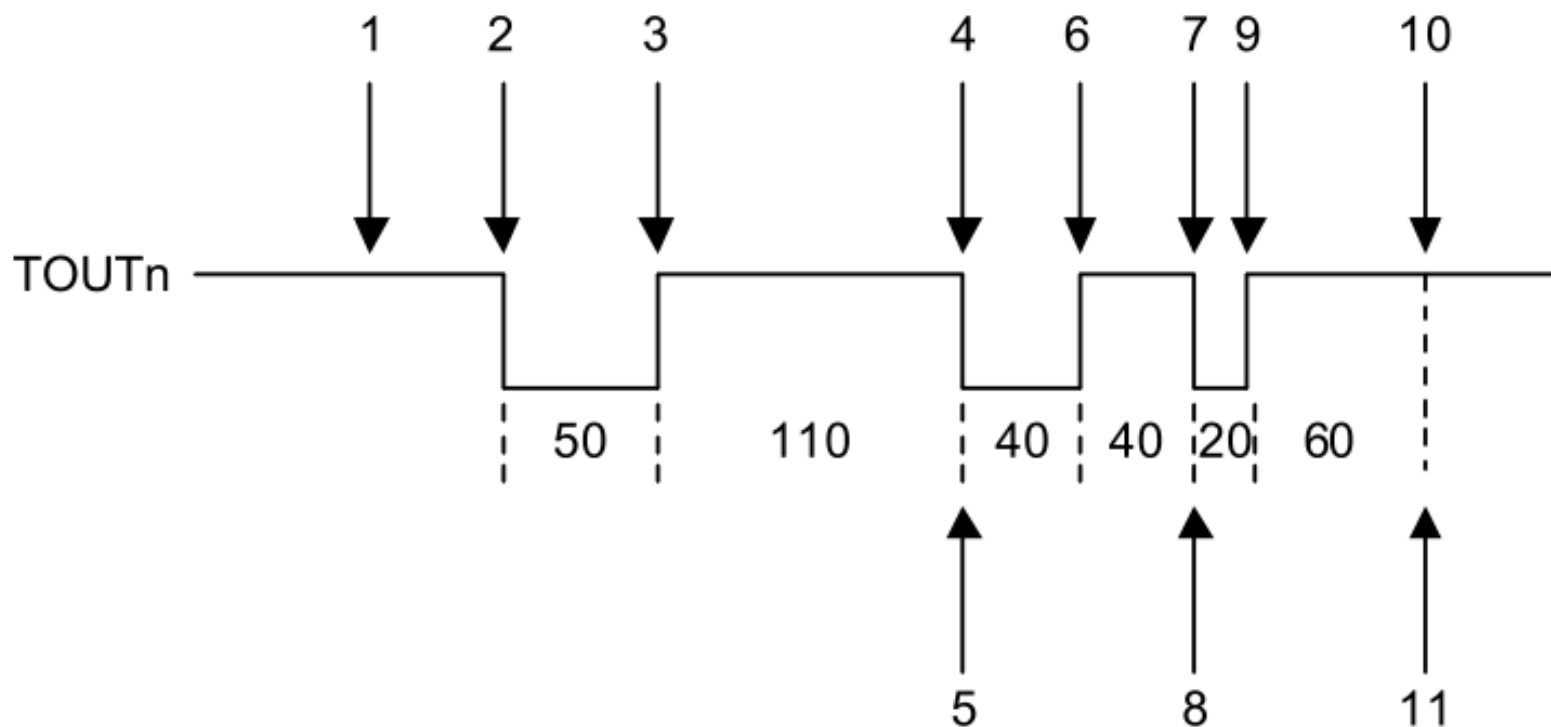
⑨当  $TCNT_n$  的值和  $TCMP_n$  相等时，则  $TOUT_n$  的逻辑电平将发生改变，由低到高；



## 5.8.2 PWM 定时器结构

⑩当  $TCNT_n$  的值为0时,  $TCNT_n$  将不再重新加载新的值, 从而定时器停止;

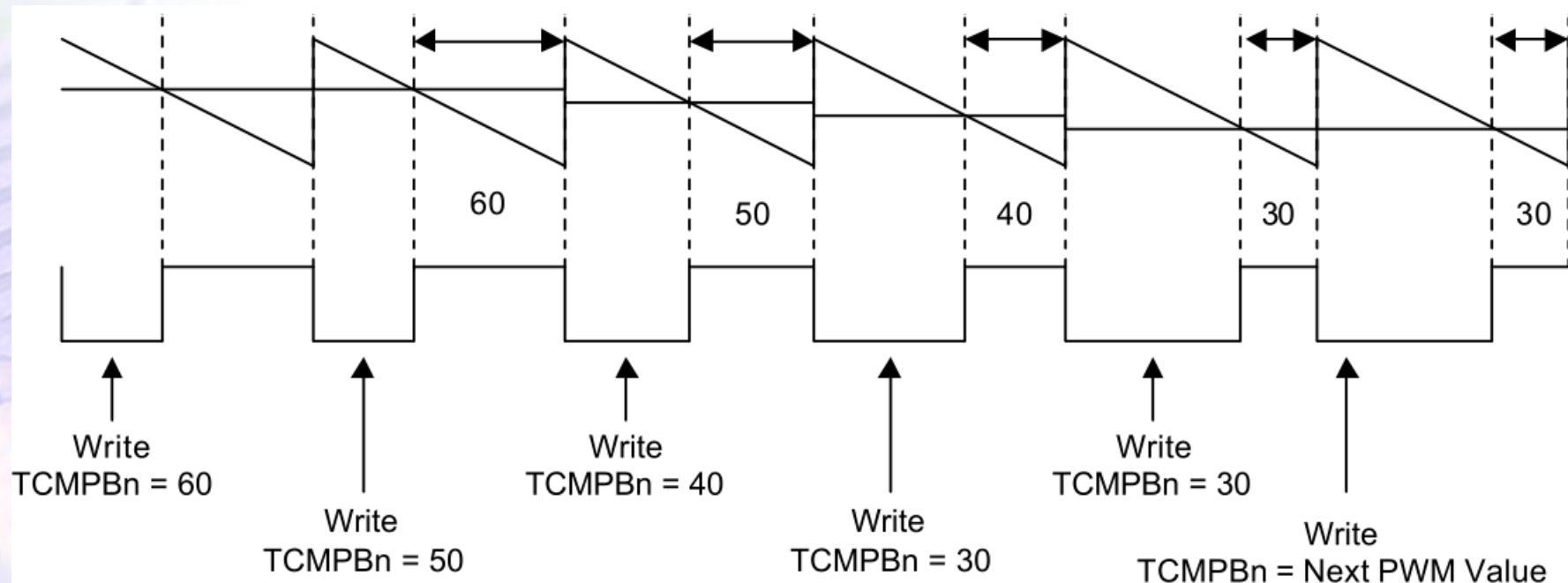
⑪由于中断请求被禁止, 不再产生中断请求。



## 5.8.2 PWM 定时器结构

### (6) 脉宽调制

脉宽调制功能可以通过改变 **TCMPBn** 的值实现。**PWM** 的频率由 **TCNTBn** 决定。下图显示一个通过改变 **TCMPBn** 的值实现 **PWM** 的例子：



## 5.8.2 PWM 定时器结构

如果想得到一个 高的PWM值，则要 增加TCMPBn的值。相反，如果想要得到一个 低的PWM值，则要 减少TCMPBn的值。如果反相器使能，则情况正好相反。

由于定时器具有双缓冲功能，则在当前周期的任何时间都可以通过 **ISR** 和其它程序改变 **TCMPBn** 的值。

## 5.8.2 PWM 定时器结构

### (7) 输出电平控制

以下步骤描述了如何在反相器关闭的情况下，控制 TOUT 的值的高或低。

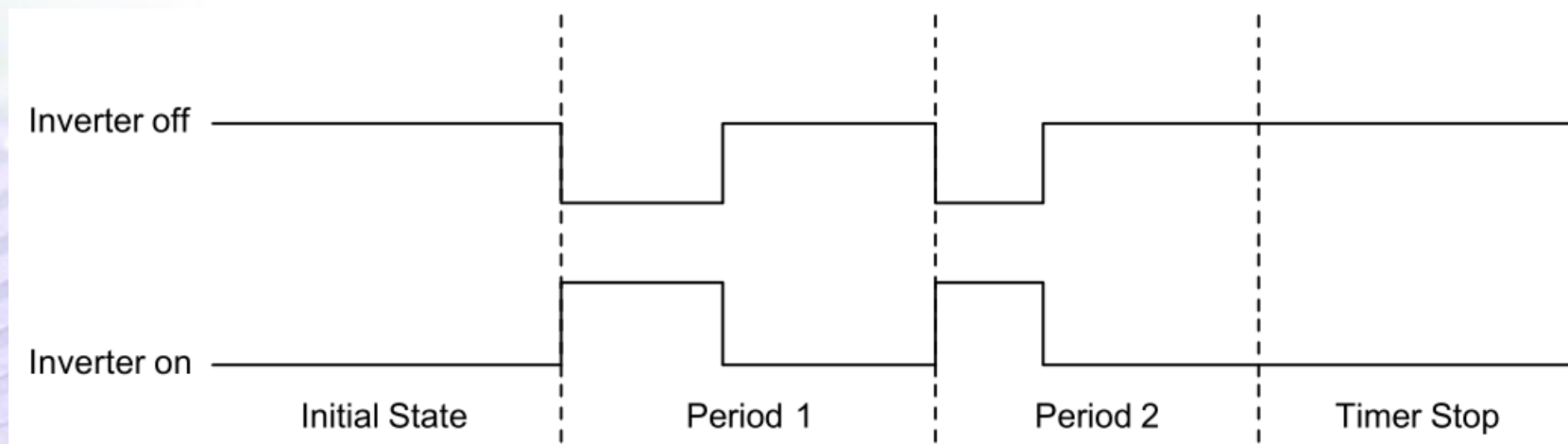
①关闭自动加载位。然后，TOUT 变高且在 TCNTn 为 0 后定时器停止运行；

②通过定时器开始位清零来停止定时器运行。如果  $TCNTn \leq TCMPn$ ，则输出高；如果  $TCNTn > TCMPn$ ，输出为低；

## 5.8.2 PWM 定时器结构

③通过改变 TCON 中的反相器开关位来使 TOUT<sub>n</sub> 为高或低。

下图为反相器 开与关 时的输出：





## 5.8.2 PWM 定时器结构

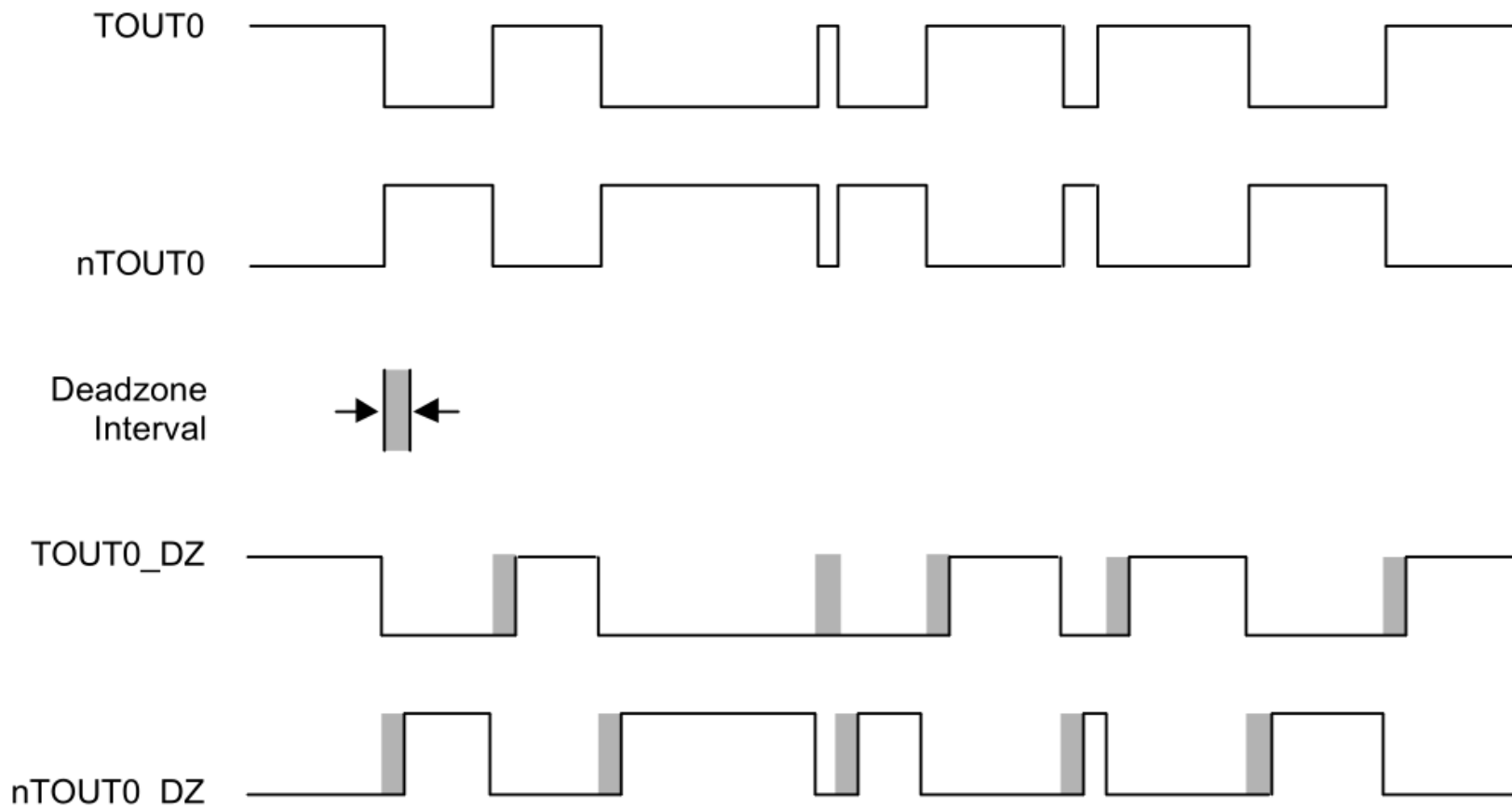
### (8) 死区发生器

死区是为了功率器件中的 **PWM控制**。这一功能 **使能** 在一个关键器件**关闭**和另一个开关器件**开启**的 **间隔时间**。这一时间间隔 **禁止** 了两个开关器件同时处于开启状态，即使是一段非常短的时间内。

**TOUT0 是一个 PWM输出。nTOUT0 是 TOUT0 的反向。如果死区使能，则 TOUT0 和 nTOUT0 的输出波形将是 TOUT0\_DZ 和 nTOUT0\_DZ 。 nTOUT0\_DZ 由 TOUT1 脚输出。**

## 5.8.2 PWM 定时器结构

在死区间隔，TOUT0\_DZ 和 nTOUT0\_DZ 将不会同时开启。下图为死区使能后的输出波形：



## 5.8.2 PWM 定时器结构

### (9) DMA请求模式

**PWM 定时器**能在每个特定时间间隔产生一个**DMA请求**。定时器保持 **DMA请求信号** (**nDMA\_REQ**) 为低 直到 **定时器接收到ACK信号**。当定时器接收到 **ACK信号**时，定时器将使请求信号无效。产生DMA请求的 **定时器由设置DMA模式位 (TCFG1)** 决定。如果一个定时器配置成 **DMA请求模式**，则此定时器将不能产生中断请求，而其他定时器将正常产生中断请求。

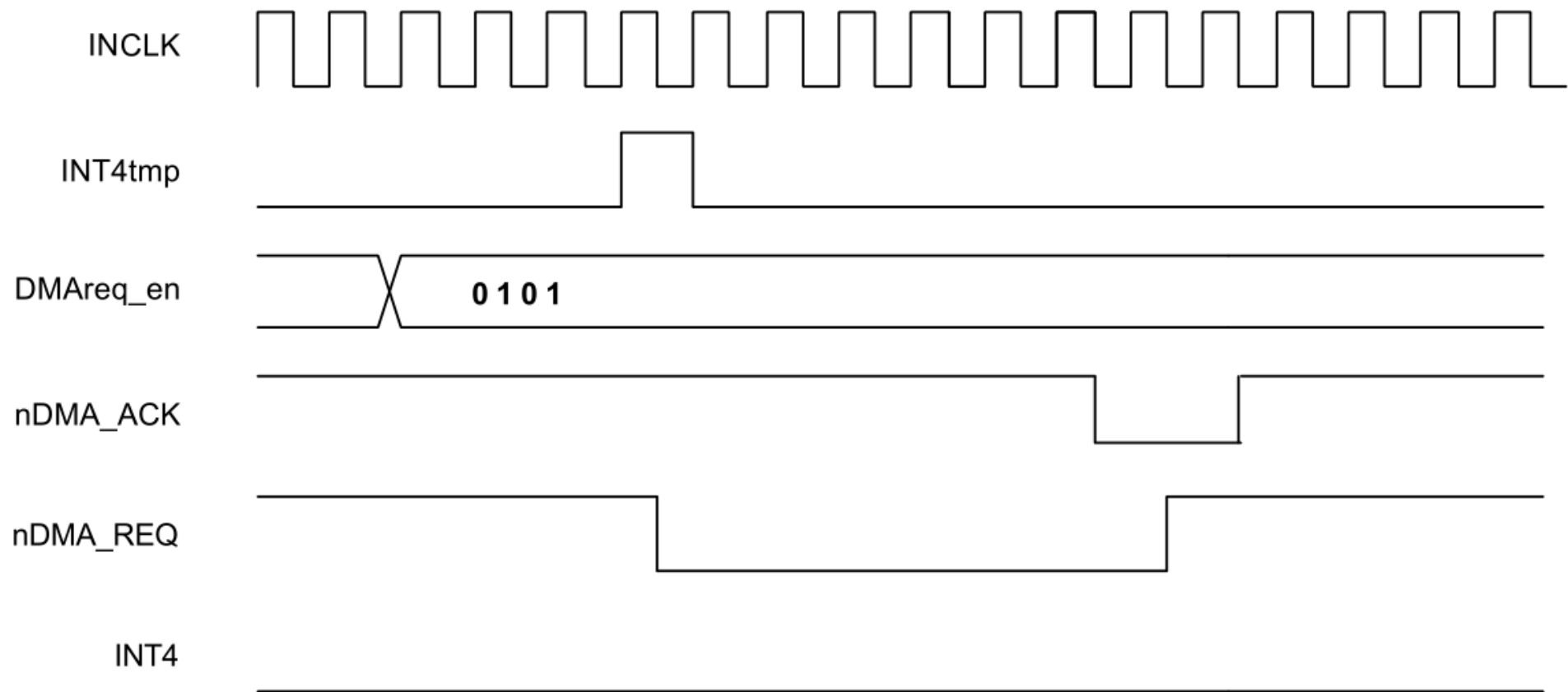
## 5.8.2 PWM 定时器结构

**DMA模式 配置如下所示：**

| DMA模式 | DMA请求     | Timer0<br>INT | Timer1<br>INT | Timer2<br>INT | Timer3<br>INT | Timer4<br>INT |
|-------|-----------|---------------|---------------|---------------|---------------|---------------|
| 0000  | No select | ON            | ON            | ON            | ON            | ON            |
| 0001  | Timer 0   | OFF           | ON            | ON            | ON            | ON            |
| 0010  | Timer 1   | ON            | OFF           | ON            | ON            | ON            |
| 0011  | Timer 2   | ON            | ON            | OFF           | ON            | ON            |
| 0100  | Timer 3   | ON            | ON            | ON            | OFF           | ON            |
| 0101  | Timer 4   | ON            | ON            | ON            | ON            | OFF           |
| 0110  | No select | ON            | ON            | ON            | ON            | ON            |

## 5.8.2 PWM 定时器结构

定时器4 的DMA模式操作如下图所示：



## 5.9 人机交互设备

人机交互设备主要分为 两类：**输入设备**和 **输出设备**。

### 1、输入设备

**键盘、鼠标、手写板、触摸屏等。**

### 2、输出设备

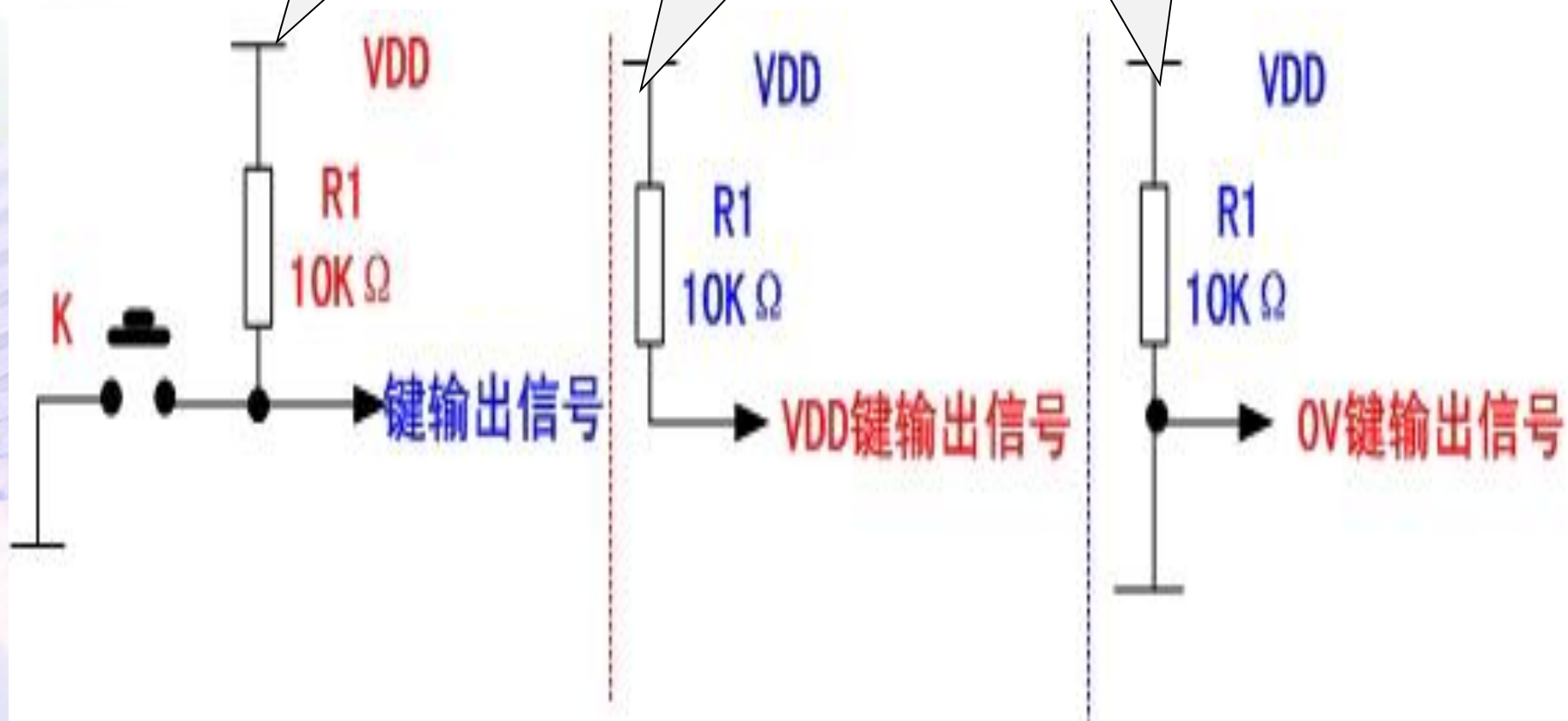
**显示器、打印机、绘图仪等。**



## 5.9.1 键盘

单个按键 未按键等路 已按键等效电路

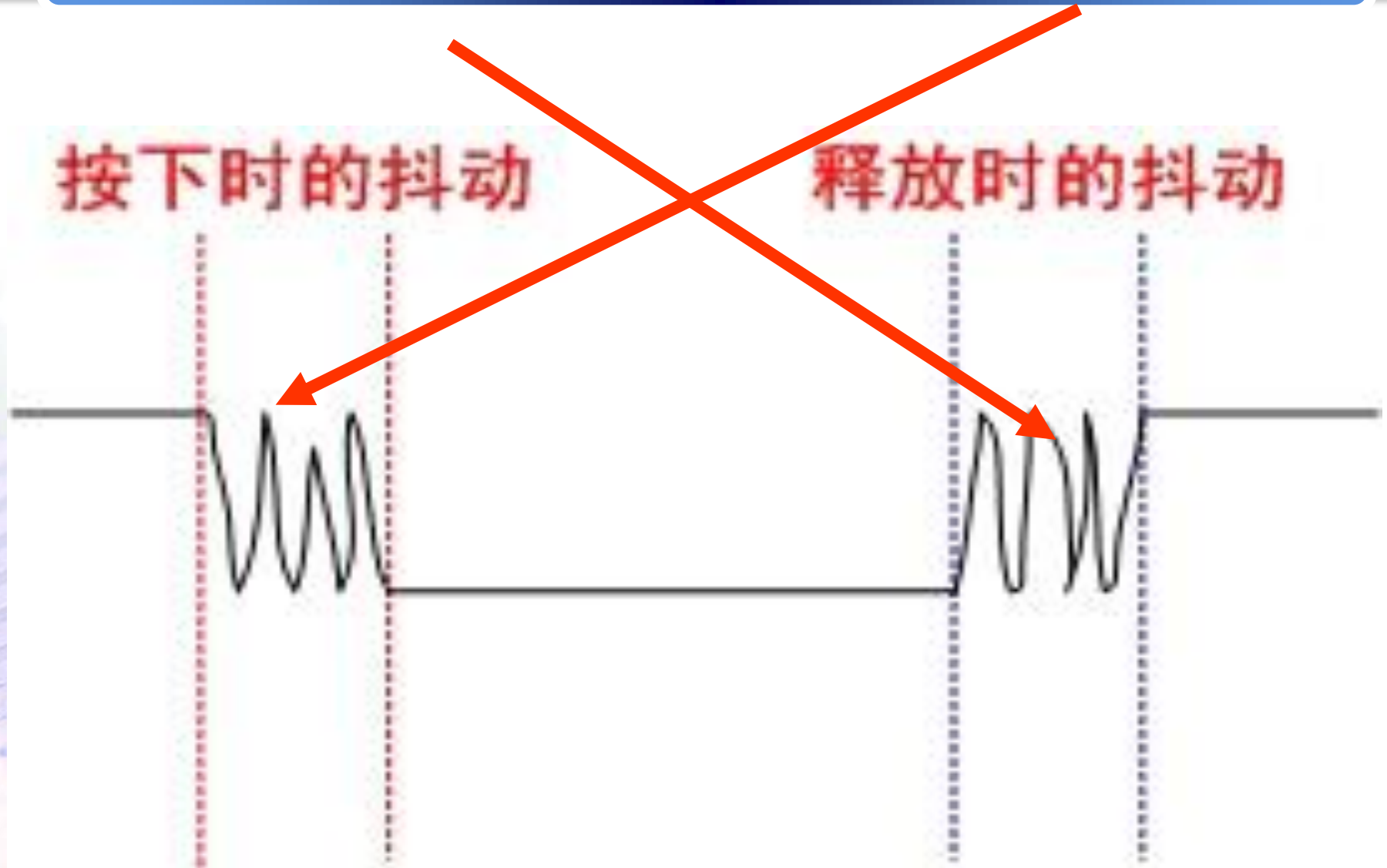
### 1、按键电路



## 5.9.1 键盘

这种简单的按键有一个缺点，即按键被按下（或被释放），触点被接通（或断开）的一瞬间，电路有一个持续 5~30ms 的若通若断的抖动阶段，如下图所示：

# 按键的抖动现象图



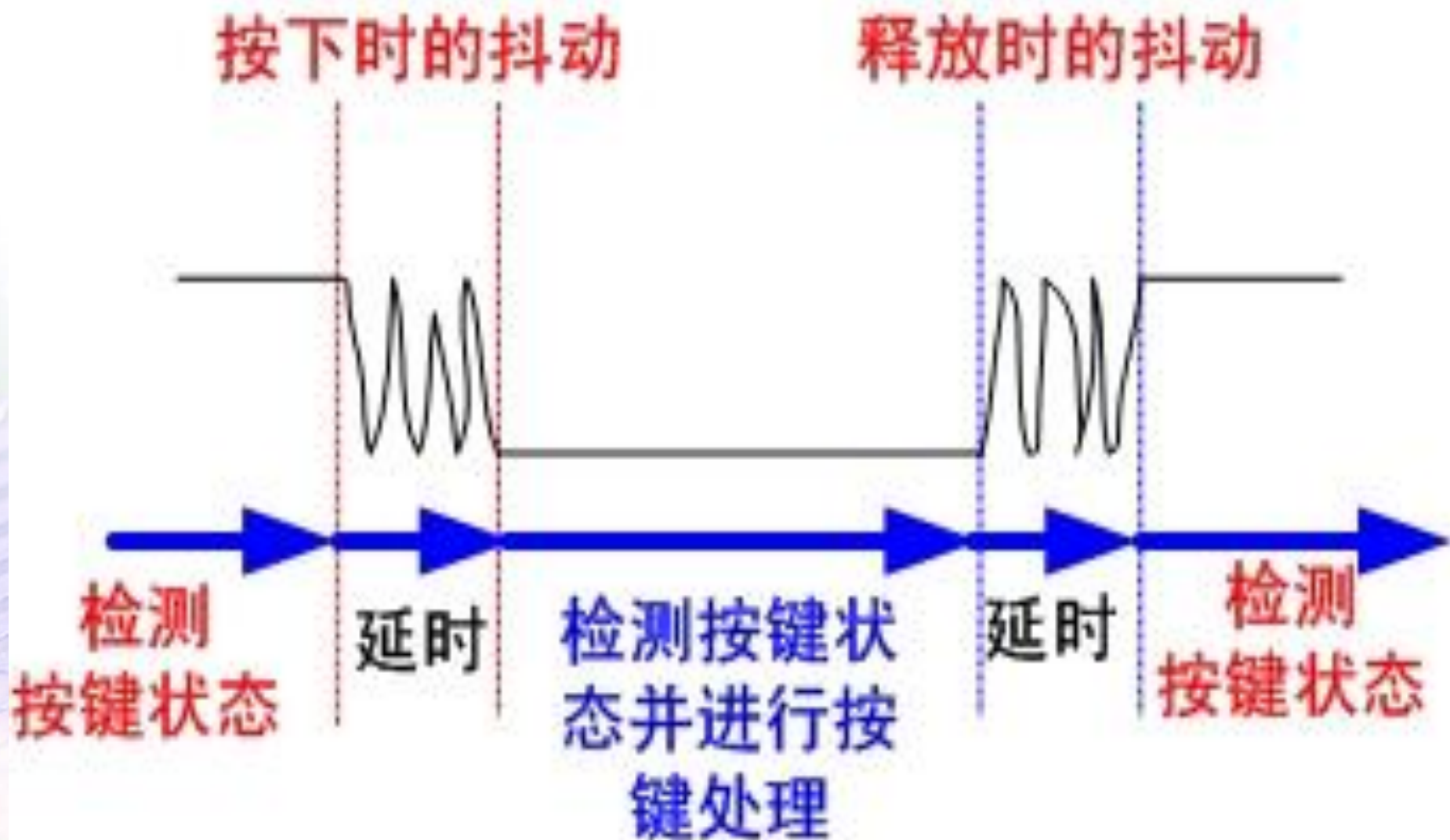
## 5.9.1 键盘

解决抖动问题的办法有 两种：

(1) 一种是使用 硬件 消抖电路（如RC电路、专用芯片）滤掉抖动波形；

(2) 另一种是当发现键盘输出电平 有变化 时，通过 延时 的方法躲过按键的抖动，待电路状态稳定之后再检测按键的输出电平，从而达到正确确定按键信息的目的。

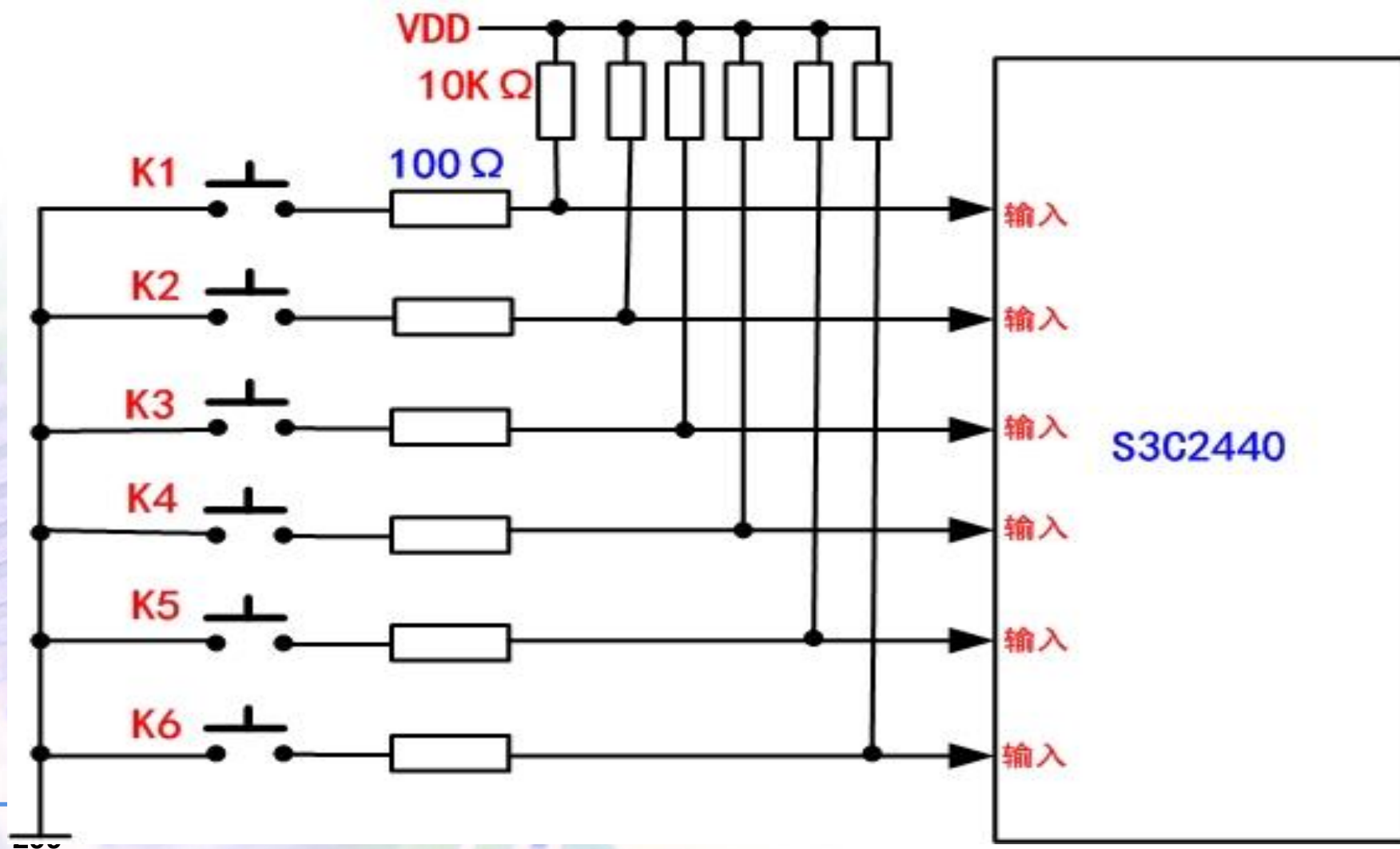
# 用软件进行消抖的流程如下所示：





## 5.9.1 键盘

### 2、独立按键键盘





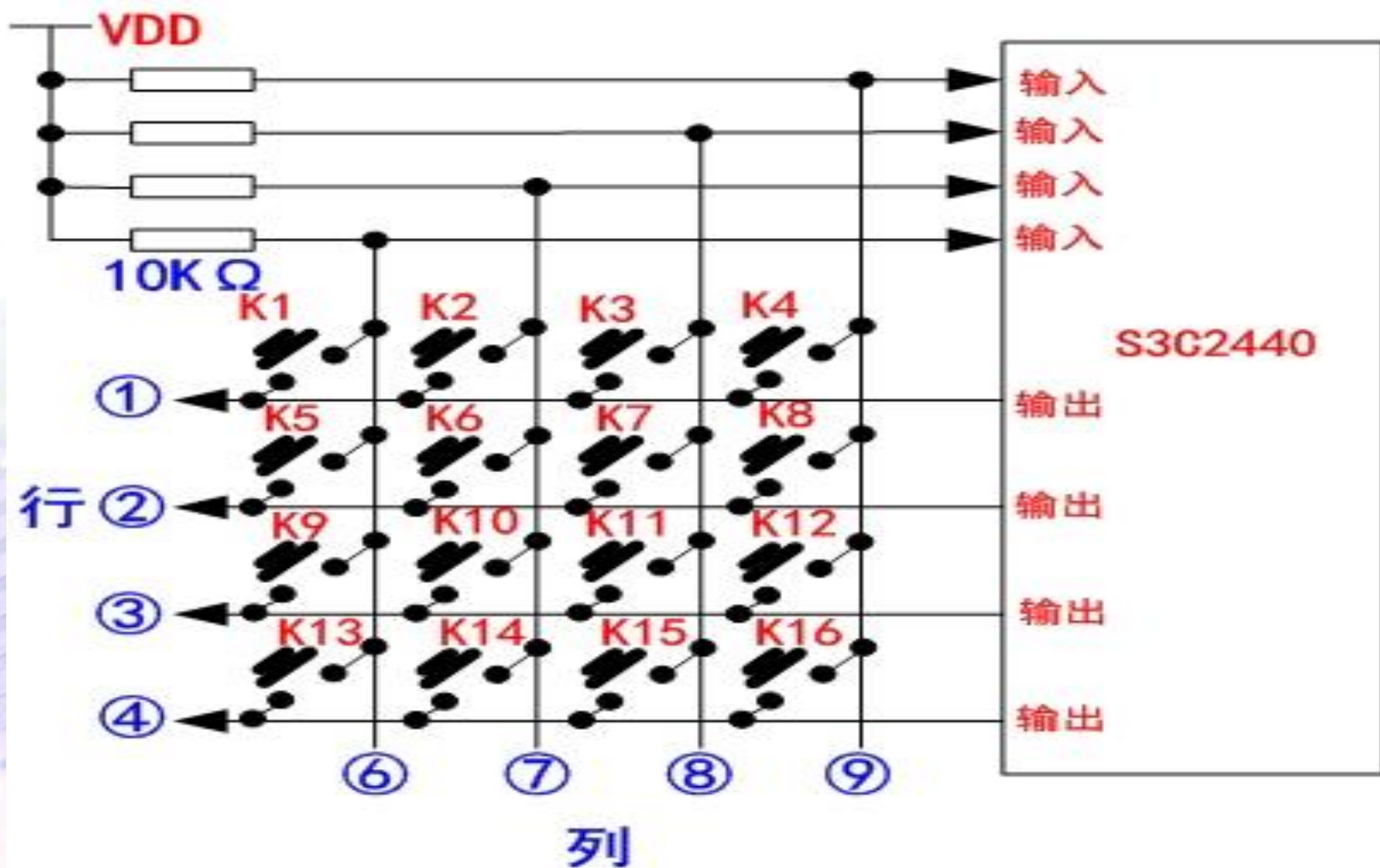
## 5.9.1 键盘

独立按键式键盘也叫做单线键盘，其特点是每一个键都占用一条接口线，所以这种键盘简单可靠，但在键数目较多时，占用接口线也较多。

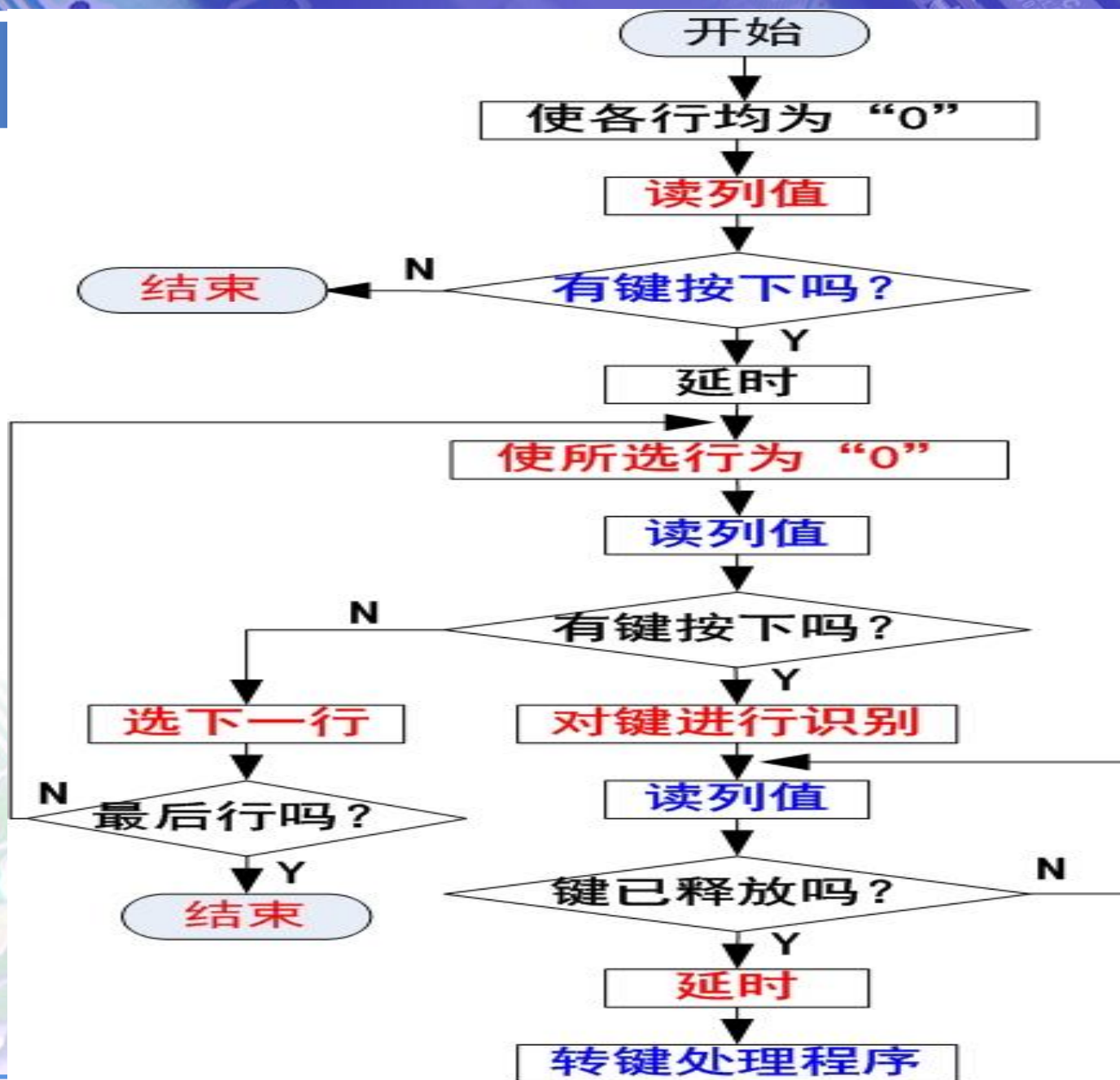
### 3、矩阵式键盘

当键盘的按键数目较大时，为了减少键盘接口线的数目，人们常常采用矩阵式键盘。

# 4 X 4 矩阵式键盘



# 行扫描法的程序流程图:



## 5.10 综合应用

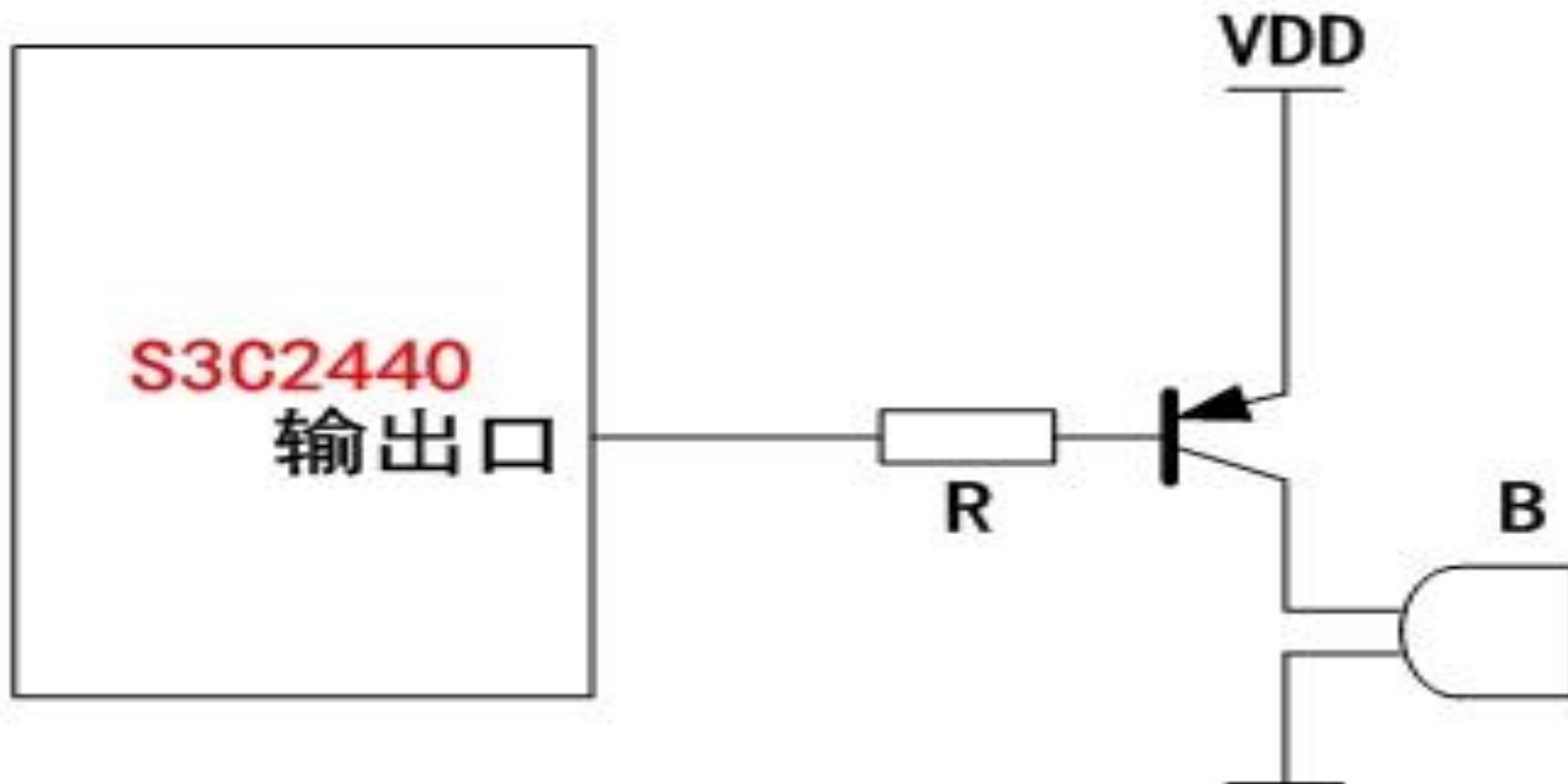
### GPIO 应用实例

#### (1) 蜂鸣器应用

在应用中，为了产生声音报警，常常使用蜂鸣器。下图为 S3C2440 中使用蜂鸣器的电路接线图。

## 5.10 综合应用

### ①蜂鸣器电路图



注：GPB[0]/TOUT0，J6引脚。



## 5.10 综合应用

### ②涉及寄存器及操作

端口控制寄存器GPBCON (0x56000010) ;

端口数据寄存器GPBDAT (0x56000014) ;

GPBCON (0x56000010)=0XXXXXXXX1;

GPBDAT (0x56000014)=0XXXXXXXXX0~  
0XXXXXXXXX1。



## 5.10 综合应用

### ③用 ARM 的汇编语言实现的程序

```
AREA Larm, CODE, READONLY ; 只读的代码段
ENTRY                      ; 程序入口点

start LDR R0, =0x56000010
      LDR R1, =0x00000001
      STR R1, [R0]
      LDR R0, = 0x56000014
LOOP  LDR R1, =0XFFFFFFFF0
      STR R1, [R0]
      BL delay
      LDR R1, =0XFFFFFFFF1
      STR R1, [R0]
      BL delay
      B LOOP
```

## 5.10 综合应用

### ③用ARM的汇编语言实现的程序（续）

**delay**     **LDR R2, =0X0000FFFF**

**delay1**    **MOV R3, R3**

**SUBS R2, R2, #1**

**BNE delay1**

**MOV PC, LR**

**END**

**; 段结束**

## 5.10 综合应用

### ④用C语言实现的程序

```
#define GPBCON (*(volatile unsigned int *)0x56000010))  
#define GPBDAT (*(volatile unsigned int *)0x56000014))
```

```
/******延时程序*****
```

```
void DelayNS(int dly)
```

```
{
```

```
    int i;
```

```
    for (; dly>0; dly --)
```

```
        for (i=0; i<5000;i++);
```

```
}
```

## 5.10 综合应用

### ④用C语言实现的程序（续）

```
/** 蜂鸣程序 **/  
int main(void)  
{  
    GPBCON = 0xFFFFFFFF1; //输出  
    while (1)  
    {  
        GPBDAT = 0xFFFFFFFF0;  
        DelayNS(10);  
        GPBDAT = 0xFFFFFFFF1;  
        DelayNS(10);  
    }  
    return(0);  
}
```